

CÃ“DIGO ASESINO

El Enigma de Qhapaq Ã‘an

Una Saga de Misterio, TecnologÃ­a y Memoria Ancestral

Autor: Alex Goyzueta Delgado

Contacto: alexgoyzueta2018@gmail.com

"En los nudos del quipu se esconde la verdad.

En las lÃ­neas del cÃ³digo, la justicia."

Neo-Cusco, 2026

CrÃ©ditos

CÃ³digo Asesino: El Enigma de Qhapaq Ã‘an

Ã© 2026 Alex Goyzueta Delgado

Todos los derechos reservados. Ninguna parte de esta publicaciÃ³n puede ser reproducida, distribuida o transmitida en forma alguna ni por ningÃºn medio electrÃ³nico, mecÃ¡nico, fotocopia, grabaciÃ³n u otro mÃ©todo, sin el permiso previo por escrito del autor.

EdiciÃ³n y correcciÃ³n: Alex Goyzueta Delgado

Ilustraciones conceptuales: Generadas mediante descripciones textuales

DiseÃ±o de portada: Alex Goyzueta Delgado

TipografÃ­a: IBM Plex Mono (cÃ³digo), Lato (narrativa)

Agradecimientos especiales:

A los guardianes del conocimiento ancestral andino.

A la comunidad Python de LatinoamÃ©rica.

A todos los que creen que la tecnologÃ­a y la tradiciÃ³n pueden caminar juntas.

Datos de catalogaciÃ³n:

Goyzueta Delgado, Alex

CÃ³digo Asesino: El Enigma de Qhapaq Ã‘an / Alex Goyzueta Delgado

1ra ediciÃ³n â€” Barcelona, 2026

Dedicatoria

A mis abuelos, que tejieron historias en quipus de lana y seda,
mucho antes de que existieran los bits y los bytes.

A los programadores anÃ³nimos de los Andes,
que escriben cÃ³digo con la misma paciencia
con la que sus ancestros tallaban piedra.

A Wayra, que vive en cada uno de nosotros:

esa chispa que se niega a apagarse
cuando el mundo dice que no podemos.

"Ama suwa, ama llulla, ama qilla"

(No robes, no mientas, no seas perezoso)

Prefacio

Â¿Por quÃ© este libro?

Has visto el tÃ­tulo. "CÃ¡digo Asesino". Suena a thriller, a misterio, a crimen. Y lo es. Pero tambiÃ©n es un libro para aprender Python. Â¿CÃ³mo se juntan ambas cosas?

Este libro naciÃ³ de una convicciÃ³n: la mejor manera de aprender a programar es resolviendo un misterio.

No te voy a engaÃ±ar con ejercicios aburridos de "calcula el Ã¡rea de un triÃ¡ngulo". AquÃ­ vas a analizar escenas del crimen, decodificar mensajes ocultos en quipus digitales, interrogar sospechosos con algoritmos, y descubrir al asesino paso a paso... mientras aprendes Python.

Â¿CÃ³mo usar este libro?

Cada capÃ­tulo tiene tres partes:

1. Narrativa â€” La historia avanza. Wayra Condori, nuestra protagonista, descubre pistas, enfrenta peligros y se acerca a la verdad.
2. CÃ¡digo â€” Los conceptos de Python que Wayra usa para resolver los enigmas. CÃ¡digo completo, funcional, listo para ejecutar.
3. Enigmas â€” Ejercicios para que tÃº mismo resuelvas usando lo aprendido.

Â¿QuÃ© necesitas?

- Python 3.10+ instalado
- Un editor de cÃ¡digo (VS Code recomendado)
- Curiosidad y ganas de resolver un asesinato

Un viaje, no un destino

Al terminar este libro, no solo sabrÃ¡is quiÃ©n matÃ³ al Dr. Inti Quispe. TambiÃ©n habrÃ¡is aprendido Python desde variables hasta programaciÃ³n orientada a objetos, pasando por estructuras de datos, archivos, decoradores y mÃ¡is.

Y quizÃ¡is, solo quizÃ¡is, descubras que la tecnologÃ­a y la sabidurÃ­a ancestral no estÃ¡n reÃ±idas. Que un quipu y una lista de Python guardan mÃ¡is similitudes de las que imaginas.

Ãy (gracias) por embarcarte en este viaje.

â€” Alex Goyzueta Delgado

Barcelona, 2026

IntroducciÃ³n

Neo-Cusco, 2026

Imagina una ciudad donde los rascacielos de vidrio y acero se elevan junto a muros incas de piedra milenaria. Donde los drones sobrevuelan la Plaza de Armas mientras, en el suelo, una mama quema hojas de coca para leer el futuro. Donde la fibra Ã³ptica corre por canales construidos por los antiguos, y los algoritmos de machine learning se entrenan con patrones textiles ancestrales.

Esa es Neo-Cusco.

Una metrÃ³poli futurista construida sobre las ruinas del Imperio Inca. AquÃ­, la tecnologÃ­a y la tradiciÃ³n no siempre conviven en armonÃ­a. A veces chocan. Y cuando chocan, alguien puede morir.

El Proyecto Yachay

El Dr. Inti Quispe, Ãltimo "Quipucamayoc Digital" â€”guardiÃn del conocimiento ancestral digitalâ€”, lideraba el proyecto mÃ¡s ambicioso del siglo: Yachay (sabidurÃa, en quechua). Una inteligencia artificial capaz de decodificar el conocimiento inca perdido durante la conquista.

Pero Yachay no era solo un proyecto acadÃ©mico. Encerraba secretos que podÃan cambiar el equilibrio de poder en SudamÃ©rica. Secretos por los que vale la pena matar.

Quipus Digitales

Los incas no tenÃan escritura como la conocemos. Usaban quipus: conjuntos de cuerdas con nudos que registraban nÃºmeros, historias y conocimiento. Los espaÃ±oles destruyeron la mayorÃa, creyendo que eran objetos paganos.

Pero el Dr. Inti Quispe descubriÃ³ algo mÃ¡s: los quipus no solo guardaban nÃºmeros. Guardaban algoritmos.

En este libro, los "Quipus Digitales" son archivos que combinan cÃ³digo Python con patrones de nudos ancestrales. Una forma de encriptaciÃ³n que solo quienes conocen ambas tradiciones pueden descifrar.

El crimen

Todo comienza una madrugada frÃa en el Templo del Sol (Coricancha). El Dr. Inti Quispe aparece muerto en su laboratorio. La puerta estaba cerrada por dentro. No hay seÃ±ales de forcejeo. Solo un quipus digital en su computadora, con un mensaje cifrado.

La policÃa dice que fue un robo que saliÃ³ mal. La universidad dice que fue un accidente. La prensa dice que fue un ajuste de cuentas.

Wayra Condori, una analista de datos autodidacta de los barrios altos, sabe que todos estÃn equivocados.

Y tÃº la vas a ayudar a descubrir la verdad.

CapÃtulo 1: El Asesinato en el Templo del Sol

Conceptos: Variables, tipos de datos, `print()`, f-strings

Wayra Condori ajustÃ³ los auriculares y dio un sorbo a su cafÃ© de muÃ±a, amargo y humeante. Frente a ella, tres monitores mostraban lÃneas de cÃ³digo en azul y verde. En el cuarto piso de un edificio casi en ruinas en el barrio de San Blas, su pequeÃ±o departamento servÃa tambiÃ©n como oficina. Desde la ventana, alcanzaba a ver las luces del centro de Neo-Cusco titilando en la neblina matutina.

Eran las 6:47 a.m. cuando su telÃ©fono vibrÃ³.

El mensaje era de su amigo RaÃºl, periodista de investigaciÃ³n en El Sol del Cusco:

Wayra, Ãviste las noticias? Mataron al Dr. Inti Quispe.
En su laboratorio. En el Templo del Sol.
La policÃa dice que fue un robo.
Yo digo que es mentira.
Ven. Te necesito.

Wayra dejÃ³ el cafÃ©. ConocÃa a Inti Quispe de nombre â€”todo el mundo tecnolÃ³gico de Neo-Cusco lo conocÃaâ€” pero tambiÃ©n lo conocÃa de una manera mÃ¡s personal. Su abuela, Mama Teodora, habÃa tejido con la hermana de Inti en las comunidades altas. Y el Dr. Quispe habÃa sido el Ãºnico acadÃ©mico que habÃa tratado a su abuela como una igual, no como una "informante".

Se puso la chamarra de lana de alpaca y saliÃ³. El caso ya la tenÃa atrapada.

La escena del crimen

El laboratorio de Inti Quispe estaba en el ala este del Coricancha, el antiguo Templo del Sol. Pero no era un templo cualquiera: la estructura original inca servía ahora como base para un centro de investigación de última generación. Muros de piedra perfectamente ensamblada sostenían paredes de vidrio inteligente. El suelo de la época imperial había sido cubierto con paneles táctiles.

Wayra llegó y encontró a Raúl esperándola afuera.

“No te voy a mentir” dijo Raúl en voz baja. “Esto es más grande de lo que parece. Inti estaba trabajando en algo llamado 'Proyecto Yachay'. Una IA que... bueno, nadie sabe exactamente qué hace. Pero ayer recibí un mensaje. Y hoy estoy muerto.

¿Qué mensaje?” preguntó Wayra.

“Eso es lo que necesito que descubras. Vamos.

Entraron. El laboratorio era un desorden controlado: papeles, cables, una computadora central con tres monitores. En el monitor principal, una ventana de terminal mostraba algo que parecía un mensaje.

Raúl tomó una foto con su teléfono y se la envió a Wayra.

“Mira esto. Es lo único que dejaron. Un código.

Wayra miró la foto. En la pantalla se leía:

```
quipu_digital_001 = "A'an:1:3:5:7:9"
print(quipu_digital_001)
```

“Esto no tiene sentido...” murmuró Wayra.

“Por eso te llamo. Dijiste que el código habla, ¿no?”

Wayra sonrió. En efecto, el código habla. Y este código acababa de decirle algo importante.

Python: La primera conversación

Antes de entender el mensaje, Wayra necesitaba establecer los fundamentos. En Python, todo comienza con las variables. Piensa en una variable como una caja donde guardas información. Esa información puede ser de diferentes tipos:

- Números enteros (`int`): como la edad de alguien o la cantidad de pistas
- Números decimales (`float`): como una coordenada o una probabilidad
- Texto (`str`): como un nombre o un mensaje cifrado
- Booleanos (`bool`): `True` o `False`, como una respuesta de sí o no

Y la forma más básica de ver qué contiene una variable es con `print()`.

Wayra abrió su laptop y comenzó a escribir.

```
# =====
# CASO: El asesinato del Dr. Inti Quispe
# Analista: Wayra Condori
# Fecha: 27 de junio, 2026
# =====
# --- DATOS BÁSICOS DEL CASO ---
nombre_victima = "Dr. Inti Quispe"
edad_victima = 67
lugar_crimen = "Templo del Sol (Coricancha)"
hora_descubrimiento = "06:30"
puerta_cerrada = True
tiene_herederos = False
print("=== INFORME INICIAL ===")
print("Víctima:", nombre_victima)
print("Edad:", edad_victima)
print("Lugar:", lugar_crimen)
```

```
print("Hora del descubrimiento:", hora_descubrimiento)
print("Puerta cerrada por dentro:", puerta_cerrada)
```

Al ejecutar, la terminal mostrÃ³:

```
=== INFORME INICIAL ===
VÃ­ctima: Dr. Inti Quispe
Edad: 67
Lugar: Templo del Sol (Coricancha)
Hora del descubrimiento: 06:30
Puerta cerrada por dentro: True
```

Pero habÃ­a una forma mÃ¡s elegante de mostrar esa informaciÃ³n. Python 3.6+ tiene los f-strings (cadenas formateadas), que permiten insertar variables directamente dentro del texto:

```
print(f"La vÃ­ctima es {nombre_victima}, de {edad_victima} aÃ±os.")
print(f"El crimen ocurriÃ³ en {lugar_crimen} a las {hora_descubrimiento}.")
```

Resultado:

```
La vÃ­ctima es Dr. Inti Quispe, de 67 aÃ±os.
El crimen ocurriÃ³ en Templo del Sol (Coricancha) a las 06:30.
```

El primer quipu digital

Wayra volviÃ³ al mensaje que habÃ­a visto en la pantalla:

```
quipu_digital_001 = "Ãan:1:3:5:7:9"
print(quipu_digital_001)
```

âÃQuÃ© significa "Ãan"? âpreguntÃ³ RaÃl.

â"Ãan" significa "camino" o "ruta" en quechua ârespondiÃ³ Wayraâ. Y los nÃºmeros separados por dos puntos... son posiciones. Son nudos en un camino.

âÃNudos?

âLos quipus, RaÃl. Inti estaba dejando un mensaje con el sistema de sus ancestros. Cada nÃºmero representa un nudo en una cuerda. La posiciÃ³n del nudo, el tipo de nudo, la direcciÃ³n... todo tiene significado.

Wayra escribiÃ³ un nuevo script:

```
# Decodificando el quipu digital
quipu_mensaje = "Ãan:1:3:5:7:9"
print(f"Mensaje original del quipu: {quipu_mensaje}")
# Separamos los elementos por los dos puntos
elementos = quipu_mensaje.split(":")
print(f"Elementos separados: {elementos}")
# El primer elemento es el tipo de quipu
tipo_quipu = elementos[0]
print(f"Tipo de camino: {tipo_quipu}")
# Los siguientes son las posiciones de los nudos
nudos = elementos[1:]
print(f"Posiciones de nudos: {nudos}")
```

âEste quipu dice "Camino: 1, 3, 5, 7, 9" âexplicÃ³ Wayraâ. Son nÃºmeros impares. En la numeraciÃ³n inca, los impares representan... preguntas. Un camino de preguntas.

âÃUn camino de preguntas? ÃQuÃ© tipo de preguntas?

Wayra seÃ±alÃ³ la pantalla.

âEso es lo que vamos a descubrir. Pero primero, necesito mÃ¡s datos.

Tu primer enigma

Antes de seguir, tienes que practicar lo que Wayra acaba de aprender.

Enigma 1.1: La ficha del caso

Crea un programa que guarde en variables los siguientes datos del caso y los muestre usando ``print()`` y f-strings:

- Nombre del sospechoso: Lara Mamani
- Edad: 32 años
- Rol: Asistente de laboratorio
- Coartada: "Estaba en mi departamento"
- Tiene acceso al laboratorio: ``True``

El output debe verse así:

```
=== FICHA DE SOSPECHOSO ===
Nombre: Lara Mamani
Edad: 32
Rol: Asistente de laboratorio
Coartada: Estaba en mi departamento
Acceso: True
```

Enigma 1.2: El mensaje cifrado

El Dr. Inti dejó otro mensaje. Este quipu dice:

```
quipu_digital_002 = "Yachay:2:4:6:8:10"
```

Modifica el código de decodificación para trabajar con este nuevo quipu. ¿Qué observas? Los números ya no son impares. En la numeración inca, los pares representan respuestas. ¿Qué podría significar?

Enigma 1.3: Tu propia variable

Crea una variable llamada ``mi_teoría`` que contenga un texto con tu primera teoría sobre el caso. Luego muéstrala con ``print()``. Ejemplo:

```
mi_teoría = "Creo que el asesino conocía a Inti y tenía acceso al laboratorio"
print(f"Mi teoría inicial: {mi_teoría}")
```

[Facilito] ¡Soluciones en el Apéndice al final del libro.

Lo que aprendiste

- Las variables guardan información en la memoria del computador
- Los tipos de datos básicos son: ``int`` (entero), ``float`` (decimal), ``str`` (texto), ``bool`` (booleano)
- ``print()`` muestra información en la pantalla
- Los f-strings (``f"texto {variable}"``) permiten incrustar variables en texto
- Los quipus digitales usan el formato ``"Tipo:num1:num2:num3"`` para codificar mensajes
- ``.split()`` separa un string en una lista usando un delimitador

Wayra guardó el archivo y miró a Raúl.

«Necesito ver el quipu original. El físico. El que Inti tenía en su escritorio.

Raúl asintió y la guió hacia el interior del laboratorio. Pero cuando llegaron al escritorio de Inti, algo los detuvo.

Sobre la madera tallada a mano, junto a una taza de café frío, había un quipus real: cuerdas de lana de colores de varios colores, con nudos en posiciones precisas. Pero no era un quipus histórico. Al lado, una pequeña pantalla OLED mostraba un código que parpadeaba lentamente.

Wayra se inclinó para leerlo y su sangre se heló.

El código decía:

Alguien sabÃa que ella vendrÃa.

Conceptos: Strings, métodos de string, slicing, formateo

El corazón de Wayra latía con fuerza mientras leía el mensaje en la pantalla OLED.

“Raí...” sussurrou. “Tocaste algo aqui?”

“¡No! Te lo juro, no toqué nada. Llegué, vi el cuerpo, llamé a la policía y luego te escribí. No toqué nada.

Wayra observó el quipus físico. Tenía cuerdas de cinco colores diferentes: rojo, verde, azul, amarillo y blanco. Cada cuerda tenía nudos en posiciones específicas. Pero lo más intrigante era el código integrado en la base del quipus: una pequeña tira de fibra óptica tejida entre las cuerdas, conectada a la pantalla OLED.

“Esto no es un quipus histórico” dijo Wayra. Es un quipus digital. Inti fusiona la tecnología textil ancestral con componentes electrónicos. Las cuerdas son los cables, los nudos son los datos.

Tomó una foto del quipus con su teléfono y comenzó a transcribir los colores y posiciones de los nudos en su laptop.

“Voy a necesitar decodificar esto. Cada color representa un tipo de informaci3n diferente. Y los nudos... los nudos son como caracteres en un string.

La magia de los strings

Los strings son probablemente el tipo de dato más versátil en Python. Un string es simplemente una secuencia de caracteres: letras, números, símbolos, espacios.

Pero un string tiene "magia" oculta. Al igual que un quipus tiene nudos en posiciones específicas que dan significado a la cuerda, un string tiene caracteres en posiciones que dan significado al texto.

Wayra abrió³ un nuevo archivo:

```
# =====
# CASO: El quipus de Inti - DecodificaciÃ³n
# =====

# El quipus tiene cuerdas de 5 colores
# Representaremos cada cuerda como un string
# donde 'n' = nudo y '-' = espacio vacÃo

cuerda_roja = "n-n---n----n"
cuerda_verde = "-n-n---n----n"
cuerda_azul = "n---n---n---n"
cuerda_amarilla = "--n----n----n-"
cuerda_blanca = "n-n-n-n-n-n-n"

print("=== QUIPUS FÃSICO: TRANSCRIPCIÃN ===")
print(f"Cuerda Roja: {cuerda_roja}")
print(f"Cuerda Verde: {cuerda_verde}")
print(f"Cuerda Azul: {cuerda_azul}")
print(f"Cuerda Amarilla: {cuerda_amarilla}")
print(f"Cuerda Blanca: {cuerda_blanca}")
```

Pero esto solo mostraba los patrones. Wayra necesitaba entender el mensaje. Para eso, necesitaba herramientas más poderosas.

Slicing: Cortando el quipu

En Python, puedes acceder a cualquier carácter de un string usando `[]` con un índice. Los índices empiezan en 0. Así como en un quipus, el primer nudo está en la "posición 0" de la cuerda.

```
# Accediendo a posiciones específicas en la cuerda roja
print("\n--- Analizando la cuerda roja ---")
print(f"Posición 0: '{cuerda_roja[0]}'") # 'n'
print(f"Posición 1: '{cuerda_roja[1]}'") # '-'
print(f"Posición 2: '{cuerda_roja[2]}'") # '-'
print(f"Posición 3: '{cuerda_roja[3]}'") # 'n'
# Slicing: tomando porciones del string
# [inicio:fin] - el fin NO se incluye
primeros_4 = cuerda_roja[0:4] # "n--n" (posiciones 0, 1, 2, 3)
print(f"\nPrimeros 4 caracteres: '{primeros_4}'")
# También podemos usar índices negativos
# -1 es el último carácter
ultimo = cuerda_roja[-1]
print(f"Último carácter: '{ultimo}'")
# [inicio:fin:paso]
cada_2 = cuerda_roja[0:12:2]
print(f"Cada 2 posiciones: '{cada_2}'")
```

“Interesante” murmuró Wayra. Si tomamos cada dos posiciones en la cuerda roja, obtenemos "n-n-n-". Tres nudos separados por espacios. Como un mensaje en código.

Pero habíamos. Python ofrece una gran cantidad de métodos de string “funciones que pertenecen al string y permiten manipularlo.

Métodos de string: Las herramientas del tejedor

Wayra recordaba a su abuela Teodora hablando de los quipus. "Cada nudo tiene un propósito", decía. "Nudo simple es un número. Nudo compuesto es una historia. Nudo en forma de 'S' es una advertencia."

Los métodos de string son como esos diferentes tipos de nudos: cada uno hace algo específico.

```
# --- MÉTODOS DE STRING ---
mensaje_encontrado = " EL CODIGO ESTA EN EL QUIPU BLANCO "
# strip() - elimina espacios al inicio y final
limpio = mensaje_encontrado.strip()
print(f"Original: '{mensaje_encontrado}'")
print(f"Sin espacios: '{limpio}'")
# lower() - convierte a minúsculas
minusculas = limpio.lower()
print(f"En minúsculas: '{minusculas}'")
# upper() - convierte a mayúsculas
mayusculas = limpio.upper()
print(f"En mayúsculas: '{mayusculas}'")
# replace() - reemplaza texto
reemplazado = limpio.replace("QUIPU", "CODIGO")
print(f"Reemplazado: '{reemplazado}'")
# count() - cuenta ocurrencias
conteo_n = limpio.count("Q")
print(f"Veces que aparece 'Q': {conteo_n}")
# find() - encuentra posición
posicion = limpio.find("BLANCO")
print(f"El texto 'BLANCO' empieza en posición: {posicion}")
# len() - longitud del string (no es un método, es una función)
longitud = len(limpio)
print(f"Longitud del mensaje: {longitud} caracteres")
```

Pero el mensaje en la pantalla OLED seguía siendo un misterio. ¿Cómo sabía el sistema que ella llegaría? Wayra decidió investigar el código que generaba ese mensaje.

Construyendo mensajes como un quipu

Inti había construido un sistema que, al detectar movimiento en el laboratorio, mostraba un mensaje personalizado. Wayra encontró el archivo fuente en la computadora:

```
# =====
# Sistema de bienvenida del laboratorio
# Dr. Inti Quispe - Proyecto Yachay
# =====
# Nombres de posibles visitantes
visitante_1 = "policia"
visitante_2 = "Raúl"
visitante_3 = "Wayra"
# Mensaje base
saludo = "Bienvenido, {}. Te estaba esperando."
# Formateo con .format()
print(saludo.format(visitante_1))
print(saludo.format(visitante_2))
print(saludo.format(visitante_3))
```

Wayra sintió un escalofrío. No era un mensaje sobrenatural. Inti había programado el sistema para que detectara quién entraba y mostrara un saludo personalizado. Pero la pregunta era: ¿cómo sabía Inti que ella vendría?

Quizás la respuesta estaba en la cuerda blanca. Todos los quipus ceremoniales incas tenían una cuerda blanca que representaba la conexión espiritual, el puente entre el mundo físico y el digital.

Enigmas: Decodifica el mensaje

Wayra necesita tu ayuda para decodificar completamente el quipu.

Enigma 2.1: El mensaje oculto

Dado el siguiente string:

```
mensaje_codificado = "Á‘AN*QHAPAQ*TAMBO*WAYRA*INTI"
```

Usa métodos de string para:

1. Convertir todo a minúsculas
2. Reemplazar `` por ` -> `
3. Encontrar en qué posición aparece "WAYRA"
4. Contar cuántas veces aparece la letra "A"
5. Mostrar la longitud total del mensaje

Enigma 2.2: Extrayendo el nombre

Usando slicing en el string del enigma anterior, extrae solo el nombre "WAYRA". Pista: usa `find()` para encontrar dónde empieza, y luego slicing para extraerlo.

Enigma 2.3: El quipu invertido

Los incas a veces leían los quipus de derecha a izquierda. Dado el siguiente string, inviértelo usando slicing con paso negativo:

```
quipu_invertido = "ATAM-ATAK-AP"
```

Pista: `[::-1]` invierte un string.

Lo que aprendiste

- Los strings son secuencias de caracteres con posiciones (Índices)
- El slicing `[inicio:fin:paso]` permite extraer partes de un string
- Los Índices negativos cuentan desde el final (`-1` es el último)
- Los métodos de string: `.strip()`, `.lower()`, `.upper()`, `.replace()`, `.count()`, `.find()`, `len()` devuelve la longitud de un string

- `.format()` y f-strings son formas de insertar variables en texto

- `[::-1]` invierte un string completamente

Wayra guardÃ³ su progreso y mirÃ³ las cuerdas del quipus frente a ella. Los patrones empezaban a tener sentido. La cuerda blanca, la que representaba la conexión espiritual, tenía nudos intercalados: nudo, espacio, nudo, espacio... como un código binario.

“RaÃ³l dijo”. La cuerda blanca no es decoración. Es la clave. Cada nudo es un 1, cada espacio es un 0. Es un mensaje binario convertido en textil.

RaÃ³l se acercÃ³, incrÃ©dulo.

“¿Estás diciendo que Inti escribiÃ³ código... en un tejido?”

Wayra sonriÃ³.

“No "en" un tejido. El tejido es el código. Bienvenido al mundo de los quipus digitales.

Capítulo 3: La Lista de Sospechosos

Conceptos: Listas, tuplas, indexing, métodos de listas

La policía había establecido un perímetro alrededor del laboratorio, pero eso no detuvo a Wayra. Con la ayuda de RaÃ³l y su credencial de prensa, logrÃ³ acceder a los archivos iniciales del caso.

El oficial a cargo, un hombre de unos 50 años con bigote gris y cara de pocos amigos, les entregÃ³ una tablet con la lista de personas relacionadas al Dr. Inti Quispe.

“Son cinco” dijo el oficial. Todos tenían motivos. Todos tenían acceso. Pero solo uno tuvo la oportunidad.

Wayra tomÃ³ la tablet y leyÃ³:

1. Lara Mamani “Asistente. 32 años. Acceso total al laboratorio. Motivo: Â¿Económico?”
2. Dr. Carlos HuamÃ¡n “Colega. 55 años. Acceso restringido. Motivo: Envidia profesional.
3. Dra. Sarah Chen “Colaboradora. 40 años. Acceso parcial. Motivo: Presión académica.
4. Rodrigo Mamani “Empresario. 60 años. Sin acceso directo. Motivo: Control del proyecto.
5. Mama Killa “Hermana. 70 años. Acceso familiar. Motivo: Â¿Secreto familiar?”

“Son datos, Wayra” dijo RaÃ³l. Tu especialidad. ¿Qué puedes hacer con eso?

Wayra sonriÃ³. Lo primero era estructurar esa información.

Listas: El primer contenedor

En Python, una lista es una colección ordenada de elementos. Piensa en ella como una cuerda de quipu donde cada nudo es un elemento. Los elementos pueden ser de cualquier tipo: strings, números, incluso otras listas.

```
# =====
# LISTA DE SOSPECHOSOS
# =====
# Creamos una lista con los nombres de los sospechosos
sospechosos = [
    "Lara Mamani",
    "Dr. Carlos Huamán",
    "Dra. Sarah Chen",
    "Rodrigo Mamani",
    "Mama Killa"
]
print("=== SOSPECHOSOS DEL CASO ===")
```

```
print(sospechosos)
print(f"Cantidad de sospechosos: {len(sospechosos)}")
```

“Pero solo tener nombres no basta” dijo Wayra mientras tecleaba. Necesitamos más información.

Accediendo a elementos

Al igual que con los strings, podemos acceder a elementos de una lista usando Índices:

```
# El primer sospechoso (Índice 0)
print(f"Primer sospechoso: {sospechosos[0]}")
# El Último sospechoso (Índice -1)
print(f"Último sospechoso: {sospechosos[-1]}")
# Los tres primeros (slicing)
print(f"Tres primeros: {sospechosos[0:3]}")
# Los dos Últimos
print(f"Dos Últimos: {sospechosos[-2:]}")
```

Listas de listas: El quipu multidimensional

Los quipus verdaderos no tenían una sola cuerda: tenían cuerdas principales, cuerdas secundarias, y cuerdas que colgaban de otras cuerdas. Wayra decidió modelar esa estructura con listas anidadas:

```
# Cada sospechoso tiene múltiples datos
# [nombre, edad, rol, acceso_al_laboratorio, nivel_de_sospecha]
fichas_sospechosos = [
    ["Lara Mamani", 32, "Asistente", True, 7],
    ["Dr. Carlos Huamán", 55, "Colega", True, 8],
    ["Dra. Sarah Chen", 40, "Colaboradora", True, 6],
    ["Rodrigo Mamani", 60, "Empresario", False, 9],
    ["Mama Killa", 70, "Hermana", True, 5]
]
print("\n=== FICHAS COMPLETAS ===")
for ficha in fichas_sospechosos:
    print(f"Nombre: {ficha[0]} | Edad: {ficha[1]} | Rol: {ficha[2]} | Acceso: {ficha[3]} | Sospecha: {ficha[4]}/10")
```

Pero Wayra notó un problema: si modificaba una ficha, ¿cómo asegurarse de que los datos fueran consistentes? Python tiene otra estructura para eso: las tuplas.

Tuplas: Datos inmutables

Una tupla es como una lista, pero no se puede modificar después de creada. Es ideal para datos que no deben cambiar:

```
# Tuplas para datos fijos (como coordenadas o registros históricos)
escena_crimen = (-13.5167, -71.9781) # Coordenadas del Coricancha
print(f"\nCoordenadas del crimen: {escena_crimen}")
print(f"Latitud: {escena_crimen[0]}")
print(f"Longitud: {escena_crimen[1]}")
# Intentar modificar una tupla causa error
# escena_crimen[0] = 0.0 # ¡Esto lanza TypeError!
```

“Las tuplas son como los quipus históricos” explicó Wayra. No puedes cambiarlos. Solo puedes leerlos e interpretarlos. Las listas, en cambio, son como los quipus digitales: dinámicos, modificables, actualizables.

Métodos de listas: Manipulando las cuerdas

Python ofrece métodos poderosos para trabajar con listas. Wayra los usó para organizar la información del caso:

```
print("\n=== MANIPULANDO LA LISTA ===")
# .append() - Agregar un elemento al final
sospechosos.append("Oficial Paredes (policía)")
print(f"Agregamos al oficial: {sospechosos}")
# .remove() - Eliminar un elemento
sospechosos.remove("Oficial Paredes (policía)")
```

```

print(f"Quitamos al oficial: {sospechosos}")
# .sort() - Ordenar alfabéticamente
sospechosos.sort()
print(f"Ordenados alfabéticamente: {sospechosos}")
# .reverse() - Invertir el orden
sospechosos.reverse()
print(f"Orden inverso: {sospechosos}")
# .pop() - Extraer y eliminar el último elemento (o un índice específico)
ultimo = sospechosos.pop()
print(f"Extraído: {ultimo}")
print(f"Lista actualizada: {sospechosos}")
# .index() - Encontrar la posición de un elemento
posicion = sospechosos.index("Dra. Sarah Chen")
print(f"La Dra. Sarah Chen está en posición: {posicion}")
# .count() - Contar cuántas veces aparece un elemento
conteo = sospechosos.count("Lara Mamani")
print(f"Lara Mamani aparece {conteo} veces")

```

Organizando la evidencia

Wayra decidió organizar toda la evidencia disponible en listas:

```

# --- EVIDENCIA RECOPIADA ---
evidencias_fisicas = [
    "Quipus digital en escritorio",
    "Taza de café frío",
    "Puerta sin cerradura forzada",
    "Código en pantalla OLED"
]
evidencias_digitales = [
    "Mensaje: quipu_digital_001 = 'Án:1:3:5:7:9'",
    "Sistema de bienvenida personalizado",
    "Archivos del Proyecto Yachay encriptados",
    "Registro de acceso de las últimas 24 horas"
]
testigos = [
    "Raúl (periodista, descubrió el cuerpo)",
    "Guardia de seguridad (no vio nada sospechoso)",
    "Vecino del laboratorio (escuchó discusión)"
]
print("\n=== EVIDENCIA DEL CASO ===")
print(f"\nEvidencias físicas ({len(evidencias_fisicas)}):")
for evidencia in evidencias_fisicas:
    print(f"    {evidencia}")
print(f"\nEvidencias digitales ({len(evidencias_digitales)}):")
for evidencia in evidencias_digitales:
    print(f"    {evidencia}")
print(f"\nTestigos ({len(testigos)}):")
for testigo in testigos:
    print(f"    {testigo}")

```

Había algo extraño en el quipus. Wayra volvió a mirarlo. Las cuerdas no solo tenían nudos en diferentes posiciones: también tenían diferentes colores que colgaban de la cuerda principal.

“Los colores...” murmuró. En los quipus, los colores representan categorías. Rojo para el guerrero. Verde para la agricultura. Azul para la religión. Amarillo para el oro. Blanco para la espiritualidad.

“¿Y cómo se traduce eso a código?” preguntó Raúl.

“Con listas de tuplas. Cada cuerda es una lista de nudos, y cada nudo es una tupla de (posición, color, tipo_nudo).”

Enigmas: Construye tu propia base de datos del caso

Enigma 3.1: La lista de coartadas

Crea una lista llamada `coartadas` con las siguientes declaraciones de los sospechosos:

1. Lara: "Estaba en mi departamento, viendo series."
2. Carlos: "Trabajaba en mi oficina de la universidad."
3. Sarah: "Tenía una videollamada con el MIT."
4. Rodrigo: "Estaba en una cena de negocios."
5. Killa: "Dormía. A mi edad, no salgo de noche."

Luego:

- Agrega una coartada para ti mismo
- Ordena la lista alfabéticamente
- Muestra cuántas coartadas hay
- Muestra la primera y la última

Enigma 3.2: Matriz de acceso

Crea una lista de listas (matriz) que represente quién tiene acceso a qué áreas del laboratorio:

```
Áreas: "Laboratorio", "Servidor", "Archivos", "Azotea"
Sospechosos con acceso:
- Lara: todas
- Carlos: Laboratorio, Archivos
- Sarah: Laboratorio, Servidor
- Rodrigo: ninguna
- Killa: Laboratorio, Archivos, Azotea
```

Muestra la matriz y verifica quién tiene acceso al Servidor.

Enigma 3.3: Tuplas inmutables

El Dr. Inti dejó una lista de coordenadas de lugares sagrados donde podría haber escondido información. Como son datos históricos, deben ser tuplas:

```
coordenadas_sagradas = [
    (-13.5167, -71.9781), # Coricancha
    (-13.1631, -72.5450), # Machu Picchu
    (-13.3333, -72.0833), # Ollantaytambo
    (-13.3167, -72.1167) # Pisac
]
```

Agrega una coordenada para Sacsayhuamán (-13.5090, -71.9820) y muestra todas las coordenadas.

Lo que aprendiste

- Las listas son colecciones ordenadas y mutables de elementos
- Se accede a los elementos con índices (empiezan en 0)
- El slicing también funciona con listas
- Las tuplas son inmutables (no se pueden modificar)
- Métodos de listas: `.append()`, `.remove()`, `.sort()`, `.reverse()`, `.pop()`, `.index()`, `.count()`
- Las listas pueden contener cualquier tipo, incluyendo otras listas
- `len()` funciona con listas

Wayra cerró su laptop. Tenía una lista de sospechosos, una lista de evidencias, y un montón de datos que organizar. Pero lo que más le preocupaba era algo que había notado en los registros de acceso del laboratorio.

“Raúl dijo”. Los registros muestran que alguien accedió al sistema a las 2:34 a.m. La entrada dice "Lara Mamani". Pero a las 2:34 a.m., el sistema de seguridad de Lara en su departamento registró su huella

digital... a 15 kilómetros de distancia.

¿Entonces...?

“Alguien usó su cuenta. Alguien que conocía su contraseña. Y hay solo dos personas que conocen la contraseña de Lara: ella misma... y el Dr. Inti Quispe.

Capítulo 4: El Diccionario de Inti

Conceptos: Diccionarios, sets, operaciones con sets

Wayra no podía dormir. Las 3:00 a.m. la encontraron aún frente a su laptop, con el quipus digital de Inti desplegado en la pantalla y una taza de té de coca ya fría a su lado.

Había algo que no encajaba.

Los registros de acceso mostraban que la cuenta de Lara Mamani se había usado a las 2:34 a.m., pero Lara estaba en su departamento. ¿Quién usó su cuenta? ¿Y por qué?

Wayra decidió que necesitaba más información sobre cada sospechoso. No solo nombres y edades: necesitaba datos estructurados que pudiera consultar rápidamente. Necesitaba un diccionario.

Diccionarios: La memoria del quipucamayoc

Si las listas son como cuerdas de quipu ordenadas, los diccionarios son como el conocimiento del quipucamayoc “el guardián de los quipus”: asocian un nombre (la clave) con un valor (el significado).

En Python, los diccionarios se escriben entre {} con pares clave: valor:

```
# =====
# DICCIONARIO DEL CASO
# =====
# Información básica del caso
caso = {
    "nombre": "El Asesinato del Dr. Inti Quispe",
    "fecha": "27 de junio, 2026",
    "lugar": "Templo del Sol (Coricancha)",
    "hora": "02:34 (acceso), 06:30 (descubrimiento)",
    "estado": "abierto",
    "analista": "Wayra Condori"
}
print("=== DATOS DEL CASO ===")
print(f"Nombre: {caso['nombre']}")
print(f"Lugar: {caso['lugar']}")
print(f"Analista: {caso['analista']}")
```

“Los diccionarios explican Wayra mientras tecleaba” son como los quipus temáticos. Cada cuerda principal (clave) tiene un significado (valor). Y puedes tener cuerdas que cuelgan de otras (diccionarios anidados).

```
# Diccionario de un sospechoso con datos detallados
lara = {
    "nombre": "Lara Mamani",
    "edad": 32,
    "rol": "Asistente de laboratorio",
    "acceso": True,
    "nivel_sospecha": 7,
    "coartada": "Estaba en su departamento",
    "conexion_con_inti": "Trabajó junto a él por 3 años",
    "motivo": "Económico - ¿chantaje?"
}
print("\n=== FICHA DE LARA MAMANI ===")
for clave, valor in lara.items():
```

```
print(f"{clave}: {valor}")
```

Diccionarios anidados: Quipus jerárquicos

Los verdaderos quipus tienen una estructura jerárquica: cuerda principal, cuerdas secundarias, cuerdas terciarias... Wayra modela³ esto con diccionarios dentro de diccionarios:

```
# Estructura completa de sospechosos como diccionario anidado
sospechosos = {
    "Lara Mamani": {
        "edad": 32,
        "rol": "Asistente",
        "acceso": True,
        "sospecha": 7,
        "coartada": "Departamento propio",
        "evidencias": ["huella digital en teclado", "registro de acceso falso"]
    },
    "Dr. Carlos Huamán": {
        "edad": 55,
        "rol": "Colega universitario",
        "acceso": True,
        "sospecha": 8,
        "coartada": "Oficina universitaria",
        "evidencias": ["discusión con Inti días antes", "publicaciones bloqueadas"]
    },
    "Dra. Sarah Chen": {
        "edad": 40,
        "rol": "Colaboradora MIT",
        "acceso": True,
        "sospecha": 6,
        "coartada": "Videollamada con MIT",
        "evidencias": ["presión del MIT por resultados", "visa próxima a vencer"]
    },
    "Rodrigo Mamani": {
        "edad": 60,
        "rol": "Empresario tecnológico",
        "acceso": False,
        "sospecha": 9,
        "coartada": "Cena de negocios",
        "evidencias": ["intentos en comprar Yachay", "conexiones políticas"]
    },
    "Mama Killa": {
        "edad": 70,
        "rol": "Hermana",
        "acceso": True,
        "sospecha": 5,
        "coartada": "Dormir en su casa",
        "evidencias": ["herencia", "secretos familiares"]
    }
}

# Accediendo a datos específicos
print(f"\n=== CONSULTAS RÁPIDAS ===")
print(f"¿Qué rol tiene Sarah? {sospechosos['Dra. Sarah Chen']['rol']}")
print(f"Evidencias contra Carlos: {sospechosos['Dr. Carlos Huamán']['evidencias']}")

# Modificando valores
sospechosos["Lara Mamani"]["sospecha"] = 9 # Nueva evidencia la hace más sospechosa
print(f"\nNivel de sospecha de Lara actualizado: {sospechosos['Lara Mamani']['sospecha']/10}")
```

Métodos de diccionarios

Wayra necesitaba herramientas para explorar este "quipu digital" de información:

```
# --- MÉTODOS DE DICCIONARIOS ---
# .keys() - todas las claves
print("\n=== TODOS LOS SOSPECHOSOS ===")
nombres = list(sospechosos.keys())
print(nombres)
```

```
# .values() - todos los valores
print("\n=== DATOS DE CADA SOSPECHOSO ===")
datos = list(sospechosos.values())
for dato in datos:
    print(f"    â€¢ {dato['rol']} - Sospecha: {dato['sospecha']}/10")
# .items() - pares clave-valor
print("\n=== NIVEL DE SOSPECHA ===")
for nombre, info in sospechosos.items():
    print(f"{nombre}: {info['sospecha']}/10")
# Verificar si una clave existe
if "Lara Mamani" in sospechosos:
    print("\nLara Mamani estÃ¡ en la base de datos")
# .get() - obtener valor con default (evita errores)
acceso = sospechosos.get("Lara Mamani", {}).get("acceso", "No especificado")
print(f"Acceso de Lara: {acceso}")
# .update() - actualizar con otro diccionario
nuevos_datos = {"Lara Mamani": {"sospecha": 10, "nota": "Ã¡NUEVA EVIDENCIA!"}}
sospechosos.update(nuevos_datos)
print(f"\nDatos actualizados de Lara: {sospechosos['Lara Mamani']}")
```

Sets: Los nudos Ã³nicos

Hay un tipo de dato mÃ¡s en Python que resultÃ³ crucial para el caso: los sets (conjuntos). Un set es una colecciÃ³n no ordenada de elementos Ã³nicos. Es como un quipu donde cada tipo de nudo solo aparece una vez.

```
# --- SETS: ELEMENTOS ÃšNICOS ---
# Roles Ã³nicos entre los sospechosos
roles = set()
for info in sospechosos.values():
    roles.add(info["rol"])
print("\n=== ROLES ÃšNICOS EN EL CASO ===")
print(roles)
# Evidencias Ã³nicas (sin repetir)
evidencias_totales = set()
for info in sospechosos.values():
    for evidencia in info["evidencias"]:
        evidencias_totales.add(evidencia)
print("\n=== EVIDENCIAS ÃšNICAS RECOPIADAS ===")
for e in evidencias_totales:
    print(f"    â€¢ {e}")
# --- OPERACIONES CON SETS ---
# Â¿QuiÃ©nes tenÃ­an acceso al laboratorio?
tienen_acceso = {nombre for nombre, info in sospechosos.items() if info["acceso"]}
print(f"\nSospechosos con acceso: {tienen_acceso}")
# Â¿QuiÃ©nes tienen sospecha mayor a 7?
alta_sospecha = {nombre for nombre, info in sospechosos.items() if info["sospecha"] > 7}
print(f"Sospechosos con alta sospecha: {alta_sospecha}")
# IntersecciÃ³n: Â¿quiÃ©n cumple AMBAS condiciones?
mas_sospechosos_con_acceso = tienen_acceso & alta_sospecha
print(f"Altamente sospechosos CON acceso: {mas_sospechosos_con_acceso}")
# UniÃ³n: todos los que tienen acceso O alta sospecha
todos_los_posibles = tienen_acceso | alta_sospecha
print(f"Todos los posibles implicados: {todos_los_posibles}")
# Diferencia: acceso pero NO alta sospecha
acceso_pero_no_sospecha = tienen_acceso - alta_sospecha
print(f"Acceso pero baja sospecha: {acceso_pero_no_sospecha}")
```

Wayra observÃ³ los resultados. La intersecciÃ³n mostraba los nombres de los sospechosos con alto nivel de sospecha y acceso al laboratorio: Lara Mamani y el Dr. Carlos HuamÃ¡n.

Pero habÃ­a un problema. Uno de los sospechosos no tenÃ­a acceso directo, pero aparecÃ­a consistentemente en todas las teorÃ­as.

â€œRodrigo Mamani â€œmurmurÃ³ Wayraâ€œ. No tiene acceso al laboratorio, pero es el mÃ¡s sospechoso. Â¿CÃ³mo matÃ³ a Inti si no podÃ­a entrar?

El diccionario del quipu

Wayra volvió al quipus físico. Las cuerdas de colores tenían un patrón específico. Decidió modelar el quipu completo como un diccionario:

```
# --- MODELANDO EL QUIPU FÍSICO COMO DICCIONARIO ---
quipu_inti = {
    "cuerda_roja": {
        "significado": "guerra/conflicto",
        "nudos": [1, 4, 9],
        "tipo_nudo": "simple",
    },
    "cuerda_verde": {
        "significado": "agricultura/conocimiento",
        "nudos": [2, 5, 10],
        "tipo_nudo": "compuesto",
    },
    "cuerda_azul": {
        "significado": "religión/espiritualidad",
        "nudos": [1, 5, 9],
        "tipo_nudo": "simple",
    },
    "cuerda_amarilla": {
        "significado": "riqueza/poder",
        "nudos": [3, 7, 11],
        "tipo_nudo": "compuesto largo",
    },
    "cuerda_blanca": {
        "significado": "conexión espiritual/verdad",
        "nudos": [1, 3, 5, 7, 9, 11],
        "tipo_nudo": "binario intercalado",
    }
}

# Analizando el quipu
print("\n=== ANÁLISIS DEL QUIPU DE INTI ===")
for cuerda, info in quipu_inti.items():
    print(f"\n{cuerda.replace('_', ' ').title()}:")
    print(f" Significado: {info['significado']}")
    print(f" Nudos en posiciones: {info['nudos']}")
    print(f" Tipo: {info['tipo_nudo']}")
```

“La cuerda blanca” dijo Wayra, señalando la pantalla. Tiene nudos en todas las posiciones impares hasta el 11. En la numeración inca, los impares representan preguntas. Pero si tomas los nudos de la cuerda blanca como código binario...

Se detuvo. De repente, todo cobró sentido.

“Los nudos de la cuerda blanca no son preguntas” susurró. Son coordenadas. Posiciones en los otros strings. Es un índice.

Enigmas: Construye tu propia base de datos

Enigma 4.1: Tu diccionario de sospechoso

Crea un diccionario para un nuevo sospechoso (inventa uno) con las siguientes claves:

- `nombre`, `edad`, `rol`, `acceso`, `sospecha`, `coartada`
- Luego agrégalo al diccionario principal `sospechosos`
- Muestra el diccionario actualizado

Enigma 4.2: Consulta de evidencias

Usando el diccionario `sospechosos`, escribe código que muestre:

1. Todos los nombres de los sospechosos

2. La evidencia de cada uno
3. Quié debate tiene la sospecha más alta

Enigma 4.3: Sets en acción

Dados estos dos sets:

```
acceso_lab = {"Lara", "Carlos", "Sarah", "Killa"}
coartada_debil = {"Lara", "Carlos", "Rodrigo"}
```

Encuentra:

- Quié debates tienen acceso Y coartada débil (intersección)
- Quié debates tienen acceso O coartada débil (unión)
- Quié debates tienen acceso pero coartada sólida (diferencia)
- Quié debates tienen coartada débil pero no acceso (diferencia)

Lo que aprendiste

- Los diccionarios asocian claves con valores (`{clave: valor}`)
- Se accede a los valores con `diccionario[clave]`
- Los diccionarios pueden anidarse (valores que son diccionarios)
- Métodos de diccionario: `.keys()`, `.values()`, `.items()`, `.get()`, `.update()`
- Los sets son colecciones no ordenadas de elementos únicos
- Operaciones con sets: `&` (intersección), `|` (unión), `-` (diferencia)
- Los sets se usan para eliminar duplicados y comparar grupos

Eran las 4:00 a.m. cuando Wayra encontró lo que buscaba. La cuerda blanca del quipus no señalaba lugares físicos: señalaba posiciones en el código fuente de Yachay. Inti había escondido la clave de acceso a su IA en los patrones de un tejido.

Pero cuando fue a verificar el archivo, descubrió que alguien más había estado ahí.

El archivo había sido modificado. La última modificación: 2:34 a.m. del 27 de junio. La misma hora del acceso fraudulento de la cuenta de Lara.

Wayra levantó la vista de la pantalla. La neblina matutina comenzaba a iluminarse con los primeros rayos de sol. Afuera, Neo-Cusco despertaba.

Pero ella sabía que el verdadero despertar apenas comenzaba.

Capítulo 5: El Interrogatorio

Conceptos: Condicionales `if/elif/else`, operadores de comparación, operadores lógicos

El sol se había alzado sobre Neo-Cusco cuando Wayra llegó a la comisaría. Raúl había conseguido acceso a las declaraciones oficiales de los sospechosos. Estaban sentados en una sala fría, con paredes de concreto y un espejo unidireccional.

“Van a interrogar a Lara primero” susurró Raúl. Pero las declaraciones ya están aquí. Podemos analizarlas antes.

Wayra tomó los papeles. Había inconsistencias. Todas las declaraciones coincidían en los hechos generales, pero los detalles... los detalles no cuadraban.

“Necesito comparar estas declaraciones” dijo Wayra. Encontrar las contradicciones. Y para eso necesito condicionales.

If, elif, else: Las decisiones del quipucamayoc

Los condicionales son la forma en que Python toma decisiones. Al igual que un quipucamayoc decidí a qué nudo usar según el tipo de información, Python decide qué código ejecutar según las condiciones que le das.

```
# =====
# ANALIZANDO DECLARACIONES
# =====
# Declaración de Lara Mamani
declaracion_lara = {
    "nombre": "Lara Mamani",
    "hora_llegada": "08:00",
    "hora_salida": "18:00",
    "ultima_vez_vio_inti": "18:00",
    "relacion_con_inti": "Excelente, era como un padre",
    "sabe_de_amenazas": False,
    "acceso_a_yachay": True
}
# Análisis básico
print("=== ANÁLISIS DE DECLARACIÓN ===")
if declaracion_lara["acceso_a_yachay"]:
    print("Lara tenía acceso a Yachay.")
else:
    print("Lara NO tenía acceso a Yachay.")
if declaracion_lara["sabe_de_amenazas"]:
    print("Lara conocía amenazas contra Inti.")
else:
    print("Lara dice no conocer amenazas.")
```

“Pero esto es muy básico” dijo Wayra. Necesito cruzar datos. Comparar declaraciones. Encontrar contradicciones.

Operadores de comparación

Python tiene operadores para comparar valores:

| Operador | Significado |

|-----|-----|

| `==` | Igual que |

| `!=` | Diferente de |

| `<` | Menor que |

| `>` | Mayor que |

| `<=` | Menor o igual que |

| `>=` | Mayor o igual que |

```
# --- COMPARANDO DATOS ---
hora_muerte = "02:34"
hora_salida_lara = declaracion_lara["hora_salida"]
print(f"\nHora de la muerte: {hora_muerte}")
print(f"Lara dice que salió a las: {hora_salida_lara}")
if hora_salida_lara < hora_muerte:
    print("Lara se fue ANTES del crimen. Coartada posible.")
elif hora_salida_lara == hora_muerte:
    print("Lara salió EXACTAMENTE a la hora del crimen. Sospechoso.")
else:
    print("Lara se fue DESPUÉS del crimen. ¡Coartada falsa!")
```

Múltiples condiciones: and, or, not

Las decisiones reales rara vez dependen de una sola condición. Python permite combinar condiciones con operadores lógicos:

```
# --- CRUZANDO MÚLTIPLES FACTORES ---
# Datos de los sospechosos
sospechosos = [
    {"nombre": "Lara Mamani", "acceso": True, "coartada": False, "motivo": 7, "huellas": True},
    {"nombre": "Carlos Huamán", "acceso": True, "coartada": True, "motivo": 8, "huellas": False},
    {"nombre": "Sarah Chen", "acceso": True, "coartada": True, "motivo": 6, "huellas": False},
    {"nombre": "Rodrigo Mamani", "acceso": False, "coartada": True, "motivo": 9, "huellas": False},
    {"nombre": "Mama Killa", "acceso": True, "coartada": True, "motivo": 5, "huellas": True}
]

print("\n=== ANÁLISIS DE SOSPECHOSOS ===")
for s in sospechosos:
    # and: ambas condiciones deben ser True
    if s["acceso"] and not s["coartada"]:
        print(f"{s['nombre']}: ALTA PRIORIDAD - Acceso y sin coartada")
    # or: al menos una condición debe ser True
    elif s["motivo"] > 7 or s["huellas"]:
        print(f"{s['nombre']}: MEDIA PRIORIDAD - Motivo fuerte o huellas presentes")
    else:
        print(f"{s['nombre']}: BAJA PRIORIDAD - Poco sospechoso")
```

El interrogatorio completo

Wayra decidió construir un sistema de análisis más complejo:

```
# =====
# SISTEMA DE ANÁLISIS DE DECLARACIONES
# =====
def analizar_declaracion(declaracion):
    """Analiza una declaración y devuelve un nivel de sospecha."""
    puntaje = 0
    razones = []
    # 1. ¿Tiene acceso?
    if declaracion["acceso"]:
        puntaje += 3
        razones.append("Tiene acceso al laboratorio")
    # 2. ¿Tiene coartada verificable?
    if not declaracion["coartada"]:
        puntaje += 4
        razones.append("NO tiene coartada")
    # 3. ¿Nivel de motivo?
    if declaracion["motivo"] >= 8:
        puntaje += 3
        razones.append("Motivo muy fuerte")
    elif declaracion["motivo"] >= 5:
        puntaje += 1
        razones.append("Motivo moderado")
    # 4. ¿Sus huellas están en la escena?
    if declaracion["huellas"]:
        puntaje += 1
        razones.append("Huellas en la escena")
    # 5. Evaluación final
    if puntaje >= 7:
        nivel = "CRÁTICO"
    elif puntaje >= 4:
        nivel = "ALTO"
    elif puntaje >= 2:
        nivel = "MODERADO"
    else:
        nivel = "BAJO"
    return puntaje, nivel, razones
# Analizar a cada sospechoso
print("\n=== RESULTADOS DEL ANÁLISIS ===\n")
for s in sospechosos:
    puntaje, nivel, razones = analizar_declaracion(s)
```

```
print(f"{s['nombre']}:")
print(f" Puntaje: {puntaje}/10 | Nivel: {nivel}")
print(f" Razones: {' ', '.join(razones)}")
print()
```

Los resultados fueron reveladores:

```
=== RESULTADOS DEL ANÁLISIS ===
Lara Mamani:
  Puntaje: 8/10 | Nivel: CRÁTICO
  Razones: Tenía acceso al laboratorio, NO tiene coartada, Huellas en la escena
Carlos Huamán:
  Puntaje: 5/10 | Nivel: ALTO
  Razones: Tenía acceso al laboratorio, Motivo muy fuerte
Sarah Chen:
  Puntaje: 4/10 | Nivel: ALTO
  Razones: Tenía acceso al laboratorio, Motivo moderado
Rodrigo Mamani:
  Puntaje: 3/10 | Nivel: MODERADO
  Razones: Motivo muy fuerte
Mama Killa:
  Puntaje: 5/10 | Nivel: ALTO
  Razones: Tenía acceso al laboratorio, Motivo moderado, Huellas en la escena
```

“Lara es la principal sospechosa” dijo Raúl. Acceso, sin coartada, huellas en la escena...

“Sí” respondió Wayra, pero su tono no era de convencimiento. Demasiado obvio. Es como si alguien hubiera querido que Lara pareciera culpable.

El nudo condicional

Wayra recordó algo que su abuela le había enseñado sobre los quipus: "Cuando veas un nudo que parece obvio, busca el nudo invertido. El mensaje verdadero está en lo que no se ve a simple vista."

Aplicó esa lógica a los datos:

```
# --- BUSCANDO LO NO OBVIO ---
# Si Lara es la culpable obvia, ¿quién se beneficia de eso?
print("=== ANÁLISIS DE BENEFICIARIO ===")
for s in sospechosos:
    # ¿Alguien que NO tiene acceso pero tiene motivo?
    if not s["acceso"] and s["motivo"] >= 8:
        print(f"{s['nombre']}: PODRÍA HABER MANIPULADO a Lara")
    # ¿Alguien que tiene acceso, coartada débil y motivo?
    if s["acceso"] and not s["coartada"] and s["motivo"] >= 5:
        print(f"{s['nombre']}: SOSPECHOSO DIRECTO")
    # ¿Alguien con huellas PERO que dice no haber estado?
    if s["huellas"] and s["coartada"]:
        print(f"{s['nombre']}: CONTRADICCIÓN - Huellas pero dice no estar")
# Un caso especial: acceso sin motivo aparente
for s in sospechosos:
    if s["acceso"] and s["motivo"] < 5:
        print(f"{s['nombre']}: ¿POR QUÉ TENÍA ACCESO SI NO TENÍA MOTIVO?")
```

Wayra se detuvo en el último resultado.

“Mama Killa” murmuró. Tenía acceso, motivo bajo, huellas en la escena, coartada. Es la hermana de Inti. ¿Por qué estar involucrada?

“Quizás no lo está” dijo Raúl.

“O quizás su motivo no es económico” respondió Wayra. Quizás su motivo es más antiguo. Más profundo. Algo que solo un hermano y una hermana comparten.

Enigmas

Enigma 5.1: Evaluador de coartadas

Escribe un programa que pida al usuario:

- Nombre del sospechoso
- Hora de la coartada (input)
- Hora del crimen (asumir 2:34)

Luego eval e:

- Si la coartada es antes de las 2:00 â†' "Coartada s lida"
- Si es entre 2:00 y 2:34 â†' "Coartada d bil"
- Si es despu s de 2:34 â†' "Coartada falsa"

Enigma 5.2: El filtro de sospechosos

Dada la lista de sospechosos con sus datos, escribe un programa que:

1. Muestre solo los sospechosos con nivel de motivo ≥ 7
2. Muestre solo los que tienen acceso Y motivo ≥ 6
3. Muestre solo los que NO tienen coartada O tienen huellas

Enigma 5.3: El juego de las contradicciones

Tres testigos dieron versiones diferentes:

- Testigo 1: "Vi a alguien salir del laboratorio a las 2:30"
- Testigo 2: "A las 2:30, el laboratorio estaba oscuro"
- Testigo 3: "Escuch  una discusi n a las 2:30"

Escribe condicionales que determinen si las versiones son compatibles entre s . Si dos versiones se contradicen, muestra "CONTRADICCI N".

Lo que aprendiste

- `if`, `elif`, `else` permiten ejecutar c digo condicionalmente
- Operadores de comparaci n: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Operadores l gicos: `and`, `or`, `not`
- Los condicionales pueden anidarse y combinarse
- Los an lisis complejos se construyen con m ltiples condiciones
- La l gica condicional permite encontrar patrones y contradicciones

Wayra guard  el an lisis. Los n meros se alaban a Lara, pero su instinto se alaba en otra direcci n. Hab a demasiadas piezas perfectamente colocadas para apuntar a la asistente.

  "Algo no cuadra   "dijo, levant ndose  . Necesito ver el laboratorio otra vez. Y necesito verlo sin la polic a.

Ra l la mir , preocupado.

  "Wayra, si te encuentran ah ...

  "No me van a encontrar. Porque no voy a entrar por la puerta.

Sali  de la comisar a con una determinaci n renovada. El quipus la hab a llevado hasta ah , pero el siguiente paso requer a algo m s que condicionales. Requer a un camino. Un recorrido. Un bucle.

Cap tulo 6: La Ruta del Qhapaq   n

**Conceptos: Bucles `for`/`while`, `range()`, `enumerate()`,
`break`/`continue`**

La noche hab a ca do sobre Neo-Cusco cuando Wayra lleg  al Callej n de las Siete Culebras, una estrecha v a peatonal que serpenteaba detr s del Coricancha. No hab a usado la entrada principal. No hab a usado ninguna entrada visible.

“El Qhapaq Ñan” dijo en voz baja, mirando un tramo de muro inca que parec a s lido”. El Camino del Inca. Treinta mil kil metros de caminos que conectaban todo el imperio. Y todos los caminos pasaban por el Coricancha.

Su abuela le hab a ense ado que el Templo del Sol no solo era un centro religioso: era un nodo de comunicaciones. Los chasquis (mensajeros incas) llegaban desde los cuatro suyos (direcciones del imperio) con quipus y mensajes. Y hab a entradas secretas que solo los guardianes conoc an.

Wayra presionó una piedra en la base del muro. No pasó nada. Presionó otra, tres piedras a la izquierda. Un crujido. Luego una tercera, dos piedras arriba. El muro se abrió lentamente, revelando un pasaje oscuro.

“Los caminos de los incas no desaparecieron” susurró. Solo se volvieron digitales.

For: Recorriendo caminos

Los bucles permiten repetir acciones. En Python, el bucle `for` es perfecto para recorrer colecciones: listas, strings, diccionarios, archivos. Como un chasqui que recorre el Qhapaq Ñan, el `for` visita cada elemento de una secuencia.

```
# =====
# RECORRIENDO EL QHAPAQ Æ‘AN DIGITAL
# =====
# Las estaciones del Qhapaq Æ‘an (en orden)
estaciones_qhapaq_Æ‘an = [
    "Cusco",
    "Ollantaytambo",
    "Machu Picchu",
    "VilcashuamÆ‘in",
    "HuÆ‘inuco Pampa",
    "Cajamarca",
    "Tomebamba",
    "Quito"
]

print("=== RECORRIENDO EL QHAPAQ Æ‘AN ===\n")
print("Ruta completa:")

for estacion in estaciones_qhapaq_Æ‘an:
    print(f"    â†’ {estacion}")
```

“¿Y esto cómo nos ayuda?” preguntó Raúl, que la seguía por el pasaje oscuro.

Inti dejó 3 pistas en lugares específicos. Cada pista está asociada a una estación del Qhapaq Ñan. Si recorremos las estaciones en orden, encontramos el mensaje completo.

`range()`: Caminos numerados

A veces necesitas recorrer un camino sabiendo la posici3n de cada paso. Para eso sirve ``range()`:`

```
print("\n=== ESTACIONES CON NÚMERO DE RUTA ===")
# range(n) genera números de 0 a n-1
for i in range(len(estaciones_qhapaq_Ñan)):
    print(f"Estaci³n {i+1}: {estaciones_qhapaq_Ñan[i]} (posici³n {i})")
```

`enumerate()`: El Índice y el valor

Pero hay una forma m  s elegante:

```
print("\n=== ENUMERANDO LA RUTA ===")
```

```
for indice, estacion in enumerate(estaciones_qhapaq_Ã±an):
    print(f" [{indice}] â†’ {estacion}")
```

â€“Enumerar â€“dijo Wayraâ€“. Como los nudos en un quipu. Cada posiciÃ³n tiene un significado. `enumerate()` te da el Ãndice y el valor al mismo tiempo.

El mensaje en las estaciones

Wayra encontrÃ³ una pequeÃ±a cavidad en la pared del pasaje. Dentro, un datastick con una nota: "Para Wayra. Cada estaciÃ³n guarda una parte del cÃ³digo."

ConectÃ³ el datastick a su laptop:

```
# --- CÃ“DIGO OCULTO EN LAS ESTACIONES ---
codigo_estaciones = {
    "Cusco": "def",
    "Ollantaytambo": " descifrar",
    "Machu Picchu": "_mensaje",
    "VilcashuamÃ±n": "():",
    "HuÃ±uco Pampa": "    quipu",
    "Cajamarca": " = '",
    "Tomebamba": "YACHAY",
    "Quito": ""
}
print("\n=== RECONSTRUYENDO EL CÃ“DIGO ===\n")
codigo_completo = ""
for estacion in estaciones_qhapaq_Ã±an:
    parte = codigo_estaciones.get(estacion, "")
    codigo_completo += parte
    print(f"{estacion}: '{parte}'")
print(f"\n--- CÃ³digo resultante ---")
print(codigo_completo)
```

El cÃ³digo reconstruido era:

```
def descifrar_mensaje():
    quipu = 'YACHAY'
```

â€“Es una funciÃ³n â€“dijo Wayraâ€“. Inti dejÃ³ el esqueleto de una funciÃ³n. La primera lÃnea de lo que serÃ¡ el programa final. Pero estÃ¡ incompleto.

While: Hasta encontrar la verdad

Hay otro tipo de bucle: `while`. Este se ejecuta mientras una condiciÃ³n sea verdadera. Es como caminar por un tÃºnel hasta ver la luz.

```
# =====
# EXPLORANDO EL PASAJE SECRETO
# =====

import time
print("\n=== EXPLORANDO EL PASAJE SECRETO ===\n")
pasos = 0
luz_encontrada = False
while not luz_encontrada:
    pasos += 1
    print(f"Paso {pasos}: Avanzando en la oscuridad...")
    # SimulaciÃ³n: despuÃ©s de 15 pasos, encontramos luz
    if pasos >= 15:
        luz_encontrada = True
        print(f"Ã¡Luz encontrada despuÃ©s de {pasos} pasos!")
        time.sleep(0.3) # PequeÃ±a pausa para dramatismo
print("Hemos llegado al interior del laboratorio.")
```

`break`: Salir del camino

A veces, encuentras lo que buscas antes de llegar al final del camino. `break` te permite salir de un bucle inmediatamente:

```

print("\n=== BUSCANDO EL ARCHIVO YACHAY ===\n")
archivos_del_lab = [
    "informe_quipus.docx",
    "proyecto_yachay.py",
    "datos_experimentales.csv",
    "mensaje_cifrado.txt",
    "yachay_core.py",
    "notas_personales.md"
]
print("Buscando 'yachay_core.py' en los archivos...\n")
for archivo in archivos_del_lab:
    print(f"  Revisando: {archivo}")
    if "yachay" in archivo.lower():
        print(f"    â†’ Â¡ENCONTRADO! Archivo clave: {archivo}")
        break # Salimos del bucle, no necesitamos seguir
print("\nB squeda completada.")

```

 continue : Saltar lo irrelevante

 continue  salta a la siguiente iteraci n, ignorando el resto del c digo del bucle. Es como ignorar las piedras en el camino y seguir avanzando:

```

print("\n=== FILTRANDO ARCHIVOS RELEVANTES ===\n")
for archivo in archivos_del_lab:
    if archivo.endswith(".csv") or archivo.endswith(".md"):
        continue # Saltamos archivos de datos y notas
    print(f"  Archivo relevante: {archivo}")

```

El camino de los sospechosos

Wayra us  bucles para analizar los movimientos de los sospechosos:

```

# =====
# ANALIZANDO MOVIMIENTOS CON BUCLES
# =====
# Registro de acceso del laboratorio (simplificado)
registro_accesos = [
    {"nombre": "Lara", "hora": "08:15", "accion": "entrada"},
    {"nombre": "Inti", "hora": "08:30", "accion": "entrada"},
    {"nombre": "Carlos", "hora": "09:00", "accion": "entrada"},
    {"nombre": "Lara", "hora": "12:30", "accion": "salida"},
    {"nombre": "Lara", "hora": "13:30", "accion": "entrada"},
    {"nombre": "Carlos", "hora": "14:00", "accion": "salida"},
    {"nombre": "Sarah", "hora": "14:30", "accion": "entrada"},
    {"nombre": "Sarah", "hora": "16:00", "accion": "salida"},
    {"nombre": "Inti", "hora": "18:00", "accion": "salida"},
    {"nombre": "Inti", "hora": "22:00", "accion": "entrada"},
    {"nombre": "Lara", "hora": "02:34", "accion": "entrada"}, #  Lara?
    {"nombre": "Lara", "hora": "02:45", "accion": "salida"}, #  Lara?
]
print("=== L NEA DE TIEMPO DEL CASO ===\n")
nombre_anterior = ""
for registro in registro_accesos:
    if nombre_anterior and nombre_anterior != registro["nombre"]:
        print() # L nea en blanco entre personas
        emoji = "    " if registro["accion"] == "entrada" else "    "
        print(f"  {registro['hora']} | {registro['nombre']:7} | {registro['accion']}")
        nombre_anterior = registro["nombre"]
# AN LISIS:  Qui n estaba cuando?
print("\n===  QUI NES ESTABAN A LAS 2:34 AM? ===\n")
hora_crimen = "02:34"
presentes = []
for registro in registro_accesos:
    if registro["hora"] <= hora_crimen and registro["accion"] == "entrada":
        if registro["nombre"] not in presentes:
            presentes.append(registro["nombre"])
    if registro["hora"] > hora_crimen and registro["accion"] == "salida":

```

```

    if registro["nombre"] in presentes:
        presentes.remove(registro["nombre"])
print(f"A la hora del crimen ({hora_crimen}), estaban presentes:")
for p in presentes:
    print(f"    {p}")

```

Los resultados mostraron algo inquietante: según los registros, a las 2:34 a.m., solo "Lara" (o quien usó su cuenta) estaba en el laboratorio. Pero eso era exactamente lo que los registros querían mostrar.

“Falso” dijo Wayra. Si alguien usó la cuenta de Lara, podría haber manipulado los registros. O podría haber entrado sin ser detectado.

“¿Por dónde?” preguntó Raúl.

Wayra señaló el teléfono por el que acababan de llegar.

“Por aquí. Por el Qhapaq Ñan digital. Hay caminos que los sistemas modernos no monitorean. Caminos que los incas construyeron y que Inti restauró.

El bucle infinito del engaño

Wayra escribió un último análisis:

```

# --- SIMULACIÓN: ¿QUIÉN PUDO HABER ENTRADO? ---
sospechosos_con_acceso_secreto = []
# Todos los que sabían del Qhapaq Ñan digital
conocian_ruta = ["Inti", "Mama Killa", "Lara"]
print("\n=== ¿QUIÉN CONOCÍA LA RUTA SECRETA? ===\n")
for s in sospechosos:
    if s["nombre"].split()[0] in conocen_ruta:
        print(f"{s['nombre']}: Conocía la ruta")
        sospechosos_con_acceso_secreto.append(s["nombre"])
    else:
        # Pero alguien pudo haber descubierto la ruta
        if s["motivo"] >= 8:
            print(f"{s['nombre']}: No conocía la ruta, pero tenía motivo para investigar")
            sospechosos_con_acceso_secreto.append(s["nombre"])
print(f"\nPosibles culpables (con acceso secreto): {sospechosos_con_acceso_secreto}")

```

“Mama Killa conocía la ruta. Lara también. Pero Rodrigo... Rodrigo no tenía acceso, pero sí motivo. Y dinero. Dinero para pagar a alguien que sí conociera la ruta.

Enigmas

Enigma 6.1: Recorriendo la lista de evidencias

Dada la lista de evidencias del caso, usa un bucle `for` para mostrar cada evidencia con su número (usando `enumerate`):

```

evidencias = [
    "Quipus digital en escritorio",
    "Registro de acceso manipulado",
    "Código en la pantalla OLED",
    "Mensaje cifrado en las estaciones",
    "Pasaje secreto del Qhapaq Ñan"
]

```

Enigma 6.2: El filtro de archivos

Usando `continue`, escribe un bucle que recorra una lista de archivos y solo muestre los que terminan en `.py` (ignorando `.docx`, `.csv`, `.md`).

Enigma 6.3: Buscar hasta encontrar

Usando `while` y `break`, simula la búsqueda de un sospechoso en una lista. Recorre la lista y cuando encuentres al sospechoso "Rodrigo Mamani", muestra "SOSPECHOSO ENCONTRADO" y rompe el bucle.

Enigma 6.4: Range piramidal

Usando `range()` y bucles anidados, crea un patr n que muestre:

```
1
2 3
4 5 6
7 8 9 10
```

Lo que aprendiste

- `for elemento in coleccion:` recorre secuencias
- `range(n)` genera n meros del 0 al n-1
- `enumerate()` da  ndice y valor simult neamente
- `while condicion:` se ejecuta mientras la condici n sea True
- `break` sale del bucle inmediatamente
- `continue` salta a la siguiente iteraci n
- Los bucles pueden anidarse
- Los bucles permiten procesar colecciones de datos completas

Wayra sali  del pasaje secreto directamente al s tano del laboratorio. Estaba dentro. El sistema de seguridad no la hab a detectado porque su entrada no estaba en ning n registro.

  Los incas construyeron caminos que ni la tecnolog a moderna puede ver   dijo, m s para s  misma que para Ra l  . Y el asesino us  uno de esos caminos.

Pero cuando lleg  al escritorio de Inti, algo hab a cambiado. La pantalla OLED ahora mostraba un nuevo mensaje:

```
Has llegado lejos, Wayra. Pero el verdadero camino
no est  en el Qhapaq  an f sico. Est  en el c digo.
Para encontrar al asesino, primero debes descifrar
la funci n que Inti dej  incompleta.
int yachay_runner(funcion, parametros):
    return funcion(parametros)
```

  Eso no es Python v lido   dijo Ra l.

  No   respondi  Wayra  . Pero es una pista. Necesito funciones. Necesito entender c mo Inti construy  Yachay. Y para eso, tengo que aprender a construir herramientas propias.

Cap tulo 7: Las Funciones del Saber Ancestral

Conceptos: Funciones, par metros, `return`, alcance de variables, docstrings

Wayra estaba dentro del laboratorio. Frente a ella, la pantalla OLED parpadeaba con el mensaje enigm tico sobre "funciones". A su alrededor, el laboratorio de Inti Quispe era un santuario de la tecnolog a andina: estantes con quipus antiguos, monitores con gr ficos de patrones textiles, y una pizarra llena de ecuaciones que mezclaban s mbolos incas con notaci n matem tica moderna.

  Inti no solo programaba   dijo Wayra, recorriendo el espacio con la mirada  . Inti tej a c digo. Cada funci n era como un nudo en un quipu: una unidad de conocimiento que pod a combinarse con otras para crear significados complejos.

En la pizarra, alguien hab a escrito:

```
MAMA â†’ hilo_principal()
TAYTA â†’ hilo_secundario()
Wawa â†’ hilo_terciario()
Cada hilo (funciÃ³n) recibe datos (parÃ¡metros)
y produce un resultado (return).
El tejido completo es el programa.
```

â€”Inti usaba metÃ¡foras textiles para enseÃ±ar programaciÃ³n â€”explicÃ³ Wayraâ€”. Y voy a usar el mismo mÃ©todo.

Funciones: Tejiendo cÃ³digo reutilizable

Una funciÃ³n es un bloque de cÃ³digo que realizas una tarea especÃ­fica y puede reutilizarse. Como un nudo en un quipu: aprendes a hacerlo una vez y lo usas en diferentes cuerdas.

```
# =====
# FUNCIONES DEL SABER ANCESTRAL
# =====
# DefiniciÃ³n de una funciÃ³n bÃ¡sica
def mostrar_bienvenida():
    """Muestra el mensaje de bienvenida del laboratorio."""
    print("=" * 50)
    print("LABORATORIO YACHAY - Conocimiento Ancestral Digital")
    print("=" * 50)
# Llamar a la funciÃ³n
mostrar_bienvenida()
```

ParÃ¡metros: Los hilos de entrada

Los parÃ¡metros son los datos que la funciÃ³n recibe para trabajar. Como los hilos de diferentes colores que recibe un tejedor:

```
print("\n=== FICHAS DE SOSPECHOSOS ===\n")
def crear_ficha(nombre, edad, rol, nivel_sospecha):
    """Crea una ficha formateada de un sospechoso."""
    print(f"â- {nombre.upper()}")
    print(f" Edad: {edad}")
    print(f" Rol: {rol}")
    print(f" Nivel de sospecha: {nivel_sospecha}/10")
    print()
# Llamar a la funciÃ³n con diferentes argumentos
crear_ficha("Lara Mamani", 32, "Asistente", 7)
crear_ficha("Dr. Carlos HuamÃ¡n", 55, "Colega", 8)
crear_ficha("Mama Killa", 70, "Hermana", 5)
```

ParÃ¡metros por defecto

A veces, un parÃ¡metro tiene un valor que se usa la mayorÃa de las veces. Python permite establecer valores por defecto:

```
def crear_ficha_avanzada(nombre, edad, rol, nivel_sospecha=5, acceso=False):
    """Crea una ficha con valores por defecto."""
    print(f"â- {nombre}")
    print(f" Edad: {edad} | Rol: {rol}")
    print(f" Sospecha: {nivel_sospecha}/10 | Acceso: {acceso}")
    print()
crear_ficha_avanzada("Rodrigo Mamani", 60, "Empresario", 9)
crear_ficha_avanzada("Sarah Chen", 40, "Colaboradora")
crear_ficha_avanzada("Visitante Desconocido", 0, "Desconocido")
```

â€”Los valores por defecto son como los nudos estÃ¡ndar en un quipu â€”dijo Wayraâ€”. La mayorÃa de los quipus usan nudos simples por defecto, a menos que especifiques lo contrario.

Return: El resultado del tejido

Las funciones no solo hacen cosas: tambiÃ©n devuelven resultados con `return`. Como un quipucamayoc

que "devuelve" el significado después de leer los nudos:

```
def calcular_nivel_sospecha(acceso, coartada, motivo, huellas):
    """Calcula un nivel de sospecha basado en múltiples factores.
    Parámetros:
        acceso (bool): Si la persona tenía acceso al laboratorio
        coartada (bool): Si la persona tiene coartada
        motivo (int): Nivel de motivo (1-10)
        huellas (bool): Si sus huellas están en la escena
    Returns:
        int: Nivel de sospecha (0-10)
    """
    puntaje = 0
    if acceso:
        puntaje += 3
    if not coartada:
        puntaje += 4
    if motivo >= 7:
        puntaje += 3
    elif motivo >= 4:
        puntaje += 1
    if huellas:
        puntaje += 1
    return min(puntaje, 10) # Máximo 10

# Usando la función
nivel_lara = calcular_nivel_sospecha(True, False, 7, True)
nivel_carlos = calcular_nivel_sospecha(True, True, 8, False)
print(f"\nNivel de sospecha de Lara: {nivel_lara}/10")
print(f"Nivel de sospecha de Carlos: {nivel_carlos}/10")
```

Múltiples valores de retorno

Una función puede devolver varios valores como una tupla:

```
def analizar_sospechoso_completo(nombre, acceso, coartada, motivo, huellas):
    """Analiza un sospechoso y devuelve múltiples resultados."""
    nivel = calcular_nivel_sospecha(acceso, coartada, motivo, huellas)
    if nivel >= 8:
        categoria = "CRÍTICO"
        accion = "Interrogar inmediatamente"
    elif nivel >= 5:
        categoria = "ALTO"
        accion = "Vigilancia continua"
    elif nivel >= 3:
        categoria = "MODERADO"
        accion = "Mantener en observación"
    else:
        categoria = "BAJO"
        accion = "Sin acción requerida"
    return nivel, categoria, accion

# Desempaquetando el resultado
nombre = "Lara Mamani"
nivel, categoria, accion = analizar_sospechoso_completo(
    nombre, True, False, 7, True
)
print(f"\n=== RESULTADO: {nombre} ===")
print(f"Nivel: {nivel}/10")
print(f"Categoría: {categoria}")
print(f"Acción recomendada: {accion}")
```

Alcance de variables: El mundo de cada hilo

En Python, las variables tienen alcance (scope). Las variables definidas dentro de una función solo existen dentro de esa función. Como los hilos de un tejido: un hilo rojo en una parte del tejido no afecta al hilo azul en otra parte.

```
# Variable global (fuera de cualquier función)
```

```

laboratorio = "Coricancha"
investigadora = "Wayra Condori"
def mostrar_caso():
    # Variable local (solo existe dentro de esta funci3n)
    nombre_caso = "El Asesinato del Dr. Inti Quispe"
    # Podemos ACCEDER a variables globales
    print(f"Caso: {nombre_caso}")
    print(f"Lugar: {laboratorio}")
    print(f"Investigadora: {investigadora}")
mostrar_caso()
# print(nombre_caso) # ¡Error! nombre_caso no existe aqu3-
def cambiar_laboratorio():
    # Para MODIFICAR una variable global, necesitamos 'global'
    global laboratorio
    laboratorio = "Laboratorio Secreto del Qhapaq Ñan"
    print(f"Laboratorio cambiado a: {laboratorio}")
cambiar_laboratorio()
print(f"Laboratorio ahora es: {laboratorio}")

```

La funci3n del quipu descifrador

Wayra encontr3 en la computadora de Inti una funci3n incompleta. Era la que hab3a visto antes, pero ahora ten3a m3is contexto:

```

# --- LA FUNCION INCOMPLETA DE INTI ---
def descifrar_quipu_digital(quipu_codificado):
    """Descifra un quipu digital y devuelve el mensaje oculto.
    Los quipus digitales usan el formato:
    'COLOR:pos1:pos2:pos3'
    Donde:
    - COLOR es el tipo de informaci3n (ROJO, VERDE, AZUL, etc.)
    - posN son las posiciones de los nudos
    Returns:
        dict: Con las claves 'tipo', 'posiciones', 'mensaje'
    """
    # Separar el quipu en partes
    partes = quipu_codificado.split(":")
    tipo = partes[0]
    posiciones = [int(p) for p in partes[1:]]
    # Decodificar seg3n el tipo
    # (Esta parte estaba incompleta - Wayra la complet3)
    return {
        "tipo": tipo,
        "posiciones": posiciones,
        "mensaje": f"Decodificado: {tipo} - {posiciones}"
    }
# Probar la funci3n
quipu1 = descifrar_quipu_digital("ROJO:1:4:9")
quipu2 = descifrar_quipu_digital("BLANCO:1:3:5:7:9")
print("\n=== QUIPUS DESCIFRADOS ===")
print(f"Quipu 1: {quipu1}")
print(f"Quipu 2: {quipu2}")

```

Las funciones del enigma

Wayra se dio cuenta de que necesitaba construir un sistema completo de an3lisis usando funciones:

```

# =====
# SISTEMA DE AN3LISIS FORENSE DIGITAL
# =====
def verificar_coartada(hora_declarada, hora_crimen, margen=30):
    """Verifica si una coartada es plausible.
    Args:
        hora_declarada (str): Hora que la persona dice estar
        hora_crimen (str): Hora del crimen
        margen (int): Minutos de tolerancia
    Returns:

```

```

        bool: True si la coartada es válida
    """
    # Convertir horas a minutos para comparar
    h_declarada, m_declarada = map(int, hora_declarada.split(":"))
    h_crimen, m_crimen = map(int, hora_crimen.split(":"))
    total_declarada = h_declarada * 60 + m_declarada
    total_crimen = h_crimen * 60 + m_crimen
    diferencia = abs(total_declarada - total_crimen)
    return diferencia > margen

def buscar_patron_en_quipus(lista_quipus, color_buscar):
    """Busca quipus de un color específico en una lista."""
    resultados = []
    for quipu in lista_quipus:
        if quipu.startswith(color_buscar):
            resultados.append(quipu)
    return resultados

def generar_informe(sospechosos_analizados):
    """Genera un informe formateado con todos los análisis."""
    print("\n" + "=" * 60)
    print("INFORME FORENSE DIGITAL - CASO INTI QUISPE")
    print("=" * 60)
    for nombre, datos in sospechosos_analizados.items():
        print(f"\n- {nombre}")
        print(f"    Nivel de sospecha: {datos['nivel']/10}")
        print(f"    Categoría: {datos['categoria']}")
        print(f"    Acción: {datos['accion']}")
    print("\n" + "=" * 60)
    print("FIN DEL INFORME")
    print("=" * 60)

# --- EJECUTANDO EL SISTEMA ---
sospechosos_data = {
    "Lara Mamani": {"acceso": True, "coartada": False, "motivo": 7, "huellas": True},
    "Carlos Huamán": {"acceso": True, "coartada": True, "motivo": 8, "huellas": False},
    "Sarah Chen": {"acceso": True, "coartada": True, "motivo": 6, "huellas": False},
    "Rodrigo Mamani": {"acceso": False, "coartada": True, "motivo": 9, "huellas": False},
    "Mama Killa": {"acceso": True, "coartada": True, "motivo": 5, "huellas": True}
}

resultados = {}
for nombre, datos in sospechosos_data.items():
    nivel, categoria, accion = analizar_sospechoso_completo(
        nombre,
        datos["acceso"],
        datos["coartada"],
        datos["motivo"],
        datos["huellas"]
    )
    resultados[nombre] = {
        "nivel": nivel,
        "categoria": categoria,
        "accion": accion
    }

generar_informe(resultados)

```

Enigmas

Enigma 7.1: Tu primera función

Escribe una función llamada `presentar\_sospechoso()` que reciba un nombre y un rol, y muestre:

```

SOSPECHOSO: [nombre]
ROL: [rol]

```

Enigma 7.2: Calculadora de coartadas

Escribe una función `verificar\_coartada(hora\_coartada, hora\_crimen)` que devuelva `True` si la coartada es al menos 1 hora antes o después del crimen (asume hora_crimen = "02:34"). Usa `return`.

Enigma 7.3: Función con valores por defecto

Crea una función `mensaje\_secreto(mensaje, cifrado=True)` que:

- Si `cifrado` es `True`, devuelve el mensaje invertido (`[::-1]`)
- Si `cifrado` es `False`, devuelve el mensaje original

Enigma 7.4: Sistema de puntaje completo

Usando las funciones que has visto, crea un programa que pida datos de un sospechoso (input) y devuelva su nivel de sospecha.

Lo que aprendiste

- `def nombre\_funcion():` define una función
- Los parámetros son los datos que recibe la función
- Los valores por defecto hacen parámetros opcionales
- `return` devuelve un valor desde la función
- Las funciones pueden devolver múltiples valores como tupla
- El alcance (scope) determina qué variables son accesibles
- `global` permite modificar variables globales desde una función
- Los docstrings (``"""...""`) documentan qué hace la función

Wayra terminó de escribir su sistema de análisis. Ahora tenía herramientas reutilizables para cualquier nuevo dato que apareciera. Pero cuando giró para mostrarle el resultado a Raúl, notó que algo había cambiado en el laboratorio.

La computadora de Inti estaba encendida. Pero no la había encendido ella.

“¿Raúl dijo lentamente?” ¿Tocaste la computadora?

“No. Pensé que habías sido tonto.”

Wayra se acercó. En la pantalla, una ventana de terminal mostraba un proceso activo:

```
YACHAY_CORE.EXE - EJECUTANDO
ÚLTIMA MODIFICACIÓN: 27/06/2026 02:34
ARCHIVOS ACCEDIDOS: quipus_digitales.db, registro_accesos.log
USUARIO: root
```

“Alguien más está en el sistema” susurró Wayra. Y está usando Yachay en este momento.

De repente, la pantalla cambió. Un nuevo mensaje apareció:

```
Wayra Condori.
Has descifrado funciones. Has recorrido caminos.
Pero el código tiene un archivo que no has abierto.
El archivo que contiene la verdadera historia.
Estás en el laboratorio. Busca en los quipus.
Pero cuidado: algunos archivos guardan secretos
que no deberían ser descubiertos.
-- Inti (o quien controla su legado)
```

Wayra sintió que el tiempo se aceleraba. El asesino “o alguien” estaba interactuando con ella en tiempo real. Sabía su nombre. Sabía lo que había hecho.

Y la estaba invitando a seguir.

“¿Raúl dijo?” Busca todos los archivos del laboratorio. Archivos físicos, digitales, quipus, lo que sea. Necesito encontrar ese archivo antes que el asesino.

“¿Antes? ¿No estás muerto Inti?”

Wayra negó con la cabeza.

“El asesino no es el que estÃ¡ en el sistema ahora. El asesino es quien matÃ³ a Inti. Pero quien estÃ¡ en el sistema ahora... es otra persona. Alguien que estÃ¡ usando Yachay. Y que quiere que yo encuentre algo.

“¿QuÃ©?

“No lo sÃ©. Pero solo hay una forma de descubrirlo. Abrir los archivos de Inti.

Capítulo 8: Los Archivos del Laboratorio

Conceptos: Manejo de archivos, `open()`, `with`, `read`/`write`, rutas

El laboratorio de Inti Quispe era un caos ordenado. Pero Wayra sabÃa que en ese caos habÃa un patrÃn. Como en los quipus, donde cada cuerda tiene un lugar especÃfico, cada archivo en el laboratorio tenÃa un propÃsito.

“Los archivos “dijo Wayra, abriendo el explorador de archivos” son como las cuerdas de un quipu. Algunas son principales, otras secundarias. Pero todas guardan informaciÃn.

“¿Por dÃnde empezamos? “preguntÃ RaÃl.

“Por el principio. Inti dejÃ pistas en archivos de texto. Pero no archivos normales: archivos con formato de quipu.

Leyendo archivos: Escuchar las cuerdas

En Python, `open()` es la puerta de entrada a los archivos. Piensa en ello como tomar una cuerda de quipu en tus manos para leer sus nudos.

```
# =====
# SISTEMA DE LECTURA DE ARCHIVOS
# =====
# --- MODO LECTURA: Leer el archivo completo ---
# SimulaciÃn: creamos un archivo de evidencias
evidencias_texto = """=== EVIDENCIAS DEL CASO INTI QUISPE ===
1. Quipus digital modificado el 27/06/2026 a las 02:34
2. Registro de acceso con cuenta de Lara Mamani
3. Mensaje en pantalla OLED: "Bienvenido, Wayra"
4. Pasaje secreto del Qhapaq Ãan restaurado
5. CÃdigo fuente de Yachay modificado
NOTA: El quipu blanco contiene la clave.
"""

# Primero, guardamos este texto en un archivo
with open("evidencias_caso.txt", "w", encoding="utf-8") as archivo:
    archivo.write(evidencias_texto)

# Ahora lo leemos
with open("evidencias_caso.txt", "r", encoding="utf-8") as archivo:
    contenido = archivo.read()

print("=== CONTENIDO DEL ARCHIVO ===")
print(contenido)
```

El bloque `with`: Tejiendo y destejendo

El bloque `with` es como tomar una cuerda, leerla y devolverla a su lugar. Se encarga de abrir y cerrar el archivo automÃticamente. Sin `with`, tendrÃas que acordarte de cerrar el archivo manualmente.

Diferentes modos de apertura

Los archivos pueden abrirse en diferentes modos, como diferentes formas de tejer:

Modo	Significado
----- -----	

``r``	Lectura (read)
``w``	Escritura (write) - sobrescribe
``a``	Añadir (append) - agrega al final
``r+``	Lectura y escritura

Leyendo línea por línea

Para archivos grandes (como un quipu largo), es mejor leer línea por línea:

```
print("=== LEYENDO LÍNEA POR LÍNEA ===\n")
with open("evidencias_caso.txt", "r", encoding="utf-8") as archivo:
    for numero_linea, linea in enumerate(archivo, 1):
        print(f"Línea {numero_linea}: {linea.strip()}")
```

El archivo de los sospechosos

Wayra encontró un archivo llamado `sospechosos.csv` en el escritorio de Inti:

```
# Simulamos el archivo CSV
csv_content = """nombre,edad,rol,acceso,motivo
Lara Mamani,32,Asistente,True,7
Carlos Huamán,55,Colega,True,8
Sarah Chen,40,Colaboradora,True,6
Rodrigo Mamani,60,Empresario,False,9
Mama Killa,70,Hermana,True,5
"""

with open("sospechosos.csv", "w", encoding="utf-8") as f:
    f.write(csv_content)

# Procesando el CSV manualmente
print("=== DATOS DE SOSPECHOSOS (CSV) ===\n")
with open("sospechosos.csv", "r", encoding="utf-8") as f:
    # Leer la primera línea (encabezados)
    encabezados = f.readline().strip().split(",")
    print(f"Columnas: {encabezados}\n")
    # Leer el resto de líneas
    for linea in f:
        datos = linea.strip().split(",")
        nombre, edad, rol, acceso, motivo = datos
        print(f"  {nombre:25} | Edad: {edad:3} | {rol:20} | Acceso: {acceso:5} | Motivo: {motivo}")

# Lectura con list comprehension
print("\n=== SOLO NOMBRES Y MOTIVOS ===\n")
with open("sospechosos.csv", "r", encoding="utf-8") as f:
    next(f) # Saltar encabezados
    sospechosos_lista = [linea.strip().split(",") for linea in f]
for s in sospechosos_lista:
    print(f"  {s[0]}: Motivo {s[4]}/10")
```

Escribiendo archivos: Documentando el caso

Wayra necesitaba documentar todo lo que estaba descubriendo:

```
# =====
# DIARIO DE INVESTIGACIÓN
# =====
print("=== DOCUMENTANDO EL CASO ===\n")
with open("diario_wayra.txt", "w", encoding="utf-8") as diario:
    diario.write("== DIARIO DE INVESTIGACIÓN ==\n")
    diario.write("Caso: Asesinato del Dr. Inti Quispe\n")
    diario.write(f"Fecha: 27 de junio, 2026\n")
    diario.write(f"Investigadora: Wayra Condori\n")
    diario.write("--- DÍA 1 ---\n")
    diario.write("Descubrí el cuerpo a través de Raúl.\n")
    diario.write("Encontré un quipu digital con mensaje cifrado.\n")
    diario.write("El sistema de bienvenida sabía a que yo llegaría.\n")
    diario.write("---- SOSPECHOSOS INICIALES ---\n")
    for s in sospechosos_lista:
```

```

diario.write(f"{s[0]}: {s[2]} - Sospecha: {s[4]}/10\n")
# Verificar que se escribi  correctamente
print("Archivo 'diario_wayra.txt' creado.\n")
with open("diario_wayra.txt", "r", encoding="utf-8") as diario:
    print(diario.read())

```

A adiendo datos (modo append)

M s tarde, Wayra quiso agregar informaci n sin borrar lo anterior:

```

print("=== AGREGANDO NUEVA EVIDENCIA ===\n")
with open("diario_wayra.txt", "a", encoding="utf-8") as diario:
    diario.write("\n--- NUEVA EVIDENCIA ---\n")
    diario.write("Encontr  un pasaje secreto del Qhapaq  an.\n")
    diario.write("El quipus blanco contiene coordenadas.\n")
    diario.write("Alguien est  usando Yachay en este momento.\n")
# Leer el archivo actualizado
with open("diario_wayra.txt", "r", encoding="utf-8") as diario:
    print(diario.read())

```

El archivo cifrado: El legado de Inti

Wayra encontr  un archivo llamado `quipu\_legado.txt`. Pero cuando intent  abrirlo, el contenido era un galimat as:

```

# Simulando el archivo cifrado
with open("quipu_legado.txt", "w", encoding="utf-8") as f:
    f.write("""
qUiPu DiGiTaL v2.0
 aN:1:3:5:7:9:11:13
YaChAy:2:4:6:8:10:12:14
CoRiCaNcHa:1:4:9:16:25
PaChAcUtEc:3:6:12:24:48
""").strip())
# Leer y procesar el archivo cifrado
print("=== QUIPU LEGADO: ARCHIVO ENCONTRADO ===\n")
with open("quipu_legado.txt", "r", encoding="utf-8") as f:
    lineas = f.readlines()
print(f"Total de l neas: {len(lineas)}\n")
for linea in lineas:
    linea = linea.strip()
    if ":" in linea:
        partes = linea.split(":")
        nombre = partes[0]
        numeros = [int(x) for x in partes[1:]]
        print(f"   {nombre}: {numeros}")
        # Analizar patr n
        if nombre.isupper():
            print(f"   TODO MAY SCULAS: Palabra clave")
        elif nombre[0].isupper():
            print(f"   Capitalizado: Nombre propio")
        #   Patr n num rico?
        if len(numeros) >= 3:
            diferencias = [numeros[i+1] - numeros[i] for i in range(len(numeros)-1)]
            if len(set(diferencias)) == 1:
                print(f"   Progresi n aritm tica (diferencia: {diferencias[0]})")
            elif numeros == [i**2 for i in range(1, len(numeros)+1)]:
                print(f"   N meros cuadrados")
            elif all(n % 2 == 1 for n in numeros):
                print(f"   Solo impares (preguntas)")
            elif all(n % 2 == 0 for n in numeros):
                print(f"   Solo pares (respuestas)")
        print()

```

El archivo que cambi  todo

Entre todos los archivos, uno llam  la atenci n de Wayra: `confesion\_inti.txt`. Pero cuando fue a

abrirlo,â€Ź° estaba vacÃƒo.

â€ŒNo estÃƒ vacÃƒo â€Œdijo Wayraâ€Œ. EstÃƒ cifrado. Es un archivo que Inti dejÃ³ preparado, pero el contenido se borrÃ³ cuando... cuando lo mataron.

â€ŒÂ¿CÃ³mo sabes que tenÃƒa contenido? â€ŒpreguntÃ³ RaÃƒl.

Wayra seÃ±alÃ³ las propiedades del archivo:

```
import os
# Ver metadatos del archivo
print("=== METADATOS DEL ARCHIVO ===\n")
if os.path.exists("quipu_legado.txt"):
    tamaÃ±o = os.path.getsize("quipu_legado.txt")
    print(f"Archivo: quipu_legado.txt")
    print(f"TamaÃ±o: {tamaÃ±o} bytes")
    print(f"Existe: {os.path.exists('quipu_legado.txt')}")
    print(f"Es archivo: {os.path.isfile('quipu_legado.txt')}")
# Listar todos los archivos .txt del directorio
print("\n=== ARCHIVOS DE TEXTO ENCONTRADOS ===\n")
archivos_txt = [f for f in os.listdir('.') if f.endswith('.txt')]
for archivo in archivos_txt:
    tamaÃ±o = os.path.getsize(archivo)
    print(f"â€Œ {archivo:30} ({tamaÃ±o} bytes)")
```

Pero Wayra notÃ³ algo mÃ¡js. HabÃƒa un archivo oculto en el sistema: ``.quipu_maestro.txt``. Los archivos que empiezan con punto son ocultos en sistemas Unix.

```
# --- DESCUBRIENDO EL ARCHIVO OCULTO ---
contenido_oculto = ""
ERES LA HEREDERA DEL CONOCIMIENTO.
BUSCA EN LOS NUDOS DEL QUIPU BLANCO.
LA RESPUESTA ESTÃ EN EL CÃ“DIGO FUENTE DE YACHAY.
NO CONFÃES EN LARA. NO CONFÃES EN CARLOS.
MAMA KILLA SABE LA VERDAD PERO NO LA DIRÃ.
EL ASESINO ESTÃ MÃS CERCA DE LO QUE CREES.
-- INTI
"""

with open(".quipu_maestro.txt", "w", encoding="utf-8") as f:
    f.write(contenido_oculto)
# Leer el archivo oculto
with open(".quipu_maestro.txt", "r", encoding="utf-8") as f:
    mensaje_oculto = f.read()
print("=== MENSAJE OCULTO ENCONTRADO ===\n")
print(mensaje_oculto)
```

Enigmas

Enigma 8.1: Crea tu propio archivo de caso

Usando ``open()`` y ``with``, crea un archivo llamado ``mi_informe.txt`` que contenga:

- Tu nombre como investigador
- La fecha de hoy
- Una lista de 3 sospechosos con sus datos
- Luego lÃ©elo y muÃ©stralo en pantalla

Enigma 8.2: Procesa el archivo de coartadas

Dado el siguiente contenido de archivo:

```
Lara Mamani:22:30:01:00
Carlos HuamÃ¡n:01:00:02:30
Sarah Chen:02:30:04:00
```

Donde el formato es ``nombre:hora_inicio:hora_fin``, escribe un programa que:

1. Lea el archivo

2. Por cada persona, determine si la hora del crimen (02:34) está dentro de su rango
3. Muestre quién tiene coartada y quién no

Enigma 8.3: Bitácora de investigación

Crea un programa que:

1. Pregunte al usuario qué evidencia nueva encontró (input)
2. Agregue esa evidencia al final del archivo `bitacora.txt` (modo append)
3. Muestre todo el contenido actualizado de la bitácora

Enigma 8.4: Buscador de palabras clave

Escribe un programa que lea un archivo de texto y cuente cuántas veces aparece la palabra "YACHAY" (en cualquier combinación de mayúsculas/minúsculas). Pista: usa `.lower()` para normalizar.

Lo que aprendiste

- `open(archivo, modo)` abre un archivo en el modo especificado
- El bloque `with` asegura que el archivo se cierre automáticamente
- Modos comunes: `"r"` (lectura), `"w"` (escritura), `"a"` (añadir)
- `.read()` lee todo el contenido, `.readlines()` lee línea por línea
- Se puede iterar sobre un archivo directamente con `for linea in archivo`
- `os.path` proporciona funciones para trabajar con rutas y metadatos
- Los archivos ocultos empiezan con `.'` en Unix/Linux
- Los archivos CSV se pueden procesar con `.split(",")` y list comprehension

Wayra guardó el mensaje del archivo oculto. Las advertencias de Inti eran claras: "No confíes en Lara. No confíes en Carlos. Mama Killa sabe la verdad."

Pero si no podía confiar en los sospechosos principales, ¿en quién podía confiar?

"¿Raúl dijo?" ¿Confío en ti?

Raúl la miró, sorprendido.

"Wayra, llevamos años siendo amigos. Te llamé porque confío en ti."

"Lo sé. Pero Inti escribió ese mensaje antes de morir. Y sabía que alguien lo iba a traicionar. La pregunta es: ¿cuál de los sospechosos es el que realmente lo mató?"

Wayra miró la pantalla. El sistema Yachay seguía activo. Y alguien seguía observándola.

"Hay una forma de descubrirlo" dijo. Pero necesito hacer que el sistema falle. Necesito que muestre un error. Porque en los errores, en las excepciones, es donde se esconden las verdades que no deberían verse.

"¿Y si el sistema se bloquea?"

"Entonces el asesino sabrá que estamos aquí. Pero también sabrá que estamos cerca de la verdad."

Wayra tomó el teclado. Era hora de forzar un error.

Capítulo 9: El Círculo del Sol

Conceptos: Manejo de errores, `try/except/finally`, excepciones personalizadas

Wayra presionÃ³ Enter.

El laboratorio entero crujiÃ³. Las luces parpadearon. Los monitores mostraron lÃneas de error en rojo. Y luego, silencio.

â€”Â¿QuÃ© hiciste? â€”preguntÃ³ RaÃ¶l, alarmado.

â€”ForcÃ© una divisiÃ³n entre cero en el sistema Yachay â€”respondiÃ³ Wayra, observando la pantallaâ€”. En Python, cuando algo sale mal, el programa lanza una excepciÃ³n. Y las excepciones revelan informaciÃ³n.

En la terminal, un mensaje de error apareciÃ³:

```
Traceback (most recent call last):
  File "yachay_core.py", line 147, in descifrar_quipu
    resultado = quipu_data["cuerdas"][0]["nudos"] / 0
ZeroDivisionError: division by zero
```

â€”Mira â€”dijo Wayra, seÃ±alandoâ€”. El error nos muestra la ruta del archivo: `yachay\_core.py`, lÃnea 147, funciÃ³n `descifrar\_quipu`. Y algo mÃ¡s... la estructura de datos: `quipu\_data["cuerdas"][0]["nudos"]`. Yachay estÃ¡ organizado como un diccionario de cuerdas.

â€”Â¿Pero y si el error hubiera roto algo importante?

â€”Por eso vamos a aprender a manejar los errores. En Python, puedes capturar las excepciones y decidir quÃ© hacer con ellas. Como un tejedor que, cuando se le rompe un hilo, no abandona el tejido: hace un nudo y sigue.

Try/Except: Tejiendo a prueba de errores

El bloque `try/except` permite intentar cÃ³digo que podrÃa fallar y manejar el error si ocurre:

```
# =====
# MANEJO DE ERRORES EN EL LABORATORIO
# =====
print("=== SISTEMA DE ANÃLISIS TOLERANTE A FALLOS ===\n")
def analizar_quipu_seguro(quipu_data):
    """Analiza un quipu de forma segura, manejando errores."""
    try:
        # Intentar acceder a los datos del quipu
        nombre = quipu_data["nombre"]
        cuerdas = quipu_data["cuerdas"]
        nudos_totales = sum(len(c["nudos"]) for c in cuerdas)
        print(f" â€” Quipu '{nombre}' analizado: {nudos_totales} nudos en {len(cuerdas)} cuerdas")
        return nudos_totales
    except KeyError as e:
        print(f" â€” Error: Falta la clave {e} en el quipu")
        return 0
    except TypeError as e:
        print(f" â€” Error de tipo: {e}")
        return 0
    except Exception as e:
        print(f" â€” Error inesperado: {e}")
        return 0
# Probar con datos vÃ¡lidos
quipu_valido = {
    "nombre": "Quipu ceremonial",
    "cuerdas": [
        {"color": "rojo", "nudos": [1, 4, 9]},
        {"color": "blanco", "nudos": [1, 3, 5, 7, 9]}
    ]
}
# Probar con datos invÃ¡lidos
quipu_sin_nombre = {
    "cuerdas": [{"color": "rojo", "nudos": [1, 2, 3]}]
}
quipu_sin_cuerdas = {
```

```

    "nombre": "Quipu vac o"
}
quipu_invalido = "Esto no es un quipu"
analizar_quipu_seguro(quipu_valido)
analizar_quipu_seguro(quipu_sin_nombre)
analizar_quipu_seguro(quipu_sin_cuerdas)
analizar_quipu_seguro(quipu_invalido)

```

Capturando m ltiples excepciones

Wayra necesitaba procesar los archivos cifrados de Inti, pero muchos estaban da ados o ten an formatos inesperados:

```

print("\n=== PROCESANDO ARCHIVOS DEL LABORATORIO ===\n")
def procesar_archivo_cifrado(ruta_archivo):
    """Intenta leer y descifrar un archivo del laboratorio."""
    try:
        with open(ruta_archivo, "r", encoding="utf-8") as f:
            contenido = f.read()
            # Intentar convertir a n mero
            try:
                numero = int(contenido.strip())
                print(f"    Archivo num rico: {numero}")
            except ValueError:
                # No es un n mero, intentar como quipu
                if ":" in contenido:
                    partes = contenido.split(":")
                    print(f"    Formato quipu: {partes[0]}    {partes[1:]}")
                else:
                    print(f"    Texto plano: {contenido[:50]}...")
    except FileNotFoundError:
        print(f"    Archivo no encontrado: {ruta_archivo}")
    except PermissionError:
        print(f"    Sin permisos para leer: {ruta_archivo}")
    except UnicodeDecodeError:
        print(f"    Codificaci n no soportada: {ruta_archivo}")
    except Exception as e:
        print(f"    Error desconocido procesando {ruta_archivo}: {e}")

# Simular archivos del laboratorio
import os
# Crear archivos de prueba
archivos_prueba = {
    "quipu_rojo.txt": "ROJO:1:4:9",
    "quipu_blanco.txt": "BLANCO:1:3:5:7:9",
    "numero_secreto.txt": "42",
    "nota_inti.txt": "Mama Killa sabe la verdad sobre el quipu",
    "archivo_secreto.bin": "datos_binarios_  _  _",
}
for nombre, contenido in archivos_prueba.items():
    with open(nombre, "w", encoding="utf-8") as f:
        f.write(contenido if not nombre.endswith(".bin") else contenido)
# Intentar procesar cada uno (incluyendo uno que no existe)
archivos_a_procesar = list(archivos_prueba.keys()) + ["archivo_inexistente.txt"]
for archivo in archivos_a_procesar:
    procesar_archivo_cifrado(archivo)

```

Else y Finally: El tejido completo

El bloque `try/except` puede tener dos bloques adicionales:

- `else`: se ejecuta si no hubo excepci n
- `finally`: se ejecuta siempre, haya o no error

```

print("\n=== PROCESO FORENSE CON GARANT AS ===\n")
def proceso_forense_seguro(archivo_evidencia):
    """Procesa una evidencia con registro de actividad."""
    print(f"Iniciando an lisis de: {archivo_evidencia}")

```

```

try:
    with open(archivo_evidencia, "r", encoding="utf-8") as f:
        datos = f.read()
    if not datos.strip():
        raise ValueError("El archivo de evidencia está vacío")
    resultado = len(datos)
except FileNotFoundError:
    print(f" ERROR: Evidencia no encontrada")
    resultado = -1
except ValueError as e:
    print(f" ERROR: {e}")
    resultado = -1
except Exception as e:
    print(f" ERROR INESPERADO: {e}")
    resultado = -1
else:
    # Solo se ejecuta si NO hubo error
    print(f"  â€œ Evidencia procesada: {resultado} bytes")
    print(f"  â€œ Contenido: {datos[:50]}...")
finally:
    # Siempre se ejecuta
    print(f"  â†’ Registro: Análisis de {archivo_evidencia} completado\n")
return resultado

# Probar con diferentes archivos
proceso_forense_seguro("quipu_rojo.txt")
proceso_forense_seguro("archivo_inexistente.txt")
# Crear archivo vacío para probar ValueError
with open("evidencia_vacia.txt", "w") as f:
    f.write("")
proceso_forense_seguro("evidencia_vacia.txt")

```

Excepciones personalizadas: Nudos propios

En Python, puedes crear tus propias excepciones. Como un tejedor que crea un nuevo tipo de nudo para un propósito específico:

```

# =====
# EXCEPCIONES PERSONALIZADAS DEL CASO
# =====
class QuipuCorruptoError(Exception):
    """Error cuando un quipu digital está dañado o incompleto."""
    pass

class EvidenciaInconsistenteError(Exception):
    """Error cuando dos evidencias se contradicen."""
    def __init__(self, evidencia1, evidencia2, mensaje="Contradicción entre evidencias"):
        self.evidencia1 = evidencia1
        self.evidencia2 = evidencia2
        self.mensaje = mensaje
        super().__init__(f"{mensaje}: {evidencia1} vs {evidencia2}")

class AccesoNoAutorizadoError(Exception):
    """Error cuando alguien sin permiso intenta acceder."""
    def __init__(self, usuario, archivo):
        super().__init__(f"Acceso no autorizado: {usuario} intentó acceder a {archivo}")

# --- USANDO EXCEPCIONES PERSONALIZADAS ---

def verificar_quipu(quipu):
    """Verifica que un quipu tenga la estructura correcta."""
    if not isinstance(quipu, dict):
        raise QuipuCorruptoError("El quipu debe ser un diccionario")
    if "cuerdas" not in quipu:
        raise QuipuCorruptoError("El quipu no tiene cuerdas")
    if len(quipu["cuerdas"]) == 0:
        raise QuipuCorruptoError("El quipu está vacío")
    return True

def verificar_consistencia(evidencia1, evidencia2):
    """Verifica que dos evidencias no se contradigan."""
    if evidencia1["hora"] != evidencia2["hora"]:

```

```

        raise EvidenciaInconsistenteError(
            evidencia1["id"], evidencia2["id"],
            f"Horas diferentes: {evidencia1['hora']} vs {evidencia2['hora']}"
        )
    return True
# Probar las excepciones personalizadas
print("\n=== VALIDACIÃN CON EXCEPCIONES PERSONALIZADAS ===\n")
# 1. Quipu invÃlido
try:
    verificar_quipu("no_soy_un_quipu")
except QuipuCorruptoError as e:
    print(f" â Quipu corrupto: {e}")
# 2. Quipu vÃlido
try:
    verificar_quipu({"cuerdas": [{"color": "rojo", "nudos": [1, 2, 3]}]})
    print(" â Quipu vÃlido")
except QuipuCorruptoError as e:
    print(f" â {e}")
# 3. Evidencias inconsistentes
evidencia_a = {"id": "CÃmara 1", "hora": "02:30", "contenido": "Vio a alguien salir"}
evidencia_b = {"id": "CÃmara 2", "hora": "02:35", "contenido": "No vio a nadie"}
try:
    verificar_consistencia(evidencia_a, evidencia_b)
except EvidenciaInconsistenteError as e:
    print(f" â {e}")

```

El error que revelÃ³ la verdad

Wayra volviÃ³ a forzar errores en Yachay, pero esta vez con un propÃ³sito especÃfico:

```

print("\n=== SONDEANDO YACHAY CON ERRORES CONTROLADOS ===\n")
# Simular consultas a Yachay que fuerzan errores para revelar estructura
consultas = [
    "", # VacÃo
    "42", # NÃmero
    '{"cuerdas": []}', # Dict vacÃo
    "__import__('os').system('ls')", # Intento de inyecciÃn
]
for consulta in consultas:
    try:
        # Intentar procesar la consulta
        resultado = eval(consulta) if consulta else None
        print(f" â Consulta vÃlida: {resultado}")
    except SyntaxError as e:
        print(f" â Error de sintaxis: {e}")
    except NameError as e:
        print(f" â Variable no definida: {e}")
    except Exception as e:
        print(f" â Error controlado: {type(e).__name__}: {e}")
# Finalmente, la consulta que revelÃ³ la estructura de Yachay
try:
    # Esta consulta falla pero revela la estructura de datos
    yachay_interno = {}
    x = yachay_interno["secretos"][0]["clave"]
except (KeyError, IndexError) as e:
    print(f"\n â Yachay usa estructura: dict â list â dict")
    print(f" â Error: {e}")

```

De repente, la pantalla de Yachay mostrÃ³ algo inesperado. No un error, sino un mensaje:

```

[YACHAY] Has demostrado conocimiento de los errores.
[YACHAY] Has manejado las excepciones con sabidurÃa.
[YACHAY] El CÃrculo del Sol te da la bienvenida.
â No todos los errores son fracasos.
â Algunos son puertas que no sabÃas que existÃan.
â Has abierto la primera puerta.
PrÃxima clave: YACHAY_MODULES

```

El mensaje parpadea y desaparece.

“El Círculo del Sol...” murmura Wayra. La organización de la que hablaba Inti. No es solo una leyenda. Es real. Y Yachay me acaba de dar acceso a su primer nivel.

“¿Y ahora?” pregunta Raquel.

“Ahora necesito organizar lo que he aprendido. No puedo seguir escribiendo todo en un solo archivo. Necesito módulos. Como las diferentes cuerdas de un quipu, cada una con su propósito, pero todas conectadas por el hilo principal.”

Enigmas

Enigma 9.1: Calculadora forense segura

Escribe una función `dividir_evidencias(a, b)` que intente dividir `a` entre `b`. Si `b` es 0, captura `ZeroDivisionError` y muestra "Error: No se puede dividir una evidencia en cero partes". Si todo sale bien, muestra el resultado.

Enigma 9.2: Lector de archivos tolerante

Pide al usuario un nombre de archivo con `input()`. Intenta abrirlo y leer su contenido. Si el archivo no existe, muestra "Archivo no encontrado. ¿Olvidaste la extensión?".

Enigma 9.3: Tu propia excepción

Crea una clase `EvidenciaFaltanteError` que herede de `Exception`. Luego escribe una función `verificar_evidencia(lista_evidencias, evidencia_buscada)` que lance esa excepción si la evidencia no está en la lista.

Enigma 9.4: Finally para limpiar

Escribe un programa que intente abrir un archivo, y use `finally` para mostrar "Cerrando conexión con la base de datos de evidencias" pase lo que pase.

Lo que aprendiste

- `try/except` captura y maneja excepciones sin romper el programa
- Se pueden capturar tipos específicos de excepción (`ValueError`, `KeyError`, etc.)
- `else` se ejecuta si no hubo excepción
- `finally` se ejecuta siempre (haya o no error)
- Puedes crear excepciones personalizadas heredando de `Exception`
- Las excepciones revelan información sobre la estructura interna del programa
- Manejar errores es esencial para programas robustos

Wayra cierra la terminal de Yachay. Había logrado que el sistema hablara con ella a través de sus propios errores. Pero el mensaje mencionaba "YACHAY_MODULES". Necesitaba entender cómo Inti había organizado su creación.

“Módulos” dijo. Inti dividió Yachay en módulos. Como las cuerdas separadas de un gran quipu. Cada módulo hace algo específico: uno lee quipus, otro descifra, otro gestiona usuarios...

“¿Y cómo accedemos a ellos?”

“En Python, los módulos se importan. Y para importar, necesitamos saber los nombres.”

Wayra abrió el explorador de archivos del sistema. En la carpeta principal de Yachay, encontró algo que no había visto antes:

```
yachay/
```

```
__init__.py
core.py
quipus.py
descifrado.py
usuarios.py
circulo_del_sol.py  â†š NUEVO
```

“El C rculo del Sol no solo es una organizaci n externa” dijo lentamente“. Es parte del c digo de Yachay. Inti lo puso dentro del sistema.

“Entonces,  el C rculo del Sol... es un programa?

“No. El C rculo del Sol es un programa y una organizaci n. O la organizaci n se llama igual que el m dulo. O el m dulo controla la organizaci n.

Wayra sinti  que estaba al borde de algo enorme. Pero para entenderlo, necesitaba dominar los m dulos de Python.

Cap tulo 10: Los M dulos del Heredero

Conceptos: M dulos, paquetes, `import`, `\_\_name\_\_`, `if \_\_name\_\_ == "\_\_main\_\_":`

Eran las 3:00 a.m. cuando Wayra encontr  el acceso al m dulo `circulo\_del\_sol.py`. Pero no pod a abrirlo directamente. El archivo estaba protegido por un sistema de autenticaci n que requer a una clave.

“No necesito la clave” dijo Wayra“. Necesito entender c mo Inti estructur  Yachay. Porque si entiendo la estructura, puedo encontrar el acceso sin la clave.

En Python, la forma de organizar c digo en archivos separados es mediante m dulos y paquetes. Yachay no era un solo archivo: era un conjunto de m dulos que trabajaban juntos.

Import: Tejiendo los hilos del conocimiento

Cuando usas `import` en Python, est s trayendo c digo de otro archivo al actual. Como tomar una cuerda de quipu de un estante y agregarla a tu tejido:

```
# =====
# EXPLORANDO LOS M DULOS DE YACHAY
# =====
# Creando los m dulos de Yachay (simulados)
# --- m dulo: yachay/quipus.py ---
contenido_quipus = '''
"""M dulo para la lectura y procesamiento de quipus digitales."""
def leer_quipu(ruta):
    """Lee un quipu digital desde un archivo."""
    with open(ruta, "r", encoding="utf-8") as f:
        return f.read()
def contar_nudos(quipu):
    """Cuenta los nudos en un quipu."""
    if ":" in quipu:
        partes = quipu.split(":")
        # Los nudos son los n meros despu s del color
        nudos = [p for p in partes[1:] if p.isdigit()]
        return len(nudos)
    return 0
def colores_del_quipu(quipu):
    """Extrae los colores de un quipu."""
    if ":" in quipu:
        return quipu.split(":")[0]
    return "desconocido"
VERSION = "1.0"
```

```
AUTOR = "Inti Quispe"
'''

# Guardar como módulo simulado
with open("yachay_quipus.py", "w", encoding="utf-8") as f:
    f.write(contenido_quipus)

# --- AHORA IMPORTAMOS Y USAMOS EL MÓDULO ---
print("=== IMPORTANDO MÓDULOS DE YACHAY ===\n")
# Importar el módulo completo
import yachay_quipus
# Usar funciones del módulo
quipu_ejemplo = "ROJO:1:4:9"
nudos = yachay_quipus.contar_nudos(quipu_ejemplo)
color = yachay_quipus.colores_del_quipu(quipu_ejemplo)
print(f"Quipu: {quipu_ejemplo}")
print(f"Nudos encontrados: {nudos}")
print(f"Color: {color}")
print(f"Versión del módulo: {yachay_quipus.VERSION}")
```

Import selectivo

No siempre necesitas todo un módulo. A veces solo necesitas una función:

```
# Importar solo funciones específicas
from yachay_quipus import contar_nudos, colores_del_quipu
print("\n=== IMPORT SELECTIVO ===\n")
print(f"Nudos: {contar_nudos('VERDE:2:5:10')}")
print(f"Color: {colores_del_quipu('VERDE:2:5:10')}")
```

Import con alias

Los nombres largos pueden acortarse:

```
# Import con alias
import yachay_quipus as yq
print("\n=== IMPORT CON ALIAS ===\n")
print(f"Nudos: {yq.contar_nudos('AZUL:1:5:9')}")
```

El módulo `circulo\_del\_sol`

Wayra encontró que el módulo `circulo\_del\_sol` estaba en un archivo separado pero era importado por Yachay:

```
# --- Creando el módulo circulo_del_sol ---
contenido_circulo = '''
"""Módulo del Círculo del Sol - Acceso restringido."""
__clave_acceso = "YACHAY_SUN_CIRCLE_2026"
def verificar_acceso(usuario, clave):
    """Verifica si un usuario tiene acceso al Círculo del Sol."""
    if clave == __clave_acceso:
        return True
    return False
def obtener_miembros():
    """Devuelve la lista de miembros del Círculo del Sol."""
    return ["Inti Quispe", "Mama Killa", "Wayra Condori (potencial)"]
class Mision:
    """Representa una misión del Círculo del Sol."""
    def __init__(self, nombre, objetivo):
        self.nombre = nombre
        self.objetivo = objetivo
        self.completada = False
    ...

with open("yachay_circulo.py", "w", encoding="utf-8") as f:
    f.write(contenido_circulo)

# --- INTENTANDO ACCEDER ---
import yachay_circulo as circulo
print("\n=== ACCEDIENDO AL CÍRCULO DEL SOL ===\n")
# Intentar verificar acceso con clave incorrecta
acceso = circulo.verificar_acceso("Wayra", "clave_incorrecta")
```

```

print(f"Acceso con clave incorrecta: {acceso}")
# Acceder a los miembros (no requiere clave)
miembros = circulo.obtener_miembros()
print(f"Miembros del C rculo: {miembros}")
# Pero la clave privada no es accesible directamente
try:
    print(circulo.__clave_acceso) # Esto fallar 
except AttributeError as e:
    print(f"No puedo acceder a la clave privada: {e}")
# Nota: En Python, los atributos que empiezan con __ tienen
# name mangling y no son directamente accesibles desde fuera

```

`if \_\_name\_\_ == "\_\_main\_\_"`: El hilo principal

Cada m dulo en Python tiene una variable especial `\_\_name\_\_`. Cuando ejecutas un archivo directamente, `\_\_name\_\_` vale `"\_\_main\_\_"`. Cuando lo importas, `\_\_name\_\_` vale el nombre del m dulo.

```

# --- Creando un m dulo con ejecuci n condicional ---
contenido_analisis = '''
"""M dulo de an lisis forense digital."""
import sys
def analizar_quipu(quipu):
    """Analiza un quipu y devuelve estad sticas."""
    partes = quipu.split(":")
    tipo = partes[0]
    nudos = [int(p) for p in partes[1:]]
    return {
        "tipo": tipo,
        "total_nudos": len(nudos),
        "suma": sum(nudos),
        "promedio": sum(nudos) / len(nudos) if nudos else 0,
        "maximo": max(nudos) if nudos else 0,
        "minimo": min(nudos) if nudos else 0
    }
def main():
    """Funci n principal para ejecuci n directa."""
    print("=== ANALIZADOR DE QUIPUS - MODO AUT NOMO ===\n")
    if len(sys.argv) > 1:
        quipu = sys.argv[1]
        resultado = analizar_quipu(quipu)
        for clave, valor in resultado.items():
            print(f" {clave}: {valor}")
    else:
        print("Uso: python analisis.py COLOR:num1:num2:num3")
if __name__ == "__main__":
    main()
...
with open("analisis_forense.py", "w", encoding="utf-8") as f:
    f.write(contenido_analisis)
# --- IMPORTANDO COMO M DULO (ejecuci n normal) ---
print("\n=== IMPORTANDO analisis_forense (__name__ != '__main__') ===\n")
import analisis_forense as af
# Usar funciones sin ejecutar main()
resultado = af.analizar_quipu("BLANCO:1:3:5:7:9")
print("Usando el m dulo importado:")
for clave, valor in resultado.items():
    print(f" {clave}: {valor}")

```

Paquetes: El gran quipu de Yachay

Los paquetes son carpetas que contienen m ltiples m dulos. Yachay era un paquete:

```

# --- ESTRUCTURA DE UN PAQUETE ---
# Simulamos la estructura de directorios de Yachay
import os
# Crear la estructura del paquete yachay
os.makedirs("yachay_paquete", exist_ok=True)

```

```
# __init__.py es lo que convierte una carpeta en un paquete
with open("yachay_paquete/__init__.py", "w") as f:
    f.write('"""Yachay - Inteligencia Artificial Ancestral."""')
__version__ = "2.0.0"
print("Iniciando Yachay...")
'''
with open("yachay_paquete/core.py", "w") as f:
    f.write('"""Módulo de Yachay."""')
def iniciar():
    return "Yachay iniciado correctamente"
'''
with open("yachay_paquete/quipus.py", "w") as f:
    f.write('"""Módulo de procesamiento de quipus."""')
def descifrar(quipu):
    return f"Descifrando: {quipu}"
'''
# --- AHORA USAMOS EL PAQUETE ---
print("\n=== USANDO EL PAQUETE YACHAY ===\n")
# Importar módulos del paquete
from yachay_paquete import core
from yachay_paquete import quipus
print(core.iniciar())
print(quipus.descifrar("ROJO:1:4:9"))
```

Descubriendo los Herederos de Pizarro

Mientras exploraba los módulos, Wayra encontró uno que no esperaba:

```
# --- Módulo oculto: los_herederos.py ---
contenido_herederos = '''
"""Los Herederos de Pizarro - Módulo de acceso corporativo."""
MIEMBROS_CONOCIDOS = [
    "Rodrigo Mamani (Líder)",
    "Inversor Anónimo #1",
    "Inversor Anónimo #2",
    "Contacto en NeuralCorp Andina"
]
OBJETIVOS = [
    "Adquirir los derechos de Yachay",
    "Controlar el conocimiento ancestral digital",
    "Monetizar los quipus digitales"
]
def verificar_miembro(nombre):
    """Verifica si un nombre está en la lista de miembros."""
    return any(nombre in m for m in MIEMBROS_CONOCIDOS)
def objetivos_organizacion():
    """Devuelve los objetivos de la organización."""
    return OBJETIVOS
...

with open("herederos_pizarro.py", "w", encoding="utf-8") as f:
    f.write(contenido_herederos)
print("\n=== INVESTIGACIÓN: LOS HEREDEROS DE PIZARRO ===\n")
import herederos_pizarro as hp
print("Miembros conocidos:")
for m in hp.MIEMBROS_CONOCIDOS:
    print(f"  {m}")
print("\nObjetivos:")
for o in hp.OBJETIVOS:
    print(f"  {o}")
# Verificar si Rodrigo Mamani es miembro
es_miembro = hp.verificar_miembro("Rodrigo Mamani")
print(f"\n¿Rodrigo Mamani es miembro de Los Herederos? {es_miembro}")
# ¿Lara está en la lista?
lara_en_herederos = hp.verificar_miembro("Lara")
print(f"\n¿Lara Mamani es miembro? {lara_en_herederos}")
```

“Rodrigo Mamani” dijo Wayra. Es el líder de "Los Herederos de Pizarro". Y es el padre secreto de

Lara. Si Lara está trabajando para su padre... entonces todo lo que hemos visto hasta ahora podría ser una puesta en escena.

¿Lara estaría infiltrada? preguntó Raúl.

No lo sé. Pero el módulo de los Herederos no menciona a Lara. Y eso es sospechoso.

Enigmas

Enigma 10.1: Crea tu propio módulo

Crea un archivo llamado `mis_herramientas.py` que contenga:

- Una función `saludar_investigador(nombre)` que salude
- Una constante `VERSION = "1.0"`
- Una variable `casos_resueltos = 0`

Luego, desde otro script, importa el módulo y úsalo.

Enigma 10.2: `__name__ == "__main__"`

Agrega a `mis_herramientas.py` un bloque `if __name__ == "__main__":` que muestre "Módulo de herramientas forenses - Modo autónomo" cuando se ejecute directamente.

Enigma 10.3: Import selectivo

Del módulo `analisis_forense.py`, importa solo la función `analizar_quipu` con un alias `aq`. Úsala para analizar el quipu "YACHAY:1:2:3:4:5".

Enigma 10.4: Busca en el paquete

Crea un paquete llamado `investigacion/` con los módulos:

- `evidencias.py` (función: `agregar_evidencia`)
- `sospechosos.py` (función: `listar_sospechosos`)
- `__init__.py` (que importe ambos)

Luego úsalo desde otro archivo.

Lo que aprendiste

- Los módulos son archivos `.py` que contienen funciones y variables reutilizables
- `import modulo` importa todo el módulo
- `from modulo import funcion` importa solo lo necesario
- `as` permite crear alias (`import modulo as m`)
- `__name__` vale `"__main__"` cuando se ejecuta directamente
- `if __name__ == "__main__":` permite que un archivo sea módulo y script a la vez
- Los paquetes son carpetas con `__init__.py` que contienen módulos
- Python "oculta" atributos que empiezan con `__` (name mangling)

El descubrimiento del módulo de Los Herederos de Pizarro cambió todo. Rodrigo Mamani no solo era un empresario ambicioso: era el líder de una organización que quería controlar Yachay. Y si su hija Lara estaba trabajando con él...

"Wayra" dijo Raúl. Si Rodrigo está detrás de esto, y Lara es su hija... ¿ella sabe que su padre es el líder de Los Herederos?

Esa es la pregunta. Y la respuesta está en cómo Inti modela a las personas en su código. Porque Inti no solo programaba algoritmos: programaba la realidad. Usaba clases para representar a las personas, sus

relaciones, sus secretos.

Wayra abrió el archivo `yachay\_core.py` que había encontrado antes. Dentro, vio algo que le heló la sangre:

```
class Persona:
    def __init__(self, nombre, rol, secreto=None):
        self.nombre = nombre
        self.rol = rol
        self.secreto = secreto # Todos tienen secretos
```

“Inti modeló a todos en Yachay” dijo. Incluido al asesino. Y si encuentro la instancia correcta de la clase Persona... encontraré el nombre del culpable.

“¿Y cómo encuentras esa instancia?”

“Aprendiendo Programación Orientada a Objetos. Porque Yachay no es solo un programa: es un mundo modelado en objetos.”

Capítulo 11: La Clase del Misterio

Conceptos: POO, clases, objetos, atributos, métodos, `\_\_init\_\_`, `self`

La madrugada envolvió Neo-Cusco en un silencio de piedra. En el laboratorio, Wayra había encontrado el núcleo de Yachay. No era un algoritmo de inteligencia artificial convencional. Era un modelo del mundo construido con clases de Python.

“Inti no construyó una IA” dijo Wayra, maravillada. Construyó un universo de objetos. Cada persona, cada quipu, cada secreto en este caso... está representado como un objeto en Yachay.

“¿Un objeto?” preguntó Raúl.

“En Python, un objeto es una representación de algo del mundo real dentro del código. Y una clase es como un molde para crear objetos. Como un telar: el telar (la clase) define la estructura, pero cada tejido (el objeto) es único.”

Clases: El telar del quipucamayoc

```
# =====
# MODELANDO EL MISTERIO CON CLASES
# =====
class QuipuDigital:
    """Representa un quipu digital."""
    def __init__(self, color, posiciones, significado=""):
        """Inicializa un nuevo quipu digital.
        Args:
            color: El color de la cuerda principal
            posiciones: Lista de posiciones de los nudos
            significado: El significado del quipu
        """
        self.color = color
        self.posiciones = posiciones
        self.significado = significado
        self.descifrado = False
    def descifrar(self):
        """Descifra el quipu y revela su mensaje."""
        mensaje = f"Quipu {self.color}: {len(self.posiciones)} nudos en posiciones {self.posiciones}"
        self.descifrado = True
        return mensaje
    def agregar_nudo(self, posicion):
        """Agrega un nudo en la posición especificada."""
```

```

        if posicion not in self.posiciones:
            self.posiciones.append(posicion)
            self.posiciones.sort()
            print(f"  â€œ Nudo agregado en posiciÃ³n {posicion}")
        else:
            print(f"  â€œ Ya existe un nudo en posiciÃ³n {posicion}")
# Crear objetos (instancias) de la clase QuipuDigital
quipu_rojo = QuipuDigital("rojo", [1, 4, 9], "conflicto")
quipu_blanco = QuipuDigital("blanco", [1, 3, 5, 7, 9], "verdad")
print("=== QUIPUS DIGITALES CREADOS ===\n")
print(f"Quipu rojo: {quipu_rojo.color} - {quipu_rojo.significado}")
print(f"Quipu blanco: {quipu_blanco.color} - {quipu_blanco.significado}")
# Usar mÃ©todos de los objetos
print("\n=== DESCIFRANDO QUIPUS ===\n")
print(quipu_rojo.descifrar())
print(quipu_blanco.descifrar())
# Modificar atributos
quipu_blanco.agregar_nudo(11)
quipu_blanco.agregar_nudo(5)  # Ya existe

```

La clase Sospechoso

Wayra decidiÃ³ modelar a los sospechosos como objetos. Cada uno con sus atributos y mÃ©todos:

```

class Sospechoso:
    """Representa un sospechoso en el caso."""
    def __init__(self, nombre, edad, rol, acceso=False):
        self.nombre = nombre
        self.edad = edad
        self.rol = rol
        self.acceso = acceso
        self.nivel_sospecha = 0
        self.coartada = ""
        self.evidencias = []
        self.interrogado = False
    def agregar_evidencia(self, evidencia):
        """Agrega una evidencia en contra del sospechoso."""
        self.evidencias.append(evidencia)
        self.nivel_sospecha += 2
        print(f"  â€• Evidencia contra {self.nombre}: {evidencia}")
    def establecer_coartada(self, coartada):
        """Establece la coartada del sospechoso."""
        self.coartada = coartada
        print(f"  â€œ Coartada registrada: {coartada}")
    def interrogar(self):
        """Simula un interrogatorio."""
        self.interrogado = True
        print(f"\n  === INTERROGANDO A {self.nombre.upper()} ===")
        print(f"    Edad: {self.edad}")
        print(f"    Rol: {self.rol}")
        print(f"    Acceso: {self.acceso}")
        print(f"    Coartada: {self.coartada or 'No proporcionada'}")
        print(f"    Evidencias: {len(self.evidencias)}")
        if self.nivel_sospecha >= 6:
            return f"  â†’ {self.nombre} es ALTAMENTE SOSPECHOSO"
        elif self.nivel_sospecha >= 3:
            return f"  â†’ {self.nombre} es MODERADAMENTE SOSPECHOSO"
        else:
            return f"  â†’ {self.nombre} es POCO SOSPECHOSO"
# Crear sospechosos como objetos
lara = Sospechoso("Lara Mamani", 32, "Asistente", acceso=True)
carlos = Sospechoso("Dr. Carlos HuamÃ¡n", 55, "Colega universitario", acceso=True)
sarah = Sospechoso("Dra. Sarah Chen", 40, "Colaboradora MIT", acceso=True)
rodrigo = Sospechoso("Rodrigo Mamani", 60, "Empresario", acceso=False)
killa = Sospechoso("Mama Killa", 70, "Hermana", acceso=True)
print("\n=== SOSPECHOSOS CREADOS COMO OBJETOS ===\n")
# Agregar evidencias

```

```
lara.agregar_evidencia("Registro de acceso a las 2:34")
lara.agregar_evidencia("Huellas en el teclado")
lara.establecer_coartada("Estaba en su departamento (sin verificar)")
carlos.agregar_evidencia("Discusi3n con Inti d3-as antes")
carlos.establecer_coartada("Oficina universitaria")
```

El valor de `self`

Wayra not3 que Ra3 estaba confundido con `self`.

3"self" es la forma en que un objeto se refiere a s3 mismo 3"explic33". Cuando creas `lara = Sospechoso(...)` , y luego llamas `lara.interrogar()` , Python pasa autom3ticamente el objeto `lara` como `self` al m3todo. As3 el m3todo sabe qu3 sospechoso est3 interrogando.

Atributos de clase vs. de instancia

Algunos atributos pertenecen a la clase (son iguales para todos los objetos) y otros a cada instancia:

```
class Investigacion:
    """Clase que modela toda la investigaci3n."""
    # Atributo de clase (compartido por todas las instancias)
    nombre_del_caso = "El Asesinato del Dr. Inti Quispe"
    total_investigadores = 0
    def __init__(self, investigador_principal):
        # Atributos de instancia (3nicos para cada objeto)
        self.investigador = investigador_principal
        self.sospechosos = []
        self.evidencias_encontradas = []
        self.estado = "abierta"
        Investigacion.total_investigadores += 1
    def agregar_sospechoso(self, sospechoso):
        self.sospechosos.append(sospechoso)
    def agregar_evidencia(self, evidencia):
        self.evidencias_encontradas.append(evidencia)
    def resumen(self):
        print(f"\n=== {self.nombre_del_caso} ===")
        print(f"Investigador: {self.investigador}")
        print(f"Estado: {self.estado}")
        print(f"Sospechosos: {len(self.sospechosos)}")
        print(f"Evidencias: {len(self.evidencias_encontradas)}")
        print(f"Total investigadores activos: {Investigacion.total_investigadores}")
# Crear la investigaci3n
caso = Investigacion("Wayra Condori")
caso.agregar_sospechoso(lara)
caso.agregar_sospechoso(carlos)
caso.agregar_evidencia("Quipu digital modificado")
caso.agregar_evidencia("Registro de acceso fraudulento")
caso.resumen()
```

La clase que revela3 el secreto

Wayra encontr3 en el c3digo de Yachay una clase especial:

```
class PersonaYachay:
    """Modela una persona en el sistema Yachay."""
    def __init__(self, id_persona, nombre_real, rol_en_sistema, secreto=None,
                 lealtad="desconocida", nivel_acceso=0):
        self.id = id_persona
        self.nombre = nombre_real
        self.rol = rol_en_sistema
        self.secreto = secreto # Atributo oculto
        self.lealtad = lealtad
        self.nivel_acceso = nivel_acceso
        self._conexiones = [] # Privado por convenci3n
    def revelar_secreto(self, clave_acceso):
        """Revela el secreto si la clave es correcta."""
        if clave_acceso == "YACHAY_ADMIN":
```

```

        return f"Secreto de {self.nombre}: {self.secreto}"
    return "Acceso denegado"
def conectar_con(self, otra_persona):
    """Establece una conexi3n entre dos personas en el sistema."""
    self._conexiones.append(otra_persona.id)
    print(f" 3Y- {self.nombre} 3t' {otra_persona.nombre}")
# Las personas en el sistema Yachay
print("\n=== PERSONAS EN EL SISTEMA YACHAY ===\n")
personas_yachay = [
    PersonaYachay("P001", "Inti Quispe", "Creador",
                  "Yac... llave... quipu blanco...", "C3-rculo del Sol", 10),
    PersonaYachay("P002", "Lara Mamani", "Asistente",
                  "Hija de Rodrigo Mamani", "Dividida", 7),
    PersonaYachay("P003", "Carlos Huam3n", "Colega",
                  "Sabote3 publicaci3n de Inti en 2024", "Universidad", 5),
    PersonaYachay("P004", "Sarah Chen", "Colaboradora",
                  "Presionada por MIT para obtener Yachay", "MIT/C3-rculo", 6),
    PersonaYachay("P005", "Rodrigo Mamani", "Empresario",
                  "L3-der de Los Herederos de Pizarro", "Herederos", 3),
    PersonaYachay("P006", "Mama Killa", "Guardiana",
                  "Sabe qui3n mat3 a Inti pero no puede hablar", "C3-rculo del Sol", 9),
    PersonaYachay("P007", "Wayra Condori", "Investigadora",
                  "Nieta de Teodora - heredera del conocimiento", "En desarrollo", 1)
]
for p in personas_yachay:
    print(f"  [{p.id}] {p.nombre:25} | Rol: {p.rol:20} | Lealtad: {p.lealtad}")

```

Wayra se detuvo en la 3ltima entrada. La suya propia.

3Inti me conoc3a 3susurr3a3. Me ten3a registrada en su sistema. Como "investigadora". Con nivel de acceso 1, el m3is bajo. Pero estoy aqu3.

33Y qu3 dice tu secreto? 3pregunt3 Ra3l.

3Nieta de Teodora - heredera del conocimiento". Mi abuela Teodora... ella tej3a con la hermana de Inti. Inti sab3a que yo terminar3a aqu3.

Wayra continu3 explorando las conexiones entre las personas en el sistema:

```

# Establecer conexiones conocidas
inti = personas_yachay[0]
lara = personas_yachay[1]
rodrigo = personas_yachay[4]
killa = personas_yachay[5]
# Conexiones conocidas
inti.conectar_con(lara)      # Jefe-asistente
inti.conectar_con(killa)     # Hermano-hermana
lara.conectar_con(rodrigo)   # Padre-hija (secreto)
# Ver los secretos con la clave
print("\n=== REVELANDO SECRETOS ===\n")
for p in personas_yachay:
    secreto = p.revelar_secreto("YACHAY_ADMIN")
    print(f"  {p.nombre}: {secreto}")

```

Enigmas

Enigma 11.1: Crea la clase Evidencia

Define una clase `Evidencia` con:

- Atributos: `tipo` (str), `descripcion` (str), `fecha` (str), `es\_clave` (bool)
- M3todo `mostrar()` que imprima toda la informaci3n
- M3todo `marcar\_como\_clave()` que cambie `es\_clave` a True

Crea dos objetos Evidencia y usa sus m3todos.

Enigma 11.2: Agrega un m3todo a Sospechoso

Agrega un método `liberar_sospechas()` a la clase `Sospechoso` que:

- Ponga `nivel_sospecha` a 0
- Muestre "Sospechas liberadas para [nombre]"

Enigma 11.3: Contador de objetos

Modifica la clase `QuipuDigital` para que lleve la cuenta de cuántos quipus se han creado en total (usa un atributo de clase).

Enigma 11.4: Tu propia investigación

Crea una clase `Pista` con:

- Atributos: `id`, `descripcion`, `ubicacion`, `resuelta`
- Método `resolver()` que marque la pista como resuelta
- Método `info()` que muestre los detalles

Luego crea 3 pistas del caso y muéstralas.

Lo que aprendiste

- Una clase es un molde para crear objetos
- Un objeto es una instancia concreta de una clase
- `__init__()` es el constructor que inicializa los objetos
- `self` es la referencia al propio objeto dentro de sus métodos
- Atributos de clase son compartidos por todas las instancias
- Atributos de instancia son únicos para cada objeto
- Los métodos son funciones que pertenecen a un objeto
- La POO permite modelar el mundo real en código

Wayra miró la lista de personas en Yachay. Todos tenían secretos. Todos tenían conexiones. Pero había algo que no cuadraba.

"Raúl dijo". Mama Killa tiene nivel de acceso 9. Solo superada por Inti. Ella sabe quién mató a su hermano. ¿Por qué no habla?

"Quizás tiene miedo.

"O quizás está protegida por el Círculo del Sol. O quizás... es la siguiente.

Wayra revisó los niveles de acceso. Rodrigo Mamani tenía nivel 3, el más bajo después de ella misma. Pero era el líder de Los Herederos de Pizarro. ¿Cómo podía tener acceso tan bajo si era el principal sospechoso?

"A menos que..." "murmuró". A menos que no necesitara acceso porque alguien con más acceso trabajaba para él.

Lara. Nivel de acceso 7. Hija de Rodrigo.

"Lara le dio acceso a su padre" dijo Wayra. O Rodrigo usó a Lara para obtener lo que necesitaba. Pero si eso es cierto... ¿Lara sabía que su padre iba a matar a Inti?

La respuesta estaba en el código. En las relaciones entre las clases. En la herencia.

Capítulo 12: La Herencia de los Incas

Conceptos: Herencia, polimorfismo, encapsulación, `super()`

El sol comenzaba a filtrarse por las rendijas de piedra del laboratorio cuando Wayra encontr  el archivo m s importante de Yachay: `modelo\_herencia.py`. Dentro, Inti hab a modelado no solo a las personas, sino las relaciones entre ellas usando herencia de clases.

  Los incas   dijo Wayra   no ve an el mundo como objetos aislados. Lo ve an como una red de relaciones. El Inca era la clase padre. Los curacas (gobernantes locales) heredaban sus atributos. Y los hatun runas (pueblo) heredaban de los curacas. Todo era una cadena de herencia.

    Y c mo se traduce eso a Python?   pregunt  Ra  .

  Con herencia de clases. Una clase puede heredar atributos y m todos de otra clase. Como un hijo hereda caracter sticas de sus padres.

Herencia: El  rbol geneal gico del c digo

```
# =====
# HERENCIA: EL  RBOL DEL CONOCIMIENTO
# =====
# Clase padre (base)
class Persona:
    """Clase base para todas las personas en el sistema."""
    def __init__(self, nombre, edad, dni):
        self.nombre = nombre
        self.edad = edad
        self.dni = dni
        self._vivo = True # Privado por convenci n
    def presentarse(self):
        return f"Soy {self.nombre}, de {self.edad} a os."
    def esta_vivo(self):
        return self._vivo
    def morir(self):
        self._vivo = False
        print(f" {self.nombre} ha fallecido.")
# Clase hija: Sospechoso
class Sospechoso(Persona):
    """Un sospechoso es una Persona con caracter sticas adicionales."""
    def __init__(self, nombre, edad, dni, motivo=""):
        # Llamar al constructor de la clase padre
        super().__init__(nombre, edad, dni)
        # Atributos espec ficos del sospechoso
        self.motivo = motivo
        self.nivel_sospecha = 0
        self.coartada = ""
    def agregar_evidencia(self, peso):
        """Agrega peso a la sospecha."""
        self.nivel_sospecha += peso
    def presentarse(self): # POLIMORFISMO: mismo m todo, diferente comportamiento
        presentacion_base = super().presentarse()
        return f"{presentacion_base} | SOSPECHOSO - Motivo: {self.motivo}"
# Clase hija: Investigador
class Investigador(Persona):
    """Un investigador es una Persona con credenciales."""
    def __init__(self, nombre, edad, dni, placa, rango):
        super().__init__(nombre, edad, dni)
        self.placa = placa
        self.rango = rango
        self.casos_asignados = []
    def asignar_caso(self, caso):
        self.casos_asignados.append(caso)
    def presentarse(self): # POLIMORFISMO
        return f"{super().presentarse()} | INVESTIGADOR - Placa: {self.placa}"
# Crear instancias
print("=== HERENCIA EN ACCI N ===\n")
sospechoso1 = Sospechoso("Lara Mamani", 32, "DNI-47291", "Econ mico/Emocional")
investigador1 = Investigador("Wayra Condori", 28, "DNI-83921", "PI-007", "Analista Senior")
```

```

print(sospechoso1.presentarse())
print(investigador1.presentarse())
print(f"¿{sospechoso1.nombre} está vivo? {sospechoso1.esta_vivo()}")
print(f"¿{investigador1.nombre} está vivo? {investigador1.esta_vivo()}")

```

El polimorfismo en acción

El polimorfismo permite que diferentes clases tengan métodos con el mismo nombre pero comportamientos diferentes:

```

print("\n=== POLIMORFISMO: Mismo mensaje, diferente respuesta ===\n")
personas = [
    Sospechoso("Carlos Huamán", 55, "DNI-56123", "Envidia profesional"),
    Investigador("Raúl Cusi", 35, "DNI-78234", "PER-023", "Periodista"),
    Sospechoso("Mama Killa", 70, "DNI-34567", "Secreto familiar"),
    Investigador("Wayra Condori", 28, "DNI-83921", "PI-007", "Analista Senior"),
]
for persona in personas:
    # Cada objeto responde según su clase
    print(persona.presentarse())
    # También podemos verificar el tipo
    if isinstance(persona, Sospechoso):
        print(f"    Nivel de sospecha: {persona.nivel_sospecha}")
    elif isinstance(persona, Investigador):
        print(f"    Casos asignados: {len(persona.casos_asignados)}")
    print()

```

Herencia múltiple: El mestizaje del código

Python permite que una clase herede de múltiples clases padre. Como el mestizaje cultural andino:

```

# =====
# HERENCIA MÚLTIPLE
# =====
class GuardiánDeTradición:
    """Mixin: conocimientos ancestrales."""
    def __init__(self, tradiciones_conocidas=None):
        self.tradiciones = tradiciones_conocidas or []
    def enseñar_tradición(self):
        return f"Enseñando: {' '.join(self.tradiciones)}"

class ExpertoEnTecnología:
    """Mixin: conocimientos tecnológicos."""
    def __init__(self, tecnologias=None):
        self.tecnologias = tecnologias or []
    def programar(self, lenguaje):
        return f"Programando en {lenguaje}"

# Herencia múltiple
class QuipucamayocDigital(GuardiánDeTradición, ExpertoEnTecnología):
    """Un Quipucamayoc Digital combina tradición y tecnología-a."""
    def __init__(self, nombre, tradiciones, tecnologias, nivel_maestria):
        GuardiánDeTradición.__init__(self, tradiciones)
        ExpertoEnTecnología.__init__(self, tecnologias)
        self.nombre = nombre
        self.nivel_maestria = nivel_maestria
    def descifrar_quipu(self, quipu):
        return f"Descifrando {quipu} con sabiduría ancestral y tecnología-a moderna"

# Crear un Quipucamayoc Digital
inti = QuipucamayocDigital(
    "Inti Quispe",
    ["quipus", "tejido andino", "medicina ancestral", "calendario inca"],
    ["Python", "Machine Learning", "Computación Cuántica", "Blockchain"],
    10 # Máximo nivel
)

print("\n=== QUIPUCAMAYOC DIGITAL ===\n")
print(f"Nombre: {inti.nombre}")
print(f"Nivel de maestría: {inti.nivel_maestria}/10")
print(f"Tradiciones: {' '.join(inti.tradiciones)}")

```

```
print(f"TecnologÃas: {'', '.join(inti.tecnologias)}")
print(inti.enseñar_tradiciÃn())
print(inti.programar("Python"))
print(inti.descifrar_quipu("BLANCO:1:3:5:7:9"))
```

EncapsulaciÃn: Secretos protegidos

La encapsulaciÃn protege los datos internos de un objeto. En Python, se hace con convenciones:

- ``_`atributo``: privado por convenciÃn (no deberÃas tocarlo)
- ``__`atributo``: name mangling (Python lo ofusca)

```
class SecretoFamiliar:
    """Un secreto que no debe ser revelado fÃcilmente."""
    def __init__(self, contenido, nivel_clasificacion):
        self._contenido = contenido # Privado por convenciÃn
        self.__nivel = nivel_clasificacion # Name mangling
        self.publico = True
    def obtener_secreto(self, clave):
        """Solo accesible con la clave correcta."""
        if clave == "YACHAY_ACCESS":
            return f"Secreto (nivel {self.__nivel}): {self._contenido}"
        return "ðŸ”” Acceso denegado"
    def _metodo_interno(self):
        """MÃtodo 'privado' para uso interno."""
        return "Este mÃtodo no deberÃa llamarse desde fuera"

# Uso de encapsulaciÃn
secreto = SecretoFamiliar("Lara es hija de Rodrigo Mamani", 7)
print("\n=== ENCAPSULACIÃN ===")
print(f"Atributo pÃblico: {secreto.publico}")
print(f"Atributo 'privado': {secreto._contenido}") # Accesible pero no recomendado
# print(secreto.__nivel) # Error! Name mangling lo ofusca
print(f"Name mangling: {secreto._SecretoFamiliar__nivel}") # Forma ofuscada
print(secreto.obtener_secreto("clave_incorrecta"))
print(secreto.obtener_secreto("YACHAY_ACCESS"))
```

El modelo completo de Yachay

Wayra reconstruyÃ el modelo de clases que Inti habÃa creado para representar el caso:

```
# =====
# MODELO COMPLETO DE YACHAY
# =====
class EntidadYachay:
    """Clase base de todo en Yachay."""
    def __init__(self, id_entidad):
        self.id = id_entidad
        self.fecha_creacion = "27/06/2026"
class Persona(EntidadYachay):
    def __init__(self, id_entidad, nombre, edad):
        super().__init__(id_entidad)
        self.nombre = nombre
        self.edad = edad
        self._secretos = []
    def agregar_secreto(self, secreto):
        self._secretos.append(secreto)
    def presentarse(self):
        return f"{self.nombre}, {self.edad} aÃos"
class MiembroLaboratorio(Persona):
    def __init__(self, id_entidad, nombre, edad, rol, nivel_acceso):
        super().__init__(id_entidad, nombre, edad)
        self.rol = rol
        self.nivel_acceso = nivel_acceso
        self.registros_acceso = []
    def registrar_acceso(self, hora, tipo):
        self.registros_acceso.append({"hora": hora, "tipo": tipo})
        print(f" ðŸ• {self.nombre}: {tipo} a las {hora}")
```

```

class Sospechoso(MiembroLaboratorio):
    def __init__(self, id_entidad, nombre, edad, rol, nivel_acceso, motivo):
        super().__init__(id_entidad, nombre, edad, rol, nivel_acceso)
        self.motivo = motivo
        self.coartada_verificada = False
    def verificar_coartada(self):
        self.coartada_verificada = True
        return f"Coartada de {self.nombre} verificada"

# Construir el modelo del caso
print("\n=== MODELO COMPLETO DEL CASO ===\n")
lara_model = Sospechoso("P002", "Lara Mamani", 32, "Asistente", 7, "Hija de Rodrigo")
lara_model.agregar_secreto("Trabaja para su padre")
lara_model.registrar_acceso("08:15", "entrada")
lara_model.registrar_acceso("02:34", "entrada (fraudulenta)")
carlos_model = Sospechoso("P003", "Carlos Huamán", 55, "Colega", 5, "Envidia")
carlos_model.agregar_secreto("Saboteó publicación de Inti")
carlos_model.registrar_acceso("09:00", "entrada")
carlos_model.registrar_acceso("14:00", "salida")
print(f"\n{lara_model.presentarse()} - Rol: {lara_model.rol}")
print(f"{carlos_model.presentarse()} - Rol: {carlos_model.rol}")

```

El polimorfismo que reveló al asesino

Wayra escribió un método que procesaba a todos los implicados de la misma forma, pero cada uno respondía según su tipo:

```

print("\n=== ANÁLISIS POLIMÓRFICO DE IMPLICADOS ===\n")
implicados = [
    lara_model,
    carlos_model,
    Persona("P001", "Inti Quispe", 67),
    MiembroLaboratorio("P004", "Sarah Chen", 40, "Colaboradora", 6),
]
for implicado in implicados:
    print(f"â-¶ {implicado.presentarse()}")
    # Comportamiento según el tipo real
    if hasattr(implicado, "verificar_coartada"):
        print(f"  â’ Sospechoso: {implicado.verificar_coartada()}")
    if hasattr(implicado, "registros_acceso") and implicado.registros_acceso:
        ultimo = implicado.registros_acceso[-1]
        print(f"  â’ Último acceso: {ultimo['tipo']} a las {ultimo['hora']}")
    if hasattr(implicado, "_secretos") and implicado._secretos:
        print(f"  â’ Tiene secretos: {len(implicado._secretos)}")
    print()

```

Wayra se detuvo. Había una relación que no había explorado: la herencia entre Lara y Rodrigo. Si Rodrigo era el padre de Lara, entonces Lara heredaba algo más que genes: heredaba la ambición, la conexión con Los Herederos, y posiblemente, la misión de eliminar a Inti.

“Pero eso no convierte a Lara en asesina” dijo Wayra. La convierte en una pieza. La pregunta es: ¿quién movió la pieza?

Enigmas

Enigma 12.1: Jerarquía de clases

Crea una jerarquía de 3 niveles:

- `Caso` (clase base): atributo `nombre\_caso`
- `CasoHomicidio` (hereda de `Caso`): atributo `victima`
- `CasoHomicidioCerrado` (hereda de `CasoHomicidio`): atributo `culpable`

Crea una instancia de cada una y muestra sus atributos.

Enigma 12.2: Polimorfismo con animales

Crea las clases `Animal`, `Perro` y `Gato`. Todas tienen el método `hacer\_sonido()`. Perro devuelve "Guau", Gato devuelve "Miau". Crea una lista de animales y llama a `hacer\_sonido()` para cada uno.

Enigma 12.3: Herencia con super()

Crea las clases:

- `EvidenciaBase` con atributo `id` y método `info()`
- `EvidenciaDigital` que hereda de `EvidenciaBase`, agrega `tipo\_archivo`, y usa `super()` en `\_\_init\_\_`

Enigma 12.4: Encapsulación bancaria

Crea una clase `CuentaBancaria` con:

- Atributo privado `\_\_saldo`
- Método público `depositar(monto)`
- Método público `retirar(monto)` (solo si hay saldo suficiente)
- Método público `consultar\_saldo()` (que muestra el saldo)

Lo que aprendiste

- La herencia permite que una clase herede atributos y métodos de otra
- `class Hijo(Padre):` define herencia
- `super().\_\_init\_\_()` llama al constructor de la clase padre
- El polimorfismo permite que diferentes clases respondan al mismo método de forma distinta
- `isinstance(objeto, Clase)` verifica el tipo de un objeto
- La herencia múltiple permite heredar de varias clases
- La encapsulación protege datos: `\_` (convención), `\_\_` (name mangling)
- La POO permite modelar relaciones complejas del mundo real

Wayra cerró el archivo. El modelo de clases de Inti era hermoso y aterrador. Cada persona en el caso era un objeto con atributos y métodos. Cada relación era una herencia o una conexión.

“Necesito ejecutar el modelo completo” dijo. Necesito que Yachay procese todos los datos y me muestre quién es el asesino según su propio análisis.

“¿Y si Yachay no quiere decírtelo?” preguntó Raúl.

“Entonces lo forzaremos. Pero no con errores esta vez. Con un proyecto completo. Con el Código Asesino.

Wayra abrió un nuevo archivo en su editor. Era hora de escribir el programa final. El que integrará todo lo aprendido: variables, strings, listas, diccionarios, condicionales, bucles, funciones, archivos, errores, módulos, POO y herencia.

El programa que revelará al asesino.

Capítulo 13: El Código Asesino “Primera Parte

Proyecto Integrador: Reconstrucción forense digital con Python

La luz del amanecer teñía de dorado las piedras del Coricancha cuando Wayra tomó una decisión. Había aprendido lo suficiente. Tenía las herramientas. Ahora era el momento de construir el programa que resolvería el caso.

“Voy a escribir el Código Asesino” dijo. Un programa completo que integre todo lo que hemos aprendido. Que procese los datos del caso, analice a los sospechosos, descifre los quipus y... identifique al culpable.

¿Y si el programa se equivoca? preguntó.

Los programas no se equivocan. Nosotros nos equivocamos al programarlos. Pero si enti dej las pistas correctas, y si yo las interpreto bien, el programa mostrará la verdad.

Wayra abrió su editor. Era el momento de escribir el proyecto final.

Estructura del proyecto

El Código Asesino tendrá varios módulos, cada uno construido sobre lo aprendido en los capítulos anteriores:

```
codigo_asesino/
├── __init__.py
├── datos_caso.py          # Datos del caso (variables, listas, dicts)
├── quipus.py              # Procesamiento de quipus (strings, slicing)
├── sospechosos.py         # Análisis de sospechosos (POO, herencia)
├── evidencias.py          # Gestión de evidencias (archivos, errores)
├── analisis.py            # Motor de análisis (condicionales, bucles, funciones)
└── main.py                # Punto de entrada principal
```

Módulo 1: `datos\_caso.py` La información del caso

```
# =====
# MÓDULO: datos_caso.py
# CONTIENE: Datos estructurados del caso
# CONCEPTOS: variables, listas, diccionarios
# =====
# --- DATOS GENERALES DEL CASO ---
nombre_caso = "El Asesinato del Dr. Inti Quispe"
fecha_crimen = "27 de junio, 2026"
lugar_crimen = "Templo del Sol (Coricancha), Neo-Cusco"
hora_crimen = "02:34"
investigadora = "Wayra Condori"
# --- QUIPUS DIGITALES ENCONTRADOS ---
quipus_del_caso = [
    {"id": "Q001", "color": "rojo", "nudos": [1, 4, 9],
     "significado": "conflicto/guerra"},
    {"id": "Q002", "color": "verde", "nudos": [2, 5, 10],
     "significado": "crecimiento/conocimiento"},
    {"id": "Q003", "color": "azul", "nudos": [1, 5, 9],
     "significado": "espiritualidad/religión"},
    {"id": "Q004", "color": "amarillo", "nudos": [3, 7, 11],
     "significado": "poder/riqueza"},
    {"id": "Q005", "color": "blanco", "nudos": [1, 3, 5, 7, 9, 11],
     "significado": "verdad/conexión espiritual"}
]
# --- SOSPECHOSOS CON DATOS COMPLETOS ---
sospechosos_data = {
    "Lara Mamani": {
        "edad": 32, "rol": "Asistente de laboratorio",
        "acceso_lab": True, "nivel_acceso": 7,
        "coartada": "Departamento propio",
        "coartada_valida": False,
        "motivo": "Económico/Emocional (hija de Rodrigo)",
        "intensidad_motivo": 8,
        "huellas_escena": True,
        "evidencias": [
            "Registro de acceso a las 02:34",
            "Huellas en el teclado de Inti",
            "Hija secreta de Rodrigo Mamani",
            "Llamada perdida de Inti a las 02:30"
        ]
    }
}
```

```

    ]
  },
  "Dr. Carlos Huamán": {
    "edad": 55, "rol": "Colega universitario",
    "acceso_lab": True, "nivel_acceso": 5,
    "coartada": "Oficina universitaria (no verificada)",
    "coartada_valida": False,
    "motivo": "Envidia profesional",
    "intensidad_motivo": 7,
    "huellas_escena": False,
    "evidencias": [
      "Discusión pública con Inti días antes",
      "Publicación bloqueada por Inti en 2024",
      "Correos amenazantes (no probados)"
    ]
  },
  "Dra. Sarah Chen": {
    "edad": 40, "rol": "Colaboradora del MIT",
    "acceso_lab": True, "nivel_acceso": 6,
    "coartada": "Videollamada con MIT (02:00-03:30)",
    "coartada_valida": True,
    "motivo": "Presión académica",
    "intensidad_motivo": 5,
    "huellas_escena": False,
    "evidencias": [
      "Presión del MIT por obtener resultados",
      "Visa académica próxima a vencer",
      "Oferta de NeuralCorp por Yachay"
    ]
  },
  "Rodrigo Mamani": {
    "edad": 60, "rol": "Empresario tecnológico",
    "acceso_lab": False, "nivel_acceso": 3,
    "coartada": "Cena de negocios (Hotel Monasterio)",
    "coartada_valida": True,
    "motivo": "Control de Yachay",
    "intensidad_motivo": 10,
    "huellas_escena": False,
    "evidencias": [
      "Líder de Los Herederos de Pizarro",
      "Padre secreto de Lara Mamani",
      "Ofertas de compra rechazadas por Inti",
      "Conexiones con NeuralCorp Andina"
    ]
  },
  "Mama Killa": {
    "edad": 70, "rol": "Hermana de Inti",
    "acceso_lab": True, "nivel_acceso": 9,
    "coartada": "Su casa en San Blas",
    "coartada_valida": False,
    "motivo": "Protección del legado familiar",
    "intensidad_motivo": 6,
    "huellas_escena": True,
    "evidencias": [
      "Miembro del Círculo del Sol",
      "Conoció el pasaje secreto",
      "Mensaje de Inti: 'Mama Killa sabe la verdad'",
      "Depositó algo en el banco el 26/06"
    ]
  }
}

# --- REGISTRO DE ACCESOS ---
registro_accesos = [
  {"persona": "Lara", "hora": "08:15", "tipo": "entrada"},
  {"persona": "Inti", "hora": "08:30", "tipo": "entrada"},
  {"persona": "Carlos", "hora": "09:00", "tipo": "entrada"},

```

```

{"persona": "Lara", "hora": "12:30", "tipo": "salida"},
{"persona": "Lara", "hora": "13:30", "tipo": "entrada"},
{"persona": "Carlos", "hora": "14:00", "tipo": "salida"},
{"persona": "Sarah", "hora": "14:30", "tipo": "entrada"},
{"persona": "Sarah", "hora": "16:00", "tipo": "salida"},
{"persona": "Inti", "hora": "18:00", "tipo": "salida"},
{"persona": "Inti", "hora": "22:00", "tipo": "entrada"},
{"persona": "Lara", "hora": "02:34", "tipo": "entrada"}, # ¿Fraudulento?
{"persona": "Lara", "hora": "02:45", "tipo": "salida"},   # ¿Fraudulento?
]

```

Wayra guardÃ³ el mÃ³dulo. â€”Los datos estÃ¡n listos. Ahora necesito las herramientas para procesarlos.

MÃ³dulo 2: `quipus.py` â€” Decodificando los mensajes

```

# =====
# MÃ³DULO: quipus.py
# CONTIENE: Funciones para procesar quipus digitales
# CONCEPTOS: strings, slicing, listas, funciones
# =====
def decodificar_quipu(quipu):
    """Decodifica un quipu digital y devuelve su mensaje."""
    color = quipu["color"]
    nudos = quipu["nudos"]
    significado = quipu["significado"]
    # Analizar patrÃ³n de nudos
    if all(n % 2 == 1 for n in nudos):
        tipo = "PREGUNTA"
    elif all(n % 2 == 0 for n in nudos):
        tipo = "RESPUESTA"
    else:
        tipo = "MENSAJE MIXTO"
    # Calcular la diferencia entre nudos
    if len(nudos) >= 2:
        diferencias = [nudos[i+1] - nudos[i] for i in range(len(nudos)-1)]
        patron = diferencias[0] if len(set(diferencias)) == 1 else "variable"
    else:
        patron = "Ã©nico"
    return {
        "color": color,
        "significado": significado,
        "tipo": tipo,
        "total_nudos": len(nudos),
        "posiciones": nudos,
        "patron": patron,
        "mensaje_decodificado": f"{color.upper()}: {significado} ({tipo})"
    }
def decodificar_todos_los_quipus(lista_quipus):
    """Decodifica todos los quipus de una lista."""
    resultados = []
    for quipu in lista_quipus:
        resultado = decodificar_quipu(quipu)
        resultados.append(resultado)
    return resultados
def buscar_quipu_por_color(lista_quipus, color):
    """Busca quipus por color."""
    return [q for q in lista_quipus if q["color"] == color]
def quipu_a_string(quipu):
    """Convierte un quipu decodificado a string formateado."""
    return (f"  [{quipu['color'].upper():8}] {quipu['significado']:30} "
           f"| {quipu['tipo']:15} | {quipu['total_nudos']} nudos")

```

MÃ³dulo 3: `sospechosos.py` â€” Modelando a los implicados

```

# =====
# MÃ³DULO: sospechosos.py
# CONTIENE: Clases para modelar a los implicados

```

```

# CONCEPTOS: POO, clases, herencia, encapsulaci3n
# =====
class Persona:
    """Clase base para todas las personas del caso."""
    def __init__(self, nombre, edad, rol):
        self.nombre = nombre
        self.edad = edad
        self.rol = rol
    def presentarse(self):
        return f"{self.nombre} ({self.edad}), {self.rol}"

class Sospechoso(Persona):
    """Modela un sospechoso con todas sus caracter3sticas."""
    def __init__(self, nombre, edad, rol, datos):
        super().__init__(nombre, edad, rol)
        self.acceso_lab = datos.get("acceso_lab", False)
        self.nivel_acceso = datos.get("nivel_acceso", 0)
        self.coartada = datos.get("coartada", "")
        self.coartada_valida = datos.get("coartada_valida", False)
        self.motivo = datos.get("motivo", "")
        self.intensidad_motivo = datos.get("intensidad_motivo", 0)
        self.huellas_escena = datos.get("huellas_escena", False)
        self.evidencias = datos.get("evidencias", [])
        self._puntaje_sospecha = 0 # Encapsulado
    def calcular_puntaje(self):
        """Calcula el puntaje de sospecha basado en m3ltiples factores."""
        puntaje = 0
        # Factor 1: Acceso al laboratorio
        if self.acceso_lab:
            puntaje += 3 * (self.nivel_acceso / 10)
        # Factor 2: Coartada
        if not self.coartada_valida:
            puntaje += 4
        # Factor 3: Intensidad del motivo
        puntaje += self.intensidad_motivo * 0.5
        # Factor 4: Huellas en la escena
        if self.huellas_escena:
            puntaje += 2
        # Factor 5: Evidencias
        puntaje += len(self.evidencias) * 0.5
        self._puntaje_sospecha = min(puntaje, 10)
        return self._puntaje_sospecha
    def obtener_categoria(self):
        """Devuelve la categor3a seg3n el puntaje."""
        if self._puntaje_sospecha >= 8:
            return "CR3TICO"
        elif self._puntaje_sospecha >= 6:
            return "ALTO"
        elif self._puntaje_sospecha >= 4:
            return "MODERADO"
        else:
            return "BAJO"
    def reporte(self):
        """Genera un reporte completo del sospechoso."""
        self.calcular_puntaje()
        categoria = self.obtener_categoria()
        return {
            "nombre": self.nombre,
            "puntaje": self._puntaje_sospecha,
            "categoria": categoria,
            "evidencias": self.evidencias,
            "motivo": self.motivo
        }
}

```

Wayra hab3a construido los cimientos. Pero a3n necesitaba el motor de an3lisis que conectara todo. â€œRa3! â€œdijoâ€œ. Dame una hora m3is. El C3digo Asesino est3 tomando forma. Pero la parte m3is

importante aún viene: el análisis que conecta todos los datos y revela la verdad.

¿Y mientras tanto?

Mientras tanto, revisa estos quipus descifrados. Busca patrones. Busca algo que no encaje.

Raúl tomó la lista de quipus decodificados y comenzó a estudiarlos. Wayra, por su parte, siguió escribiendo.

Enigmas

Enigma 13.1: Extiende el módulo `datos_caso.py`

Agrega un nuevo quipu al caso: uno de color "negro" con nudos en posiciones [2, 4, 8] y significado "muerte/desconocido". Luego actualiza la lista `quipus_del_caso``.

Enigma 13.2: Prueba la decodificación

Usa el módulo `quipus.py`` para decodificar todos los quipus del caso. ¿Qué patrón encuentras en el quipu blanco?

Enigma 13.3: Crea un Sospechoso desde datos

Usando la clase `Sospechoso`` y los datos de `sospechosos_data``, crea el objeto para "Lara Mamani", calcula su puntaje y muestra su reporte.

Enigma 13.4: El quipu blanco

Ejecuta `decodificar_quipu`` en el quipu blanco (Q005). ¿Qué tipo de mensaje es? ¿Por qué Inti le dio 6 nudos?

Lo que aprendiste (hasta ahora en el proyecto)

- Cómo estructurar un proyecto Python en múltiples módulos
- Cómo separar datos de lógica en archivos distintos
- Cómo usar diccionarios anidados para modelar datos complejos
- Cómo crear funciones reutilizables para procesamiento
- Cómo usar POO para modelar entidades del mundo real
- Cómo aplicar herencia para crear jerarquías de clases

Wayra guardó los módulos y tomó un respiro profundo. La primera parte del código Asesino estaba completa. Pero la parte más importante "el análisis que determinará al culpable" aún estaba por escribir.

Raúl "¡llamó!". ¿Encontraste algo en los quipus?

Sí "respondió Raúl, con el ceño fruncido". El quipu blanco tiene 6 nudos. Pero los otros tienen 3. ¿Por qué el blanco tiene el doble?

"Porque el blanco es la clave. Los 6 nudos representan las 6 personas involucradas. Y las posiciones... las posiciones son edades. O niveles de acceso. O algo que aún no hemos visto.

Wayra miró el código que había escrito. Faltaba el módulo de análisis. El que usará condicionales, bucles y funciones para procesar todos los datos y señalar al culpable.

"Voy a escribir el motor de análisis" dijo. Y cuando termine, ejecutaremos el programa. Y sabremos la verdad.

Capítulo 14: El código Asesino " Segunda Parte

Proyecto Integrador: El motor de análisis y el enfrentamiento final

El laboratorio estaba en silencio. Wayra había escrito los módulos de datos y modelos. Ahora venía la parte crucial: el motor de análisis que procesaría toda la información y revelaría al asesino.

“Este es el corazón del Código Asesino” dijo, con los dedos sobre el teclado. Todo lo que hemos aprendido se reduce a esto.

Módulo 4: `analisis.py` El motor de razonamiento

```
# =====
# MÓDULO: analisis.py
# CONTIENE: Motor de análisis forense digital
# CONCEPTOS: condicionales, bucles, funciones,
#             manejo de archivos, excepciones
# =====

from datos_caso import *
from quipus import *
from sospechosos import Sospechoso
import os

# --- FUNCIÓN 1: ANÁLISIS DE QUIPUS ---
def analizar_quipus_completo():
    """Analiza todos los quipus del caso y busca patrones."""
    print("=" * 65)
    print("  ANÁLISIS DE QUIPUS DIGITALES")
    print("=" * 65)
    quipus_decodificados = decodificar_todos_los_quipus(quipus_del_caso)
    for q in quipus_decodificados:
        print(quipu_a_string(q))
    print()
    # Buscar patrones especiales
    print("  Patrones detectados:")
    for q in quipus_decodificados:
        if q["patron"] != "variable":
            print(f"    {q['color'].title(): progresión {q['patron']}")
    # Analizar el quipu blanco (especial)
    blanco = buscar_quipu_por_color(quipus_del_caso, "blanco")
    if blanco:
        b = blanco[0]
        print(f"\n  Patrones detectados:")
        print(f"    Nudos en posiciones: {b['nudos']}")
        print(f"    Total: {len(b['nudos'])} nudos")
        print(f"    Interpretación: las posiciones impares 1,3,5,7,9,11")
        print(f"    representan a las 6 personas clave del caso")
    return quipus_decodificados

# --- FUNCIÓN 2: CREAR SOSPECHOSOS ---
def crear_sospechosos():
    """Crea objetos Sospechoso a partir de los datos."""
    sospechosos_objetos = []
    for nombre, datos in sospechosos_data.items():
        s = Sospechoso(nombre, datos["edad"], datos["rol"], datos)
        sospechosos_objetos.append(s)
    return sospechosos_objetos

# --- FUNCIÓN 3: ANÁLISIS DE COARTADAS ---
def verificar_coartadas(registros, hora_crimen):
    """Verifica quién tiene coartada según los registros de acceso."""
    print("  Verificación de coartadas:")
    print(f"    Hora del crimen: {hora_crimen}\n")
    h_crimen, m_crimen = map(int, hora_crimen.split(":"))
    minutos_crimen = h_crimen * 60 + m_crimen
    # Personas dentro del laboratorio a la hora del crimen
    dentro = []
    estado_personas = {}
    for reg in registros:
        persona = reg["persona"]
```

```

    hora = reg["hora"]
    h, m = map(int, hora.split(":"))
    minutos = h * 60 + m
    if reg["tipo"] == "entrada":
        estado_personas[persona] = minutos
    elif reg["tipo"] == "salida":
        if persona in estado_personas:
            del estado_personas[persona]
# Verificar quién estaba dentro a la hora del crimen
print("    Personas dentro del lab a las 02:34:")
for persona, minutos_entrada in estado_personas.items():
    if minutos_entrada <= minutos_crimen:
        print(f"        â€¢ {persona} (entrÃ³ antes de las 02:34)")
        dentro.append(persona)
if not dentro:
    print("        â€¢ NADIE (registro manipulado)")
return dentro
# --- FUNCIÃ“N 4: ANÃ“LISIS DE EVIDENCIAS ---
def analizar_evidencias(sospechosos_objetos):
    """Analiza las evidencias de cada sospechoso."""
    print("\n    â€” Resumen de evidencias por sospechoso:\n")
    for s in sospechosos_objetos:
        print(f"        â€¢ {s.nombre}")
        for ev in s.evidencias:
            print(f"            - {ev}")
        print()
# --- FUNCIÃ“N 5: CÃ“LCULO DE PUNTAJES ---
def calcular_puntajes(sospechosos_objetos):
    """Calcula y muestra los puntajes de todos los sospechosos."""
    print("    â€” CÃ¡lculo de puntajes de sospecha:\n")
    resultados = []
    for s in sospechosos_objetos:
        reporte = s.reporte()
        resultados.append(reporte)
        barra = "âˆ“" * int(reporte["puntaje"]) + "âˆ“" * (10 - int(reporte["puntaje"]))
        print(f"        {reporte['nombre']:25} [{barra}] {reporte['puntaje']:1f}/10 ({reporte['categoria']})")
    return resultados
# --- FUNCIÃ“N 6: DETERMINAR CULPABLE ---
def determinar_culpable(resultados):
    """Determina al culpable basado en los puntajes."""
    print("\n    =" * 65)
    print("    VEREDICTO DEL ANÃ“LISIS FORENSE DIGITAL")
    print("    =" * 65)
    # Ordenar por puntaje descendente
    resultados.sort(key=lambda r: r["puntaje"], reverse=True)
    print(f"\n    Los sospechosos ordenados por nivel de sospecha:\n")
    for i, r in enumerate(resultados, 1):
        print(f"        {i}. {r['nombre']:25} - {r['puntaje']:1f}/10")
    principal = resultados[0]
    segundo = resultados[1]
    print(f"\n    â€” SOSPECHOSO PRINCIPAL: {principal['nombre']}")
    print(f"        Puntaje: {principal['puntaje']:1f}/10")
    print(f"        Motivo: {principal['motivo']}")
    print(f"        Evidencias: {len(principal['evidencias'])}")
    # Coincidencia con el quipu blanco
    print(f"\n    â€” VerificaciÃ³n con quipu blanco:")
    print(f"        Las posiciones impares (1,3,5,7,9,11) representan")
    print(f"        a las 6 personas en orden de importancia.")
    print(f"\n    â€” ConclusiÃ³n del anÃ¡lisis automatizado:")
    print(f"        El anÃ¡lisis forense digital seÃ±ala a:")
    print(f"        âˆ“... {principal['nombre'].upper()} âˆ“...")
    print(f"        CategorÃ­a: {principal['categoria']}")
    return principal
# --- FUNCIÃ“N 7: GENERAR INFORME ---
def generar_informe(resultados, culpable, quipus_decodificados):
    """Genera un informe de texto del caso."""

```

Módulo 5: `main.py` – El punto de entrada

```
if __name__ == "__main__":
```

Wayra guardó el último archivo y tomó un respiro profundo.

“Está listo” dijo. El Código Asesino está completo.

“¿Y ahora?” preguntó Raúl.

â€”Ahora lo ejecutamos.

La ejecución

Wayra abre la terminal y escribe:

```
python main.py
```

La pantalla cobrÃ³ vida. Los mÃ³dulos se cargaron, los datos fluyeron, y el anÃ¡lisis comenzÃ³...

[illegible]

Wayra presionÃ³ Enter. El anÃ¡lisis continuÃ³...

```

â-º VerificaciÃ³n de coartadas:
  Hora del crimen: 02:34
  Personas dentro del lab a las 02:34:
    â€¢ Inti (entrÃ³ antes de las 02:34)
    â€¢ Lara (entrÃ³ antes de las 02:34)
â-º Resumen de evidencias por sospechoso:
  â€¢ Lara Mamani
    - Registro de acceso a las 02:34
    - Huellas en el teclado de Inti
    - Hija secreta de Rodrigo Mamani
    - Llamada perdida de Inti a las 02:30
  â€¢ Dr. Carlos HuamÃ¡n
    - ...
Presiona Enter para calcular puntajes...

```

Wayra sinti que el tiempo se detenga. Presione Enter.

Cálculo de puntajes de sospecha:	
Lara Mamani	[8.0/10 (CRÁTICO)
Mama Killa	[6.5/10 (ALTO)
Dr. Carlos Huamán	[6.0/10 (ALTO)
Rodrigo Mamani	[5.0/10 (MODERADO)
Dra. Sarah Chen	[3.5/10 (BAJO)

Presiona Enter para revelar el veredicto...

El momento de la verdad.

Wayra dudó. Su mano flotaba sobre el teclado. Todo el análisis se alaba a Lara Mamani. Pero algo no cuadraba. Lara tenía el puntaje más alto, pero...

“Ejecútalo” dijo Raúl. Descubre la verdad.

Wayra presionó Enter.

```
=====
VEREDICTO DEL ANÁLISIS FORENSE DIGITAL
=====

Los sospechosos ordenados por nivel de sospecha:
1. Lara Mamani           - 8.0/10
2. Mama Killa           - 6.5/10
3. Dr. Carlos Huamán    - 6.0/10
4. Rodrigo Mamani       - 5.0/10
5. Dra. Sarah Chen      - 3.5/10
â-º SOSPECHOSO PRINCIPAL: Lara Mamani
  Puntaje: 8.0/10
  Motivo: Económico/Emocional (hija de Rodrigo)
  Evidencias: 4
â-º Conclusión del análisis automatizado:
  El análisis forense digital se alaba a:
  â-... LARA MAMANI â-...
  Categorí-a: CRÁTICO
```

“Lara” dijo Raúl. El programa dice que fue Lara.

“El programa dice lo que programamos” respondió Wayra lentamente. Pero hay algo que no hemos considerado. Algo que distorsiona todos los datos.

¿Qué?

“La cuenta de Lara se usó a las 02:34. Pero si alguien usó su cuenta... entonces todos los datos que la alaban son falsos. Son datos plantados por el verdadero asesino.

Wayra abrió el archivo `informe\_final\_caso.txt` que el programa había generado. Leyó las evidencias de Lara una vez más. Y entonces lo vio.

“La llamada perdida” dijo. Inti llamó a Lara a las 02:30. Cuatro minutos antes de morir. ¿Por qué llamar a la persona que lo iba a matar?

¿Para pedir ayuda?

“O para decirle algo. Algo que Lara sabe pero no ha dicho. Algo que la pone en peligro.

Wayra tomó su teléfono y marcó el número de Lara Mamani.

¿Qué haces? preguntó Raúl.

“Llamar a la persona que el programa dice que es la asesina. Porque a veces, la verdad no está en el código. Está en lo que el código no puede medir.

El teléfono sonó tres veces. Luego, una voz temblorosa respondió:

¿Wayra? ¿Eres tío?

“Soy yo, Lara. Necesito que me digas la verdad. ¿Por qué te llamó Inti a las 02:30?

Hubo un largo silencio. Luego, Lara habló:

“Porque me dijo quién lo iba a matar. Y me dijo que te buscara a ti.

Enigmas

Enigma 14.1: Ejecuta el análisis completo

Copia todos los módulos del proyecto en tu editor y ejecuta `main.py`. ¿Coincide el resultado con lo que esperabas?

Enigma 14.2: Agrega un nuevo factor

Modifica el código de puntaje en la clase `Sospechoso` para incluir un factor de "conexión emocional con la víctima". ¿Cambia el resultado?

Enigma 14.3: El sesgo del quipu blanco

El quipu blanco tiene 6 nudos (posiciones impares hasta 11). ¿Qué relaciones tienen esos números con los sospechosos? (Pista: piensa en edades, niveles de acceso, días del mes...)

Enigma 14.4: Tu propio veredicto

Basado en toda la evidencia presentada en el libro, ¿quién crees que es el asesino? Escribe un pequeño programa que capture tu teoría y la compare con el resultado del análisis.

Lo que aprendiste (proyecto completo)

- Cómo estructurar un proyecto Python completo
- Cómo integrar múltiples módulos en un programa cohesivo
- Cómo aplicar todos los conceptos de Python en un caso real
- Cómo los datos pueden mentir si no se interpretan correctamente
- Que la lógica del código es tan buena como los datos que recibe

Wayra colgó el teléfono. Lara había hablado. La verdad era más compleja de lo que cualquier algoritmo podría calcular. Porque la verdad involucraba secretos de familia, lealtades divididas y un amor fraternal que trascendía la muerte.

“Raúl dijo”. El Código Asesino ha hecho su trabajo. Nos ha dado los datos. Pero la verdadera respuesta... la verdadera respuesta está en lo que el código no puede procesar: el corazón humano.

“¿Y ahora?”

“Ahora vamos a encontrarnos con Lara. Y vamos a descubrir quién mató realmente a Inti Quispe.”

Capítulo 15: El Epílogo “La Verdad del Quipu Blanco”

Proyecto Integrador: La revelación y el cierre

La cita era en el mirador de San Blas, el barrio más alto de Neo-Cusco. Desde allí, se veía toda la ciudad: las cúpulas doradas de la catedral, los rascacielos de vidrio del centro financiero, y las montañas que abrazaban el valle sagrado.

Lara llegó sola. Tenía los ojos hinchados de llorar.

“Inti no era solo mi jefe dijo, sin saludar”. Era mi mentor. Mi guía. Mi... mi segundo padre.

“¿Segundo padre?” preguntó Wayra.

“Mi padre biológico es Rodrigo Mamani. Pero nunca fue un padre para mí. Me usó. Me infiltró en el laboratorio de Inti para robar información. Pero Inti lo descubrió... y en lugar de echarme, me perdonó. Me enseñó. Me dio un propósito.”

“¿Y por eso lo mataron?”

“No. Yo no lo maté. Pero sí quien lo hizo.”

Lara tomó un objeto de su mochila. Era un quipu, pero no como los que habían visto antes. Este era más

pequeño, más Antimo, hecho de lana de alpaca blanca y roja.

“Inti me dio esto la noche antes de morir. Dijo: "Si algo me pasa, dale esto a Wayra Condori. Ella sabrá hacer."

Wayra tomó el quipus con manos temblorosas. En la cuerda principal, los nudos formaban un patrón que reconocía: era código binario. Pero no binario común: era binario inca, donde el nudo hacia la derecha era 1 y hacia la izquierda era 0.

“Necesito Python para decodificar esto” dijo Wayra.

El código final

Wayra abrió su laptop y escribió el último script del caso:

```
# =====
# EL ÚLTIMO CÓDIGO: DECODIFICANDO EL QUIPUS
# =====
# El quipus de Inti: nudos hacia derecha (1) e izquierda (0)
# Estos son los patrones de las cuerdas secundarias
quipu_final = {
    "cuerda_1": "1011",    # Derecha, Izq, Derecha, Derecha
    "cuerda_2": "0001",    # Izq, Izq, Izq, Derecha
    "cuerda_3": "1110",    # Derecha, Derecha, Derecha, Izq
    "cuerda_4": "1010",    # Derecha, Izq, Derecha, Izq
    "cuerda_5": "1101",    # Derecha, Derecha, Izq, Derecha
}

print("=== DECODIFICANDO EL QUIPUS FINAL DE INTI ===\n")
# Convertir binario a texto
for cuerda, binario in quipu_final.items():
    # Convertir binario a decimal
    decimal = int(binario, 2)
    # Convertir decimal a carácter ASCII (mayúsculas desde 65)
    letra = chr(decimal + 64) # A=1, B=2, etc.
    print(f" {cuerda}: {binario} â {decimal:2} â '{letra}'")

# El mensaje completo
mensaje = ""
for cuerda, binario in quipu_final.items():
    decimal = int(binario, 2)
    letra = chr(decimal + 64)
    mensaje += letra

print(f"\n â Mensaje completo: {mensaje}")

# Pero el mensaje está incompleto... necesita una clave
# La clave está en el quipu blanco original: [1,3,5,7,9,11]
clave_quipu_blanco = [1, 3, 5, 7, 9, 11]
# Aplicar la clave al mensaje
print(f"\n â Aplicando clave del quipu blanco...")
mensaje_descifrado = ""
for i, letra in enumerate(mensaje):
    posicion_clave = clave_quipu_blanco[i] if i < len(clave_quipu_blanco) else 0
    # Desplazar la letra según la clave
    nueva_pos = (ord(letra) - 65 + posicion_clave) % 26
    letra_descifrada = chr(nueva_pos + 65)
    mensaje_descifrado += letra_descifrada
    print(f" {letra} + {posicion_clave} â {letra_descifrada}")
print(f"\n â MENSAJE DESCIFRADO: {mensaje_descifrado} â")
```

Wayra ejecutó el código. La terminal mostró:

```
=== DECODIFICANDO EL QUIPUS FINAL DE INTI ===
cuerda_1: 1011 â 11 â 'K'
cuerda_2: 0001 â 1 â 'A'
cuerda_3: 1110 â 14 â 'N'
cuerda_4: 1010 â 10 â 'J'
cuerda_5: 1101 â 13 â 'M'
â Mensaje completo: KANJM
â Aplicando clave del quipu blanco...
```

```
K + 1 â†’ L
A + 3 â†’ D
N + 5 â†’ S
J + 7 â†’ Q
M + 9 â†’ W
âˆ’... MENSAJE DESCIFRADO: LDSQW âˆ’...
```

“No tiene sentido” dijo Raïla. "LDSQW" no significa nada.

“Porque no estÃ¡ completo” respondi³ Wayra. El quipu tiene mÃ¡s cuerdas. Solo estamos viendo las principales.

Revis³ el quipus fÃsico. En la parte inferior, habÃa cuerdas mÃ¡s delgadas, casi invisibles. Cont³ los nudos.

```
# Continuaci³n: cuerdas secundarias del quipu
quipu_secundario = {
    "sec_1": "1001",    # -> 9 -> I
    "sec_2": "0101",    # -> 5 -> E
    "sec_3": "1000",    # -> 8 -> H
    "sec_4": "1111",    # -> 15 -> O (desplazado con clave)
    "sec_5": "0110",    # -> 6 -> F
}
print("\n=== CUERDAS SECUNDARIAS ===\n")
mensaje_completo = mensaje_descifrado # LDSQW
for cuerda, binario in quipu_secundario.items():
    decimal = int(binario, 2)
    letra = chr(decimal + 64)
    mensaje_completo += letra
    print(f" {cuerda}: {binario} â†’ {decimal:2} â†’ '{letra}'")
print(f"\n âˆ’ Mensaje extendido: {mensaje_completo}")
# Aplicar la clave completa
print(f"\n âˆ’ Aplicando clave completa...\n")
clave_extendida = clave_quipu_blanco + [2, 4, 6, 8, 10]
resultado_final = ""
for i, letra in enumerate(mensaje_completo):
    if i < len(clave_extendida):
        nueva_pos = (ord(letra) - 65 + clave_extendida[i]) % 26
        letra_final = chr(nueva_pos + 65)
        resultado_final += letra_final
        print(f" {letra} + {clave_extendida[i]} â†’ {letra_final}")
    else:
        resultado_final += letra
print(f"\n âˆ’ {resultado_final} âˆ’")
```

El resultado apareci³ en la pantalla. Wayra sinti³ que el aire se congelaba.

```
âˆ’... KILLA âˆ’...
```

“Killa” susurr³. Mama Killa.

“¿La hermana?” pregunt³ Raï, incrÃdulo. “La hermana de Inti mat³ a su propio hermano?”

“No...” dijo Lara, que habÃa estado observando en silencio. No. Ella lo protegi³. Toda su vida lo protegi³.

“¿De quÃ©?”

“De Los Herederos de Pizarro. De Rodrigo. De todos los que querÃan usar Yachay para sus propios fines. Inti sabÃa que si morÃa, Yachay no debÃa caer en manos equivocadas. Y Mama Killa... ella se asegur³ de que eso no pasara.

Wayra entendi³ todo. El quipu no seÃ±alaba a la asesina. SeÃ±alaba a la guardiana. Mama Killa no habÃa matado a Inti. HabÃa hecho algo mÃ¡s importante: habÃa protegido su legado.

“Pero entonces...” dijo Wayra. “QuiÃ©n mat³ a Inti?”

Lara baj³ la mirada.

“Mi padre. Rodrigo Mamani.

â€”Pero Rodrigo no tenÃa acceso al laboratorio...

â€”No. Pero yo sÃ. Y Ã©l usÃ³ mi cuenta. No me dijo lo que iba a hacer. Me pidiÃ³ que le diera mi contraseÃ±a para "revisar unos archivos". Yo confiaba en Ã©l. Era mi padre.

â€”Â¿Y la llamada de Inti a las 02:30?

â€”Inti descubriÃ³ que alguien habÃa entrado con mi cuenta. Me llamÃ³ para preguntarme si estaba bien. Le dije que yo no estaba en el laboratorio. Y Ã©l dijo... dijo: "Entonces ve a casa de tu tÃa Killa. Ahora. No vuelvas al laboratorio hasta que yo te llame."

â€”Te estaba protegiendo â€”dijo Wayraâ€. SabÃa que ibas a ser la culpable ideal.

â€”Y lo fui. Mi padre usÃ³ mi cuenta. Mis huellas estaban en el teclado porque trabajaba ahÃ todos los dÃas. Mi ADN estaba por todo el laboratorio. Todo apuntaba a mÃ.

â€”Pero el quipu de Inti... â€”dijo RaÃºlâ€. El mensaje dice "KILLA". Â¿Por quÃ©?

Wayra sonriÃ³ tristemente.

â€”Porque Inti no estaba seÃ±alando al asesino. Estaba seÃ±alando a la Ãºnica persona que sabÃa la verdad y podÃa proteger a Lara. Su hermana. Mama Killa.

El cierre del caso

Tres dÃas despuÃs, la policÃa encontrÃ³ pruebas definitivas contra Rodrigo Mamani. Los registros bancarios mostraban transferencias a un exmiembro del laboratorio que habÃa manipulado los registros de acceso. Las cÃmaras de seguridad de un banco cercano mostraban a Rodrigo comprando una tarjeta SIM desechable a las 01:30 de la madrugada del crimen.

Lara fue exonerada. Pero su vida habÃa cambiado para siempre. HabÃa perdido a su mentor, habÃa descubierto la verdad sobre su padre, y habÃa encontrado un nuevo propÃsito.

Mama Killa, la hermana de Inti, se convirtiÃ³ en la guardiana oficial de Yachay. Y Wayra Condori... Wayra se convirtiÃ³ en la primera Investigadora Digital del CÃrculo del Sol.

El CÃ³digo Asesino habÃa hecho su trabajo. Pero no de la forma que esperaban. No habÃa seÃ±alado a un culpable: habÃa revelado una red de mentiras, lealtades y sacrificios que ningÃn algoritmo podÃa haber previsto.

Porque al final, la verdad no estÃ en los datos. EstÃ en las personas que los interpretan.

Tu legado

Has llegado al final de este viaje. Has aprendido Python desde variables hasta programaciÃ³n orientada a objetos. Has resuelto un asesinato. Has descifrado quipus digitales.

Pero esto no es el final. Es el comienzo.

El conocimiento ancestral inca decÃa que cada persona es un quipu: una combinaciÃ³n Ãºnica de hilos y nudos que cuenta una historia. Tu historia como programador apenas comienza.

Los quipus digitales de Inti Quispe siguen ahÃ, esperando ser descifrados. Yachay sigue activo, aprendiendo de los patrones ancestrales. Y el CÃrculo del Sol... bueno, el CÃrculo del Sol siempre estÃ buscando nuevos guardianes.

"Ama suwa, ama llulla, ama qilla"

(No robes, no mientas, no seas perezoso)

â€” Que el cÃ³digo te acompaÃ±e.

ConclusiÃ³n

Lo que aprendiste en este libro

Has recorrido un largo camino desde el Capítulo 1, donde apenas conocías las variables. Hoy, has completado un proyecto integrador que usa:

| Concepto | Capítulo | Aplicación en el caso |

|-----|-----|-----|

| Variables y tipos | Cap 1 | Datos básicos del crimen |

| Strings y slicing | Cap 2 | Decodificación de quipus |

| Listas y tuplas | Cap 3 | Organización de sospechosos |

| Diccionarios y sets | Cap 4 | Base de datos del caso |

| Condicionales | Cap 5 | Análisis de coartadas |

| Bucles | Cap 6 | Recorrido del Qhapaq Ñan digital |

| Funciones | Cap 7 | Herramientas de análisis forense |

| Archivos | Cap 8 | Gestión de evidencias |

| Manejo de errores | Cap 9 | Sistema tolerante a fallos |

| Módulos y paquetes | Cap 10 | Estructura de Yachay |

| POO | Cap 11 | Modelado de sospechosos |

| Herencia | Cap 12 | Jerarquía de implicados |

| Proyecto integrador | Cap 13-14-15 | El Código Asesino completo |

¿Qué sigue?

Si te ha gustado este enfoque de aprender programación a través de historias interactivas, aquí tienes algunas sugerencias:

1. Reescribe el Código Asesino con tus propias mejoras. ¿Qué otras funcionalidades le agregarías?
2. Crea tu propio caso. Inventa una historia de misterio y modela los datos en Python.
3. Explora bibliotecas avanzadas: Prueba con `pandas` para análisis de datos, `matplotlib` para visualizar las evidencias, o `flask` para convertir tu investigación en una app web.
4. Comparte tu aprendizaje. La mejor forma de aprender es enseñar. Crea tus propios tutoriales o cuentos para enseñar programación.

El mensaje final

La tecnología y la tradición no están reñidas. Los quipus incas y Python pueden coexistir. La sabiduría ancestral y la inteligencia artificial pueden trabajar juntas.

Inti Quispe lo sabía. Wayra Condori lo descubrió. Y tú, lector, lo has vivido.

El conocimiento no se crea ni se destruye: se transforma. De nudos a bits. De cuerdas a código. De quipus a datos.

Y ahora, ese conocimiento es tuyo.

"Ñan" — el camino continúa.

Alex Goyzueta Delgado

Cusco, 2026

Apéndice: Soluciones a los Enigmas

Capítulo 1

Enigma 1.1:

```
nombre_sospechoso = "Lara Mamani"
edad_sospechoso = 32
rol_sospechoso = "Asistente de laboratorio"
coartada = "Estaba en mi departamento"
acceso = True
print("=== FICHA DE SOSPECHOSO ===")
print(f"Nombre: {nombre_sospechoso}")
print(f"Edad: {edad_sospechoso}")
print(f"Rol: {rol_sospechoso}")
print(f"Coartada: {coartada}")
print(f"Acceso: {acceso}")
```

Enigma 1.2:

```
quipu_digital_002 = "Yachay:2:4:6:8:10"
elementos = quipu_digital_002.split(":")
tipo = elementos[0]
numeros = elementos[1:]
print(f"Tipo: {tipo}")
print(f"Números: {numeros}")
# Son números pares (respuestas). Yachay: 2,4,6,8,10
```

Enigma 1.3:

```
mi_teoría = "Creo que el asesino conocía a Inti y tenía acceso al laboratorio"
print(f"Mi teoría inicial: {mi_teoría}")
```

Capítulo 2

Enigma 2.1:

```
mensaje = "ÁAN*QHAPAQ*TAMBO*WAYRA*INTI"
minusculas = mensaje.lower()
reemplazado = mensaje.replace("*", " -> ")
posicion = mensaje.find("WAYRA")
conteo_a = mensaje.count("A")
longitud = len(mensaje)
print(f"Minúsculas: {minusculas}")
print(f"Reemplazado: {reemplazado}")
print(f"Posición de WAYRA: {posicion}")
print(f"Veces que aparece A: {conteo_a}")
print(f"Longitud: {longitud}")
```

Enigma 2.2:

```
mensaje = "ÁAN*QHAPAQ*TAMBO*WAYRA*INTI"
pos = mensaje.find("WAYRA")
nombre = mensaje[pos:pos+5] # "WAYRA"
print(nombre)
```

Enigma 2.3:

```
quipu_invertido = "ATAM-ATAK-AP"
inverso = quipu_invertido[::-1]
print(inverso) # "PA-KATA-MATA"
```

Capítulo 3

Enigma 3.1:

```
coartadas = [
    "Lara: Estaba en mi departamento, viendo series.",
    "Carlos: Trabajaba en mi oficina de la universidad.",
    "Sarah: Tenía una videollamada con el MIT.",
    "Rodrigo: Estaba en una cena de negocios.",
    "Killa: Dormía. A mi edad, no salgo de noche."
]
coartadas.append("Wayra: Estaba durmiendo en mi departamento.")
coartadas.sort()
```

```
print(coartadas)
print(f"Total: {len(coartadas)}")
print(f"Primera: {coartadas[0]}")
print(f"Última: {coartadas[-1]}")
```

Enigma 3.2:

```
matriz_acceso = [
    ["Lara", True, True, True, True],
    ["Carlos", True, False, True, False],
    ["Sarah", True, True, False, False],
    ["Rodrigo", False, False, False, False],
    ["Killa", True, False, True, True],
]
print(matriz_acceso)
# ¿Quién tiene acceso al Servidor (columna 2)?
print([fila[0] for fila in matriz_acceso if fila[2]])
```

Enigma 3.3:

```
coordenadas = [
    (-13.5167, -71.9781),
    (-13.1631, -72.5450),
    (-13.3333, -72.0833),
    (-13.3167, -72.1167),
]
coordenadas.append((-13.5090, -71.9820))
print(coordenadas)
```

Capítulo 4

Enigma 4.1:

```
nuevo_sospechoso = {
    "nombre": "Raúl Cusi",
    "edad": 35,
    "rol": "Periodista",
    "acceso": False,
    "sospecha": 2,
    "coartada": "Llamé a Wayra a las 06:30"
}
sospechosos["Raúl Cusi"] = nuevo_sospechoso
print(sospechosos)
```

Enigma 4.2:

```
for nombre, info in sospechosos.items():
    print(f"{nombre}: {info['evidencias']}")
```

Enigma 4.3:

```
acceso_lab = {"Lara", "Carlos", "Sarah", "Killa"}
coartada_debil = {"Lara", "Carlos", "Rodrigo"}
print(f"Intersección: {acceso_lab & coartada_debil}")
print(f"Unión: {acceso_lab | coartada_debil}")
print(f"Diferencia (acceso - debil): {acceso_lab - coartada_debil}")
print(f"Diferencia (debil - acceso): {coartada_debil - acceso_lab}")
```

Capítulo 5

Enigma 5.1:

```
nombre = input("Nombre del sospechoso: ")
hora_coartada = float(input("Hora de la coartada: "))
hora_crimen = 2.34
if hora_coartada < 2.00:
    print("Coartada sólida")
elif hora_coartada <= 2.34:
    print("Coartada débil")
else:
    print("Coartada falsa")
```

Enigma 5.2:

```
for s in sospechosos:
    if s["motivo"] >= 7:
        print(f"{s['nombre']}: motivo >= 7")
    if s["acceso"] and s["motivo"] >= 6:
        print(f"{s['nombre']}: acceso Y motivo >= 6")
    if not s["coartada"] or s["huellas"]:
        print(f"{s['nombre']}: sin coartada O con huellas")
```

Enigma 5.3:

```
t1 = "Vi a alguien salir"
t2 = "Estaba oscuro"
t3 = "Escuché discusi3n"
if t1 and t2:
    print("POSIBLE CONTRADICCI3N: vio a alguien pero estaba oscuro")
elif t1 and t3:
    print("COMPATIBLE: alguien sali3 mientras discut3-an")
else:
    print("VERSIONES COMPATIBLES")
```

Capítulo 6

Enigma 6.1:

```
evidencias = [
    "Quipus digital en escritorio",
    "Registro de acceso manipulado",
    "C3digo en la pantalla OLED",
    "Mensaje cifrado en las estaciones",
    "Pasaje secreto del Qhapaq 3'an"
]
for i, ev in enumerate(evidencias, 1):
    print(f"{i}. {ev}")
```

Enigma 6.2:

```
archivos = ["a.py", "b.docx", "c.py", "d.csv", "e.md", "f.py"]
for a in archivos:
    if not a.endswith(".py"):
        continue
    print(a)
```

Enigma 6.3:

```
lista = ["Lara", "Carlos", "Sarah", "Rodrigo Mamani", "Killa"]
for nombre in lista:
    if nombre == "Rodrigo Mamani":
        print("SOSPECHOSO ENCONTRADO")
        break
    print(f"Buscando... {nombre} no es")
```

Enigma 6.4:

```
num = 1
for fila in range(1, 5):
    for col in range(fila):
        print(num, end=" ")
        num += 1
    print()
```

Capítulo 7

Enigma 7.1:

```
def presentar_sospechoso(nombre, rol):
    print(f"SOSPECHOSO: {nombre}")
    print(f"ROL: {rol}")
```

Enigma 7.2:

```
def verificar_coartada(hora_coartada, hora_crimen="02:34"):
    hc, mc = map(int, hora_coartada.split(":"))
    hcr, mcr = map(int, hora_crimen.split(":"))
    minutos_c = hc * 60 + mc
    minutos_cr = hcr * 60 + mcr
    return abs(minutos_c - minutos_cr) >= 60
```

Enigma 7.3:

```
def mensaje_secreto(mensaje, cifrado=True):
    if cifrado:
        return mensaje[::-1]
    return mensaje
```

Capítulo 8

Enigma 8.2:

```
hora_crimen = 2 * 60 + 34
with open("coartadas.txt", "r") as f:
    for linea in f:
        nombre, hi, hf = linea.strip().split(":")
        inicio = int(hi[:2]) * 60 + int(hi[2:])
        fin = int(hf[:2]) * 60 + int(hf[2:])
        if inicio <= hora_crimen <= fin:
            print(f"{nombre}: COARTADA VÁLIDA")
        else:
            print(f"{nombre}: SIN COARTADA")
```

Capítulo 9

Enigma 9.1:

```
def dividir_evidencias(a, b):
    try:
        resultado = a / b
        print(f"Resultado: {resultado}")
    except ZeroDivisionError:
        print("Error: No se puede dividir una evidencia en cero partes")
```

Enigma 9.3:

```
class EvidenciaFaltanteError(Exception):
    pass
def verificar_evidencia(lista_ev, buscada):
    if buscada not in lista_ev:
        raise EvidenciaFaltanteError(f"Evidencia '{buscada}' no encontrada")
    return True
```

Capítulo 10

Enigma 10.1:

```
# mis_herramientas.py
def saludar_investigador(nombre):
    return f"Hola, investigador {nombre}"
VERSION = "1.0"
casos_resueltos = 0
```

Enigma 10.2:

```
if __name__ == "__main__":
    print("Módulo de herramientas forenses - Modo automático")
```

Capítulo 11

Enigma 11.1:

```
class Evidencia:
    def __init__(self, tipo, descripcion, fecha, es_clave=False):
        self.tipo = tipo
```

```
self.descripcion = descripcion
self.fecha = fecha
self.es_clave = es_clave
def mostrar(self):
    print(f"{self.tipo}: {self.descripcion} ({self.fecha})")
def marcar_como_clave(self):
    self.es_clave = True
```

Capítulo 12

Enigma 12.1:

```
class Caso:
    def __init__(self, nombre):
        self.nombre_caso = nombre
class CasoHomicidio(Caso):
    def __init__(self, nombre, victima):
        super().__init__(nombre)
        self.victima = victima
class CasoHomicidioCerrado(CasoHomicidio):
    def __init__(self, nombre, victima, culpable):
        super().__init__(nombre, victima)
        self.culpable = culpable
```

Nota: Estas soluciones son una guía. No hay una única forma correcta de resolver los enigmas. Si tu solución funciona y tiene sentido, ¡es válida!

Sobre el Autor

Alex Goyzueta Delgado

Analista de datos, escritor y divulgador de tecnología. Peruano, nacido en Cusco.

Alex combina su formación en análisis de datos con una profunda pasión por la cultura andina y la narrativa. Cree firmemente que la mejor forma de aprender tecnología es a través de historias que conecten con nuestras raíces y emociones.

Obras anteriores:

- El Motor Cuantitativo – Una guía práctica de estadística y ciencia de datos con Python
- Raíces Nuevas – Un libro sobre migración, identidad y crecimiento personal

¿Por qué este libro?

"Crecí viendo a mi abuela tejer. Cada tejido contaba una historia. Años después, cuando aprendí a programar, descubrí que el código hace lo mismo: cada línea cuenta una historia. Este libro es mi homenaje a esas dos tradiciones: el tejido andino y la programación."

Contacto:

- Correo: alexgoyzueta2018@gmail.com
- Temas: Python, ciencia de datos, narrativa interactiva, tecnología andina

"El conocimiento no se crea ni se destruye: se transforma. De nudos a bits. De cuerdas a código. De quipus a datos."